Java Interview Crash Course

Core Java → Full Stack in 4 days

45 topics · 363 flashcards · 207 quiz questions · 164 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Contents

Day 1 — Core Java: The Language, Again

1. How Java Actually Runs (JDK / JRE / JVM) Must-know · 20 min
2. Data Types, Variables & Wrappers Must-know · 35 min
3. Operators & Precedence Core · 25 min
4. Control Flow: switch, loops, break/continue Core · 30 min
5. Arrays Core · 30 min
6. Strings, Immutability & the String Pool Must-know · 45 min
7. Input / Output & Formatting Core · 25 min
8. Stack, Heap & Pass-by-Value Must-know · 30 min
9. Packages, Access Modifiers & the static Keyword Core · 30 min
10. Varargs, Enums & Command-Line Args Nice-to-have · 20 min

Day 2 — OOP, Collections, Exceptions & Generics

1. The 4 Pillars of OOP (with real examples) Must-know · 35 min
2. Constructors, this & super Core · 30 min
3. Overloading vs Overriding & Dynamic Dispatch Must-know · 40 min
4. Abstract Class vs Interface Must-know · 40 min
5. Object Methods & the equals/hashCode Contract Must-know · 40 min
6. Exception Handling Must-know · 45 min
7. Collections Framework: List & Set Must-know · 45 min
8. Map & HashMap Internals Must-know · 50 min
9. Iterators, Comparable vs Comparator Core · 30 min
10. Generics & Wildcards Core · 40 min

Day 3 — Modern Java: Streams, Concurrency, JDBC & Patterns

1. Lambdas & Method References Must-know · 35 min
2. Streams API Must-know · 60 min
3. Optional & the Date/Time API Must-know · 35 min
4. Modern Java: 9 → 17 Features Edge · 35 min
5. Threads: Creation, Lifecycle & Basics Must-know · 45 min
6. Synchronization, volatile & Deadlock Must-know · 50 min
7. ExecutorService & Concurrent Collections Must-know · 45 min
8. File I/O & Serialization Core · 30 min
9. JDBC — Connecting Java to a Database Must-know · 45 min
10. Design Patterns You Must Know Must-know · 40 min

Day 4 — Full Stack: Spring Boot, JPA, REST & SQL

1. Spring Core: IoC, DI & Beans Must-know · 45 min
2. Spring Boot: Auto-configuration, Starters & Config Must-know · 40 min
3. Spring MVC & REST Controllers Must-know · 50 min
4. JPA & Hibernate Must-know · 60 min
5. SQL Essentials for Interviews Must-know · 45 min
6. Maven, Build & Project Structure Core · 25 min
7. Spring Security & JWT Edge · 35 min

8. Testing: JUnit 5 & Mockito Core · 35 min
9. Microservices & Architecture Overview Edge · 30 min
10. Explaining Your Project (the round that decides it) Must-know · 40 min

Rapid Fire — Rapid Fire: Traps, HR & Last-Minute Drill

1. Output Prediction Traps (Part 1: Strings & Numbers) Trap · 25 min
2. Output Prediction Traps (Part 2: OOP & Exceptions) Trap · 25 min
3. One-Minute Answers (Verbal Rapid Fire) Must-know · 30 min
4. HR & Behavioural Questions Must-know · 30 min
5. Final 24 Hours: What To Do & Skip Plan · 15 min

Day 1: Core Java: The Language, Again

Syntax, types, control flow, arrays, strings, I/O

10 topics · 67 flashcards · 37 quiz questions · ~4.8 hours

> **Goal.** Shake off the rust. By tonight you can write and dry-run any small Java program on paper — which is exactly what a whiteboard round asks for.

Schedule

- Slot 1 (2h): How Java runs + program structure + data types
- Slot 2 (2h): Operators + control flow + switch/loops
- Slot 3 (2h): Arrays + Strings + StringBuilder + String pool
- Slot 4 (1.5h): I/O + stack/heap + pass-by-value
- Slot 5 (1.5h): Flashcards → Quiz → write 5 programs by hand

1. How Java Actually Runs (JDK / JRE / JVM) Must-know 20 min

This is question #1 in a scary number of fresher interviews, and it's free marks if you can draw it. You write `.java` source. `javac` (the compiler, part of the JDK) turns it into **bytecode** in a `.class` file — bytecode is not machine code, it's an instruction set the JVM understands. The JVM then loads, verifies and executes that bytecode. On most JVMs, hot code is compiled to native machine code at runtime by the JIT compiler.

Key points

- **JVM** (Java Virtual Machine) — the engine that *runs* bytecode. Platform-specific, which is why there's a different JVM download for Windows/Linux/Mac.
- **JRE** (Java Runtime Environment) = JVM + the standard libraries. Enough to *run* Java, not to *write* it.
- **JDK** (Java Development Kit) = JRE + compiler (`javac`), debugger, `javadoc`, `jar`. This is what you install as a developer.
- JDK 11+ ships **without a separate JRE**. If someone asks 'JRE or JDK?' the answer is: JDK, it contains everything.
- **WORA** — Write Once, Run Anywhere: the same `.class` file runs on any OS that has a JVM. The portability belongs to the *bytecode*, not the source.
- Java is **compiled AND interpreted**: compiled to bytecode ahead of time, then interpreted/JIT-compiled by the JVM.

Reference

Term	What it is	Interview one-liner
JVM	Abstract machine that executes bytecode	"It's the runtime engine — it makes Java platform-independent."
JRE	JVM + libraries	"Runtime only — enough to run a Java app."
JDK	JRE + dev tools (javac, jar, javadoc)	"What a developer installs."
Bytecode	Intermediate code in .class files	"Not machine code — the JVM's instruction set."
JIT	Just-In-Time compiler	"Compiles hot bytecode to native code for speed."

Code

Compile and run — the two commands they may ask you to write

```
javac Hello.java     # source  -> bytecode (Hello.class)
java  Hello          # bytecode -> run (NO .class extension!)
java --version       # check which JDK you're on
```

```
Output:
Hello, World!
```

The smallest Java program, annotated

```
// File name MUST match the public class name: Hello.java
public class Hello {
   public static void main(String[] args) {   // the entry point
       System.out.println("Hello, World!");   // println = print + newline
   }
}
```

```
Output:
Hello, World!
```

Why main is public static void main(String[] args)

```
public  -> JVM (outside your package) must be able to call it
static  -> no object exists yet, so it must be callable on the class itself
void    -> returns nothing to the JVM; use System.exit(0) for a status code
String[] args -> command-line arguments (`java MyApp a b` -> args = ["a","b"])

// All of these are legal:
static public void main(String[] args)      // modifiers can be reordered
public static void main(String args[])      // C-style array syntax
public static void main(String... args)     // varargs
// This one is NOT an entry point (no String[]):
public static void main()                   // just an overloaded method
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java is compiled to bytecode by javac, and the JVM interprets or JIT-compiles that bytecode at runtime. That bytecode layer is what gives Java platform independence — the same class file runs anywhere a JVM exists."

Flashcards

Q. Difference between JDK, JRE and JVM?
A. JVM runs bytecode. JRE = JVM + libraries, enough to run. JDK = JRE + dev tools like javac, used to develop.

Q. Is Java compiled or interpreted?
A. Both. javac compiles source to bytecode; the JVM interprets/JIT-compiles that bytecode into native machine code.

Q. What does WORA mean and what makes it possible?
A. Write Once Run Anywhere. The .class bytecode is platform-neutral; each OS has its own JVM that executes it.

Q. Why must main be static?
A. The JVM calls main before any object of the class exists, so it must be invocable on the class itself, not an instance.

Q. Can you overload main()?
A. Yes — you can write many main methods with different parameters, but only public static void main(String[]) is the entry point.

Q. Since Java 11, do you need to install the JRE separately?
A. No. The JDK includes everything; there's no separate JRE download for modern versions.

Quiz

Q1. What is the output of compiling a file named `Test.java` that contains `class A {}` and `public class B {}`?

[ ] A. Compiles fine, produces A.class and B.class
[x] B. Compile error: class B is public, should be declared in a file named B.java
[ ] C. Compiles, produces only B.class
[ ] D. Runtime error

Why: A source file can contain at most one public top-level class, and its name must match the file name. `public class B` in `Test.java` is a compile-time error.

Q2. Which command runs a compiled Java class named `App`?

[ ] A. java App.class
[x] B. java App
[ ] C. javac App
[ ] D. run App

Why: `java` takes the *class name*, not the file name. `java App.class` looks for a class literally called `App.class` and fails with ClassNotFoundException / NoClassDefFoundError.

Q3. `public static void main(String[] args)` — which part can you legally change and still have a valid entry point?

[ ] A. Change void to int
[ ] B. Remove static
[ ] C. Remove public
[x] D. Write `String... args` instead of `String[] args`

> **Why:** Varargs `String... args` compiles to `String[]`, so the signature still matches. Return type, static and public are all part of what the JVM looks for.

Q4. What does the JIT compiler do?

[ ] A. Compiles .java to .class
[x] B. Compiles frequently-executed bytecode into native machine code at runtime
[ ] C. Verifies bytecode is safe
[ ] D. Converts Java to JavaScript

> **Why:** javac does source→bytecode. The JIT sits inside the JVM and compiles hot methods to native code for speed, which is why long-running Java apps get faster after warm-up.

2. Data Types, Variables & Wrappers Must-know 35 min

Java has **8 primitive types** — they are not objects and live on the stack when local. Everything else is a reference to an object on the heap. Knowing the sizes and ranges is a classic rapid-fire question, and the wrapper classes are where the sneaky interview traps live.

Key points

- `byte(1) short(2) int(4) long(8) float(4) double(8) char(2) boolean(~1 bit, JVM-dependent)` — sizes in bytes, and they are the SAME on every platform (that's part of Java's portability promise).
- Whole-number literals are `int` by default; you need the `L` suffix for `long`: `long x = 10000000000L;` won't compile without it.
- Decimal literals are `double` by default; `float f = 3.14;` is a compile error — write `3.14f`.
- `char` is a **16-bit unsigned Unicode** value (0 to 65535), so `char c = 65;` is `'A'` and char arithmetic works.
- Instance/class fields get default values (`0`, `0.0`, `false`, `'\u0000'`, `null`). **Local variables do not** — using one before assignment is a compile error.
- `final` makes a variable a constant (reference can't be reassigned — the object it points to can still mutate).

Reference

Primitive	Size	Wrapper	Range / notes	Default
byte	1 byte	Byte	-128 to 127	0
short	2 bytes	Short	-32,768 to 32,767	0
int	4 bytes	Integer	-2^31 to 2^31-1 (~2.1 billion)	0
long	8 bytes	Long	-2^63 to 2^63-1 — use L suffix	0L
float	4 bytes	Float	~7 decimal digits — use f suffix	0.0f
double	8 bytes	Double	~15 decimal digits, default for decimals	0.0
char	2 bytes	Character	0 to 65,535 (Unicode, unsigned)	'\u0000'
boolean	JVM-dependent	Boolean	true / false — NOT 0/1 in Java	false

Code

Autoboxing, unboxing and the Integer cache trap (VERY common question)

```
Integer a = 127, b = 127;
System.out.println(a == b);      // true  -> same cached object

Integer c = 128, d = 128;
System.out.println(c == d);      // false -> different objects (outside cache)

System.out.println(c.equals(d)); // true  -> compares VALUE (always use this!)

Integer e = new Integer(127), f = new Integer(127);
System.out.println(e == f);      // false -> `new` ALWAYS makes a new object

int g = 128, h = 128;
System.out.println(g == h);      // true  -> primitives compare by value
```

```
Output:
true
false
true
false
true
```

> Integer caches -128..127 (Byte, Short, Long, Character too; Character is 0..127). `new` bypasses the cache. **Never compare wrappers with ==.**

Widening vs narrowing: implicit vs explicit casts

```
int i = 100;
long l = i;          // widening   - automatic, no data loss
double d = l;        // widening   - automatic
// int x = l;        // ERROR - narrowing needs a cast
int x = (int) l;     // explicit cast

double pi = 3.99;
int trunc = (int) pi;          // 3  -> truncates, does NOT round
long rounded = Math.round(3.5);// 4  -> Math.round returns long for double

byte b1 = 10;                  // OK: constant fits in byte
// byte b2 = 200;              // ERROR: 200 > 127
byte b3 = (byte) 200;          // -56 -> overflow wraps around

System.out.println(10 / 3);    // 3   -> integer division!
System.out.println(10.0 / 3);  // 3.3333333333333335
System.out.println(10 % 3);    // 1   -> remainder, works on doubles too
```

```
Output:
3
3.3333333333333335
1
```

Type promotion rules in expressions

```
byte a = 10, b = 20;
// byte c = a + b;      // ERROR! a+b is promoted to int
byte c = (byte) (a + b); // must cast back

System.out.println('A' + 0);        // 65   -> char promotes to int
System.out.println("5" + 5);        // "55" -> int becomes String
System.out.println(5 + 5 + "Java"); // "10Java" -> + is left-to-right
System.out.println("Java" + 5 + 5); // "Java55"
System.out.println(1.0 / 0);        // Infinity (no exception for doubles!)
// System.out.println(1 / 0);       // ArithmeticException: / by zero
```

```
Output:
65
55
10Java
Java55
Infinity
```

> Any arithmetic on byte/short/char promotes to int. String + anything = String concatenation.

Parsing and converting — the methods you'll use in every coding question

```
int n    = Integer.parseInt("42");
double v = Double.parseDouble("3.14");
long big = Long.parseLong("9999999999");
// Integer.parseInt("abc");  -> NumberFormatException (unchecked)

String s1 = String.valueOf(42);   // "42"  (works with null -> "null")
String s2 = Integer.toString(42); // "42"
String s3 = "" + 42;              // "42"  (works but ugly/slow)

Integer boxed   = Integer.valueOf(42);  // uses the cache
int    unboxed  = boxed;                // auto-unboxing
// Integer nullInt = null; int y = nullInt;  -> NullPointerException on unbox!
```

SAY IT LIKE THIS IN THE INTERVIEW

"There are 8 primitives with fixed sizes across platforms; everything else is a reference. Wrappers give me objects for collections and generics, and autoboxing converts automatically — but I always compare them with equals(), because == compares references and the Integer cache only covers -128 to 127."

Flashcards

Q. Why does `Integer a=127, b=127; a==b` print true but `128` prints false?
A. The Integer cache reuses objects for -128..127. Above that range, each boxing creates a new object, so == compares different references.

Q. What's the default value of a local variable vs an instance variable?
A. Instance/class fields get defaults (0, false, null). Local variables have NO default — using one before assigning is a compile error.

Q. Is `boolean` 1 bit in Java?
A. Size is JVM-dependent (often 1 byte in arrays, 4 bytes as a local). What's guaranteed: it holds only true/false, and unlike C, 0/1 are not booleans.

Q. Why does `byte c = a + b;` fail to compile when a and b are bytes?
A. byte/short/char are promoted to int during arithmetic, so the result is int and needs an explicit cast back to byte.

Q. What's the difference between `parseInt` and `valueOf`?
A. parseInt returns a primitive int; valueOf returns an Integer object (and uses the -128..127 cache).

Q. What exception does `Integer.parseInt("abc")` throw?
A. NumberFormatException — it's unchecked (extends IllegalArgumentException / RuntimeException).

Q. What is autoboxing? Give a risk it creates.
A. Automatic primitive↔wrapper conversion (int → Integer). Risk: unboxing a null wrapper throws NullPointerException, and == on wrappers compares references.

Q. final on a variable — what exactly is final?
A. The reference can't be reassigned. The object's internal state can still change (final List can still have elements added).

Quiz

Q1. What does this print? `System.out.println("Result: " + 2 + 3 * 4);`

[ ] A. Result: 14
[x] B. Result: 212
[ ] C. Result: 24
[ ] D. Compile error

Why: `*` binds tighter than `+`, so `3*4 = 12`. Then `"Result: " + 2` → `"Result: 2"`, then `+ 12` → `"Result: 212"`. Evaluation is left-to-right once precedence is applied.

Q2. Which statement about `char` is true?

[ ] A. char is signed and 8-bit
[x] B. char is an unsigned 16-bit Unicode value
[ ] C. char cannot be used in arithmetic

[ ] D. char defaults to '\0' only in arrays

> **Why:** char is 16-bit unsigned (0–65535) UTF-16 code unit. It's the only unsigned primitive in Java, and it participates in arithmetic via int promotion.

Q3. What happens here? `Integer x = null; int y = x;`

[ ] A. y becomes 0
[x] B. NullPointerException at runtime
[ ] C. Compile error
[ ] D. y becomes null

> **Why:** Unboxing calls x.intValue() on a null reference → NPE. This bites in real code when a Map<Integer,Integer> misses a key.

Q4. `long big = 9_000_000_000;` — does this compile?

[ ] A. Yes, underscores are allowed
[x] B. No, needs the L suffix: 9_000_000_000L
[ ] C. No, underscores aren't allowed in Java
[ ] D. No, the value is too large for long

> **Why:** Underscores in numeric literals are legal since Java 7 (readability only). But the literal is still an int by default and 9 billion overflows int, so you need the L suffix.

Q5. What is printed? `double d = 1.0 / 0; System.out.println(d);`

[ ] A. ArithmeticException
[x] B. Infinity
[ ] C. NaN
[ ] D. 0.0

> **Why:** Floating-point division by zero yields Infinity (or NaN for 0.0/0.0), not an exception. Only *integer* division by zero throws ArithmeticException.

3. Operators & Precedence Core 25 min

You use these every day, so the only reason they show up in interviews is the gotchas: `&&` vs `&`, short-circuiting, integer division, and the increment operators.

Key points

- **Short-circuit**: `&&` and `||` stop as soon as the result is known. `&` and `|` on booleans always evaluate both sides.
- Use short-circuiting deliberately — it's the idiomatic null guard: `if (s != null && s.length() > 3)`.
- `&`, `|`, `^` are also **bitwise** operators on integers (`^` swaps two numbers without a temp).
- `>>` keeps the sign, `>>>` fills with zeros (unsigned shift), `<<` multiplies by 2^n.
- `++i` (pre) increments then uses the value; `i++` (post) uses the value then increments.
- Ternary `?:` is right-associative and must be a complete expression, not a statement.

Reference

Operator	Meaning	Gotcha
&& / ||	Short-circuit logical	Skips the right side when possible — the null-guard trick
& / |	Non-short-circuit logical; also bitwise	On booleans it evaluates BOTH sides — can NPE where && wouldn't
^	XOR	Bitwise on ints: a^b^b == a (swap trick)
>> / >>>	Signed / unsigned right shift	>>> fills with 0, so negative numbers become huge positives
?:	Ternary	Right-associative; both branches must have compatible types
instanceof	Type check	Java 16+: `if (o instanceof String s)` binds s directly

Code

Post vs pre increment — the classic dry-run question

```
int i = 5;
System.out.println(i++);   // 5  -> prints, THEN increments
System.out.println(i);     // 6
System.out.println(++i);   // 7  -> increments, THEN prints

int a = 10;
a = a++;                   // a stays 10! (old value assigned back)
System.out.println(a);

int x = 5, y = 5;
System.out.println(x++ + ++x);   // 5 + 7 = 12
System.out.println(y-- - --y);   // 5 - 3 = 2
```

```
Output:
5
6
7
10
12
2
```

Short-circuit saves you — and its absence breaks you

```
String s = null;

if (s != null && s.length() > 3) { }   // safe: right side never runs
// if (s != null & s.length() > 3) { } // NullPointerException!

int n = 10;
if (n > 5 || n / 0 == 1) {            // safe: || stops at true
   System.out.println("short-circuited");
}

// Ternary with a null-guard
String label = (s == null) ? "none" : s.toUpperCase();
```

Output:
```
short-circuited
```

Bit tricks that show up in coding rounds

```
// swap without a temp
int a = 5, b = 9;
a = a ^ b; b = a ^ b; a = a ^ b;   // a=9, b=5

// fast multiply / divide by powers of two
int m = 7 << 2;    // 28   (7 * 4)
int d = 28 >> 2;   // 7    (28 / 4)

// check even/odd
boolean even = (n & 1) == 0;

// unsigned shift on a negative number
int neg = -8;
System.out.println(neg >>  1);   // -4   (keeps sign)
System.out.println(neg >>> 1);   // 2147483644 (fills with 0)
```

Output:
```
-4
2147483644
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java operators follow C-like precedence. The two things I'm careful about: `++i` vs `i++` in expressions, and using `&&`/`||` deliberately for short-circuiting — `&` and `|` evaluate both operands and can turn a safe null check into an NPE."

Flashcards

Q. Difference between && and &?
A. && short-circuits (skips the right operand if the left decides the result); & always evaluates both. On integers, & is bitwise AND.

Q. What is `a = a++;` when a is 10?
A. a stays 10. a++ returns the old value (10) and that value is assigned back to a.

Q. What is -8 >>> 1?
A. 2147483644. >>> is the unsigned right shift — it fills with zeros instead of the sign bit.

Q. How do you swap two ints with XOR?
A. a = a ^ b; b = a ^ b; a = a ^ b; — works because x^x == 0 and x^0 == x.

Q. Is the ternary operator a statement or an expression?
A. An expression — it produces a value, so both branches must have compatible types and it can't stand alone as a statement.

Q. How do you check if a number is a power of two?
A. n > 0 && (n & (n - 1)) == 0 — powers of two have exactly one set bit.

Quiz

Q1. What is the value of `x` after: `int x = 3; x += x++ * 2 + ++x;`

[ ] A. 15
[x] B. 16
[ ] C. 17
[ ] D. 18

> **Why:** x++ → 3 (x becomes 4), ++x → 5 (x becomes 5). So 3*2 + 5 = 11, then x = 3 + 11 = 14? Careful: compound assignment evaluates the *current* x on the left at the end: x = 3 + (3*2 + 5) = 14. Actually re-check: left operand of += is read first (3), right side = 3*2 + 5 = 11, total = 14.

Q2. Which operator has the HIGHEST precedence in Java?

[ ] A. * (multiplication)
[x] B. ++ (postfix)
[ ] C. &&
[ ] D. == (equality)

> **Why:** Postfix `++/--` and `. / [] / (params)` bind tighter than everything else, including unary prefix and multiplication.

Q3. `System.out.println(7 / 2 * 2.0);` prints:

[ ] A. 7.0
[x] B. 6.0
[ ] C. 7
[ ] D. 6

> **Why:** Left-to-right: `7 / 2` is integer division → 3, then `3 * 2.0` → 6.0. The int result is cast to double only at the second step, so precision is already lost.

4. Control Flow: switch, loops, break/continue Core 30 min

Modern Java (14+) rewrote `switch`. Interviewers love asking the *old* fall-through question, but if you volunteer the arrow syntax and switch expressions you signal you know current Java — a nice edge for a fresher.

Key points

- Classic `switch` uses `break`; without it, execution **falls through** to the next case.
- Arrow switch (`case 1 ->`) does NOT fall through and needs no break — this is the one to write.
- Switch **expressions** (Java 14+) return a value and must be exhaustive (use `default` or cover all enum constants); use `yield` in a block.
- `switch` works on: byte/short/char/int, their wrappers, String, enums, and (Java 21) patterns with type matching.
- `break` exits the loop; `continue` skips to the next iteration. **Labelled** break/continue escapes a specific outer loop.
- Enhanced for (`for (T x : coll)`) works over arrays and anything Iterable.

Code

Old switch vs new switch — write it the new way

```
// OLD: colon syntax, needs break, falls through if you forget
String oldWay(int day) {
   String name;
   switch (day) {
       case 1: name = "Mon"; break;
       case 2: name = "Tue"; break;
       default: name = "Other";
   }
   return name;
}

// NEW (Java 14+): arrow syntax, no fall-through, no break
String newWay(int day) {
   return switch (day) {
       case 1 -> "Mon";
       case 2, 3, 4, 5 -> "Weekday";
       case 6, 7 -> "Weekend";
       default -> throw new IllegalArgumentException("bad day: " + day);
   };
}

// Switch expression with a block -> use `yield`
int quarter = switch (month) {
   case 1, 2, 3    -> 1;
   case 4, 5, 6    -> 2;
   case 7, 8, 9    -> 3;
   case 10, 11, 12 -> 4;
   default -> { System.out.println("invalid"); yield -1; }
};
```

Fall-through: the question they ask to see if you've been burned

```
int n = 2;
switch (n) {
   case 1: System.out.println("one");
   case 2: System.out.println("two");
   case 3: System.out.println("three"); break;
   default: System.out.println("other");
}
// Prints: two, three  (no break after case 2 -> falls into case 3)
```

```
Output:
two
three
```

Labelled break — escaping nested loops cleanly

```
outer:
for (int i = 0; i < 3; i++) {
   for (int j = 0; j < 3; j++) {
       if (i == 1 && j == 1) break outer;   // leaves BOTH loops
       System.out.println(i + "," + j);
   }
}
// 0,0  0,1  0,2  1,0  -> then break outer
continue outer;  // jumps to the next iteration of the OUTER loop
```

```
Output:
0,0
0,1
0,2
1,0
```

Loops you must be able to write from memory

```
// for
for (int i = 0; i < 5; i++) { }

// while
int i = 0;
while (i < 5) { i++; }

// do-while: body ALWAYS runs at least once
do { System.out.println("runs once"); } while (false);

// enhanced for (read-only over the elements)
for (String s : new String[]{"a", "b"}) { System.out.println(s); }

// infinite loop + break
while (true) { if (done()) break; }

// foreach with index when you need both
int idx = 0;
for (String s : list) { System.out.println(idx++ + ": " + s); }
```

SAY IT LIKE THIS IN THE INTERVIEW

"I default to the modern arrow switch and switch expressions — no fall-through, it returns a value, and the compiler forces exhaustiveness. I know the classic colon form falls through without break, which is exactly why I avoid it."

Flashcards

Q. What is switch fall-through and how do you avoid it?
A. In colon-style switch, without `break` execution continues into the next case. Use arrow syntax `case 1 ->` (no fall-through) or add break.

Q. What is a switch expression and which keyword yields a value from a block?
A. A switch that returns a value (Java 14+): `int x = switch (...) { case 1 -> 10; default -> { yield 0; } };`. `yield` returns a value from a block.

Q. Which types can you switch on?
A. byte, short, char, int and their wrappers, String, enums, and since Java 21, type patterns. Not long, float, double, or boolean.

Q. Difference between break, continue and a labelled break?
A. break exits the innermost loop/switch; continue skips to the next iteration; `label: ... break label;` exits an outer labelled loop.

Q. Which loop always executes its body at least once?
A. do-while — the condition is checked after the body.

Q. Can you use `break` outside a loop or switch?
A. No — compile error, unless it's a labelled break targeting an enclosing labelled block.

Quiz

Q1. In a switch expression, what happens if no case matches and there's no default?

[ ] A. It returns null
[x] B. Compile error (for enums/ sealed types) or MatchException at runtime
[ ] C. It falls through to the end
[ ] D. It throws NullPointerException

> **Why:** Switch expressions must be exhaustive. For enums the compiler checks it (error if a constant is missing and no default); otherwise Java throws MatchException at runtime.

Q2. How many times does this loop body run? `for (int i = 0; i < 10; i++) { if (i % 2 == 0) continue; System.out.print(i); }`

[ ] A. 10
[x] B. 5
[ ] C. 4
[ ] D. 0

> **Why:** `continue` skips even values, so only 1,3,5,7,9 print — 5 executions.

Q3. Does `case 1 -> ...` fall through to `case 2`?

[ ] A. Yes, always
[x] B. No — arrow cases never fall through
[ ] C. Only if you omit break
[ ] D. Only with yield

> **Why:** Arrow-form switch labels do not fall through. That's the main motivation for the new syntax.

5. Arrays Core 30 min

Arrays are fixed-size, zero-indexed objects stored contiguously. You'll use `ArrayList` in real code, but arrays still rule coding rounds (two-pointer, sliding window, matrices) and at least one interview question will be array-flavoured.

Key points

- Length is fixed at creation — `arr.length` is a **field** (not a method, unlike `String.length()` and `List.size()`).
- Elements get default values automatically (`0`, `false`, `null`), so you never have to initialise a new array.
- 2D arrays in Java are **arrays of arrays** — rows can have different lengths (jagged arrays).
- `ArrayIndexOutOfBoundsException` for a bad index; `ArrayStoreException` for putting the wrong type in.
- `Arrays` utility class = `toString`, `sort`, `binarySearch`, `equals`, `fill`, `copyOf`, `asList`, `stream`.
- Array covariance: `String[]` IS-A `Object[]`, so `Object[] o = new String[1]; o[0] = 5;` compiles and throws at runtime.

Code

Creating, filling and printing arrays

```
int[] a = new int[5];              // [0,0,0,0,0]
int[] b = {1, 2, 3, 4, 5};         // literal (only at declaration)
int[] c = new int[]{1, 2, 3};      // anonymous form
// int[] d = new int[3]{1,2,3};    // ERROR: can't give size AND values

System.out.println(a.length);              // 5   (field, no parentheses)
System.out.println(Arrays.toString(b));    // [1, 2, 3, 4, 5]
System.out.println(b);                     // [I@1b6d3586 <- useless! arrays
                                          // don't override toString()
```

```
Output:
5
[1, 2, 3, 4, 5]
[I@1b6d3586
```

2D arrays and jagged arrays

```
int[][] m = new int[3][4];          // 3 rows, 4 columns
int[][] t = { {1,2}, {3,4,5}, {6} }; // jagged: rows differ in length

System.out.println(m.length);       // 3  -> number of ROWS
System.out.println(m[0].length);    // 4  -> columns in row 0
System.out.println(Arrays.deepToString(t)); // [[1, 2], [3, 4, 5], [6]]

// Iterate a matrix
for (int r = 0; r < m.length; r++)
   for (int c = 0; c < m[r].length; c++)
       m[r][c] = r * c;

for (int[] row : m) System.out.println(Arrays.toString(row));
```

```
Output:
3
4
[[1, 2], [3, 4, 5], [6]]
```

The java.util.Arrays toolkit (memorise these)

```
int[] a = {5, 3, 1, 4, 2};

Arrays.sort(a);                       // in-place ascending -> [1,2,3,4,5]
int idx = Arrays.binarySearch(a, 4);  // 3  (array MUST be sorted first!)
Arrays.fill(a, 0);                    // [0,0,0,0,0]
int[] copy = Arrays.copyOf(a, 3);     // [0,0,0]  (truncate or pad)
int[] range = Arrays.copyOfRange(a, 1, 3);
Arrays.equals(a, copy);               // value equality (NOT a.equals(copy)!)

// asList gotcha: the list is FIXED-SIZE and backed by the array
List<String> fixed = Arrays.asList("a", "b", "c");
// fixed.add("d");   -> UnsupportedOperationException
List<String> real = new ArrayList<>(Arrays.asList("a", "b", "c")); // mutable

// sorting objects
Arrays.sort(words, Comparator.comparing(String::length).reversed());
```

`Arrays.asList` returns a fixed-size view. Wrap it in `new ArrayList<>(...)` or use `List.of(...)` (immutable) when you need a real list.

Two patterns that solve half of array coding problems

```
// 1. Two pointers: check if a sorted array has a pair summing to target
static boolean hasPair(int[] a, int target) {
   int l = 0, r = a.length - 1;
   while (l < r) {
       int sum = a[l] + a[r];
       if (sum == target) return true;
       if (sum < target) l++; else r--;
   }
   return false;
}

// 2. Kadane's algorithm: maximum subarray sum, O(n)
static int maxSubArray(int[] a) {
   int best = a[0], cur = a[0];
   for (int i = 1; i < a.length; i++) {
       cur = Math.max(a[i], cur + a[i]);
       best = Math.max(best, cur);
   }
   return best;
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"Arrays are fixed-size, zero-indexed, and default-initialised. `arr.length` is a field, not a method. For anything dynamic I'd reach for ArrayList instead, but for coding rounds I'm comfortable with two-pointer and sliding-window patterns directly on arrays."

Flashcards

Q. Why do you need Arrays.toString(arr) instead of printing the array?
A. Arrays don't override toString(), so `System.out.println(arr)` prints the default Object form like [I@1b6d3586. Arrays.toString() gives readable output.

Q. arr.length vs String.length() vs List.size()?
A. Array length is a public final *field*; String and List use *methods* — s.length() and list.size().

Q. What is a jagged array in Java?
A. A 2D array whose rows have different lengths — Java's multi-dimensional arrays are arrays of arrays, so each row is independent.

Q. What does Arrays.asList() return and what's the catch?
A. A fixed-size List backed by the array — add/remove throw UnsupportedOperationException, but set() works and mutates the original array.
Q. Difference between List.of() and Arrays.asList()?
A. List.of() (Java 9) is fully immutable and rejects nulls; Arrays.asList() is fixed-size but allows set() and nulls.

Q. What must be true before Arrays.binarySearch()?
A. The array must be sorted in ascending order, otherwise the result is undefined.

Q. Default values in a new int[] vs new String[]?
A. int[] → 0; String[] → null. All arrays are default-initialised.

Quiz

Q1. What does this print? `int[] x = {1,2,3}; int[] y = x; y[0] = 99; System.out.println(x[0]);`

[ ] A. 1
[x] B. 99
[ ] C. Compile error
[ ] D. ArrayStoreException

> **Why:** Arrays are objects, so `y = x` copies the *reference*, not the array. Both variables point at the same array. Use Arrays.copyOf() or clone() for a real copy.

Q2. Which line throws at compile time? (a) `Object[] o = new String[3];` (b) `o[0] = new Integer(5);`

[ ] A. (a) — arrays are invariant
[x] B. (b) — the assignment compiles, so it fails at runtime with ArrayStoreException
[ ] C. Both compile and run fine
[ ] D. (b) throws ClassCastException at compile time

> **Why:** Arrays are covariant: String[] IS-A Object[]. But the JVM remembers the component type and throws ArrayStoreException when you store an Integer at runtime.

Q3. What's the time complexity of Kadane's maximum subarray algorithm?

[ ] A. O(n log n)
[x] B. O(n)
[ ] C. O(n^2)
[ ] D. O(1)

> **Why:** Single pass keeping the best sum ending here and the best overall — O(n) time, O(1) space.

Q4. `int[][] m = new int[3][4];` — what is `m.length` and `m[0].length`?

[ ] A. 4 and 3
[x] B. 3 and 4
[ ] C. 12 and 3
[ ] D. 3 and 3

> **Why:** `m.length` is the number of rows (3); `m[0].length` is the length of the first row (4).

6. Strings, Immutability & the String Pool Must-know 45 min

If there's one topic guaranteed in a Java fresher interview, it's this one. String immutability, the string pool, `==` vs `equals()`, and StringBuilder — all in one conversation.

Key points

- `String` is **immutable**: every 'change' creates a new object. That's what makes it thread-safe, hashable and safe to use as a HashMap key.
- **String Pool** (String Constant Pool) — a special heap area where literal strings are interned and reused. `new String("hi")` deliberately bypasses it.
- `==` compares **references**; `equals()` compares **contents**. String overrides `equals()`.
- `String` overrides `equals()` AND `hashCode()`, so it's a safe HashMap key.
- **StringBuilder** (fast, not thread-safe) vs **StringBuffer** (synchronized, thread-safe, slower). Use StringBuilder.
- String concatenation in a loop with `+` is O(n^2) — the compiler creates a new StringBuilder per iteration. Use an explicit StringBuilder.

Reference

Class	Mutable?	Thread-safe?	When to use
String	No (immutable)	Yes (naturally)	Default choice; keys, constants, literals
StringBuilder	Yes	No	Building strings in a single-threaded method (most cases)
StringBuffer	Yes	Yes (synchronized)	Legacy / shared across threads — rare; prefer StringBuilder
CharSequence	interface	-	Parameter type when you accept any of the above

Code

The String pool, visualised in code — memorise this output

```
String s1 = "java";            // literal -> goes in the pool
String s2 = "java";            // same literal -> REUSES the pooled object
String s3 = new String("java");// `new` -> forces a NEW object on the heap
String s4 = s3.intern();       // returns the pooled instance

System.out.println(s1 == s2);          // true  -> same pooled object
System.out.println(s1 == s3);          // false -> different objects
System.out.println(s1 == s4);          // true  -> intern() found the pool copy
System.out.println(s1.equals(s3));     // true  -> contents match

// Compile-time constant folding
String a = "ja" + "va";                // constant expression -> "java"
System.out.println(a == s1);           // true

String x = "ja";
String b = x + "va";                   // runtime concat -> new object
System.out.println(b == s1);           // false
System.out.println(b.equals(s1));      // true
```

```
Output:
true
false
true
true
true
false
true
```

`+` on two compile-time constants is folded by the compiler. `+` involving a variable happens at runtime and creates a new String.

Immutability in practice — why `toUpperCase()` 'does nothing'

```
String s = "hello";
s.toUpperCase();          // returns a NEW String, discards it
System.out.println(s);    // hello  <- unchanged!

s = s.toUpperCase();      // reassign to capture the result
System.out.println(s);    // HELLO

// Same for every "modifying" String method:
// concat, replace, substring, trim, strip... they ALL return new objects
```

```
Output:
hello
HELLO
```

StringBuilder — and the loop performance trap

```
// BAD: O(n^2). A new StringBuilder is built every iteration.
String result = "";
for (int i = 0; i < 100_000; i++) {
   result += i;          // never do this in a loop
}

// GOOD: one builder, O(n)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100_000; i++) {
   sb.append(i);
}
String result2 = sb.toString();

// Chained API
String msg = new StringBuilder()
       .append("Hello, ").append(name).append("!")
       .reverse().toString();

// Note: append(i) with int is fine; append(char) is the fastest overload
```

String methods you'll be asked to use in coding rounds

```
String s = "  Java Programming  ";

s.length();                       // 20
s.trim();                         // removes chars <= U+0020 (old)
s.strip();                        // Unicode-aware (Java 11+) - prefer this
s.stripLeading(); s.stripTrailing();
s.toLowerCase(); s.toUpperCase();
s.charAt(0);                      // ' '
s.substring(2, 6);                // "Java"  (begin inclusive, end EXCLUSIVE)
s.indexOf("Java");                // 2  (-1 if not found)
s.lastIndexOf("m");               // index of last occurrence
s.contains("Prog");               // true
s.startsWith("  Ja");             // true
s.replace("Java", "Kotlin");      // all occurrences (CharSequence version)
s.replaceAll("\\s+", " ");        // regex version
s.split(" ");                     // regex split
s.toCharArray();                  // char[]
s.chars();                        // IntStream of chars
s.repeat(2);                      // Java 11
s.isBlank();                      // Java 11 - true for whitespace-only
s.lines();                        // Java 11 - Stream<String>
"".isEmpty();                     // true (length == 0)
"  ".isBlank();                   // true (Java 11)

// Reversing a string (classic question)
String rev = new StringBuilder(s).reverse().toString();

// Comparing
s.compareTo(other);               // lexicographic, 0 when equal
s.equalsIgnoreCase(other);        // case-insensitive equality
```

Text blocks (Java 15+) — multi-line strings without the noise

```
String json = """
       {
         "name": "Parvez",
         "role": "Java Developer"
       }
       """;
// Incident leading whitespace is stripped automatically.
System.out.println(json);
```

```
Output:
{
 "name": "Parvez",
 "role": "Java Developer"
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"String is immutable — every modification returns a new object, which is why it's thread-safe and safe as a HashMap key. Literals are interned in the string pool and reused; `new String()` forces a separate object. I compare strings with equals(), never ==, and I use StringBuilder when I'm building strings in a loop."

Flashcards

Q. Why is String immutable in Java?
A. For security, thread-safety, hashcode caching and string-pool reuse. Since strings are used as keys and parameters everywhere, immutability makes them safely shareable.

Q. How many objects are created by `String s = new String("hello");`?
A. Two (if "hello" isn't already pooled): one literal in the pool and one new object on the heap. One if the literal is already interned.

Q. What does intern() do?
A. Returns the canonical pooled instance of the string, adding it to the pool if absent. Useful for making == work and saving memory with many duplicates.

Q. StringBuilder vs StringBuffer?
A. Both mutable; StringBuilder is not synchronized (faster, default choice), StringBuffer methods are synchronized (thread-safe, legacy).

Q. String vs StringBuilder vs StringBuffer on equals()?
A. String overrides equals() to compare contents. StringBuilder/StringBuffer do NOT — their equals() is reference equality.

Q. Does `String s = "a" + "b";` create 3 objects?
A. No — the compiler folds it into the single literal "ab", which is pooled.

Q. trim() vs strip()?
A. trim() removes characters <= U+0020 only; strip() (Java 11) is Unicode-aware and removes all whitespace. Prefer strip().

Q. Can a String be used as a key in HashMap safely?
A. Yes — String is immutable and caches its hashcode, so the hash never changes after insertion.

Q. What does `s.substring(1, 4)` return for "Programming"?
A. "rog" — the begin index is inclusive and the end index is exclusive.

Quiz

Q1. How many objects does `String s = new String("abc");` create (assuming "abc" is not already pooled)?

[ ] A. 1
[x] B. 2
[ ] C. 3
[ ] D. 0

> **Why:** One String object for the literal in the pool, and one on the heap created by `new`. The variable `s` refers to the heap one.

Q2. What is printed? `String a = "hi"; String b = "h" + "i"; String c = "h"; String d = c + "i"; System.out.println(a==b); System.out.println(a==d);`

[ ] A. true true
[x] B. true false
[ ] C. false false
[ ] D. false true

> **Why:** `"h"+"i"` is a compile-time constant → folded to "hi" → pooled → == true. `c + "i"` involves a variable → evaluated at runtime → a new String → == false.

Q3. Which is the correct way to compare two strings for content equality?

[ ] A. s1 == s2
[x] B. s1.equals(s2)
[ ] C. s1 = s2
[ ] D. s1.compareTo(s2) == 1

> **Why:** equals() compares contents (String overrides it). `==` compares references. Bonus safety tip: call equals on the constant — `"CONST".equals(variable)` — to avoid NPE.

Q4. Why is repeated `+` concatenation in a loop slow?

[x] A. Strings are immutable, so each + allocates a new String (and a StringBuilder)
[ ] B. Because + is synchronized
[ ] C. Because + uses reflection
[ ] D. It isn't — the compiler optimises it away

> **Why:** Each iteration builds a new StringBuilder and String, copying everything so far → $O(n^2)$. An explicit StringBuilder outside the loop makes it $O(n)$.

Q5. What is the output? `System.out.println("Java".substring(1,3));`

[x] A. av
[ ] B. ava

[ ] C. Ja
[ ] D. Jav

Why: beginIndex inclusive, endIndex exclusive → characters at 1 and 2 → "av".

Q6. Which is true about StringBuilder's equals()?

[ ] A. It compares contents, like String
[x] B. It compares references (not overridden)
[ ] C. It throws UnsupportedOperationException
[ ] D. It compares lengths

Why: StringBuilder inherits Object.equals() → reference comparison. Convert with toString() before comparing contents.

7. Input / Output & Formatting Core 25 min

You won't be grilled on this, but you need it to solve coding problems when you're practising on a platform that uses stdin. `Scanner` for simplicity, `BufferedReader` for speed.

Key points

- `Scanner` is easy but **slow** (regex-based) — fine for small inputs, times out on large ones.
- `BufferedReader` + `StringTokenizer`/`split` is the fast pattern for competitive-style input.
- `Scanner.nextLine()` after `nextInt()` is the #1 beginner bug — `nextInt()` leaves the newline in the buffer.
- `System.out.printf` / `String.format` use format specifiers: `%d %s %f %.2f %n`.
- `System.out.println` is slow in a loop — wrap in a `BufferedWriter` or `StringBuilder` for bulk output.

Code

Scanner — and the newline bug that wastes 20 minutes

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter age: ");
int age = sc.nextInt();       // reads the number, leaves "\n" in the buffer

sc.nextLine();                // <-- consume the leftover newline!
System.out.print("Enter name: ");
String name = sc.nextLine();  // now reads the actual line

// Or: always read lines and parse them
int n = Integer.parseInt(sc.nextLine());

sc.close();                   // good hygiene (don't close System.in in online judges)

// Reading until EOF
while (sc.hasNext()) { System.out.println(sc.next()); }
```

BufferedReader — the fast pattern used in coding platforms

```
import java.io.*;
import java.util.*;

class FastIO {
   public static void main(String[] args) throws Exception {
       BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
       StringTokenizer st = new StringTokenizer(br.readLine());

       int n = Integer.parseInt(st.nextToken());
       int[] arr = new int[n];
       st = new StringTokenizer(br.readLine());
       for (int i = 0; i < n; i++) arr[i] = Integer.parseInt(st.nextToken());

       // Fast bulk output
       StringBuilder out = new StringBuilder();
       for (int v : arr) out.append(v * 2).append('\n');
       System.out.print(out);
       br.close();
   }
}
```

Formatting output — printf / String.format

```
String name = "Parvez";
int age = 24;
double salary = 75432.126;

System.out.printf("Name: %s, Age: %d%n", name, age);
System.out.printf("Salary: %.2f%n", salary);       // 75432.13 (rounds)
System.out.printf("|%10s|%n", name);              // right-aligned width 10
System.out.printf("|%-10s|%n", name);             // left-aligned
System.out.printf("Hex: %x, Octal: %o%n", 255, 8); // ff, 10
System.out.printf("%,d%n", 1000000);              // 1,000,000

String s = String.format("%s is %d", name, age);  // build instead of print

// Common specifiers: %s String, %d int, %f float/double, %c char,
// %b boolean, %n platform-independent newline (prefer over \n in printf)
```

Output:

```
Name: Parvez, Age: 24
Salary: 75432.13
|    Parvez|
|Parvez    |
Hex: ff, Octal: 10
1,000,000
```

SAY IT LIKE THIS IN THE INTERVIEW

"I use Scanner for quick input but BufferedReader with StringTokenizer for large inputs, because Scanner is regex-based and slow. And I know the classic nextInt-then-nextLine newline bug — you consume the leftover newline or read lines and parse."

Flashcards

Q. Scanner vs BufferedReader for input?
A. Scanner is convenient (nextInt, nextLine, hasNext) but slow because it uses regex. BufferedReader reads raw characters fast and you parse manually — preferred for large inputs.

Q. Why does nextLine() return empty right after nextInt()?
A. nextInt() reads the number but leaves the newline in the buffer, so nextLine() immediately consumes that empty remainder. Fix: an extra nextLine() to flush it, or read everything as lines.

Q. What's the difference between %n and \n in printf?
A. %n is the platform-independent line separator; \n is always a Unix newline. Prefer %n in formatted output.

Q. How do you print a double with exactly 2 decimals?
A. System.out.printf("%.2f", value) — it rounds automatically (HALF_UP for most values, though it uses the default rounding mode).

Q. Why avoid println inside a large loop?
A. Each call flushes/does I/O work; it's dramatically slower than accumulating into a StringBuilder or BufferedWriter and printing once.

Quiz

Q1. What's the fastest safe way to read 1 million integers from stdin in Java?

[ ] A. Scanner.nextInt() in a loop
[x] B. BufferedReader + StringTokenizer + Integer.parseInt
[ ] C. System.console().readLine()
[ ] D. DataInputStream.readInt() in a loop

Why: BufferedReader buffers the stream and StringTokenizer splits without regex — 5–10x faster than Scanner. (For raw speed you'd write a custom byte-buffer reader.)

Q2. What does `System.out.printf("%.1f", 2.25);` print?

[x] A. 2.2
[ ] B. 2.3

[ ] C. 2.25
[ ] D. 2.0

Why: Formatter uses half-up rounding, but 2.25 is exactly representable and rounds-to-even applies in some paths — in practice `%.1f` of 2.25 on the JDK prints 2.2. The safe interview answer: it rounds to one decimal place.

8. Stack, Heap & Pass-by-Value Must-know 30 min

"Is Java pass-by-value or pass-by-reference?" — the answer is **pass-by-value, always**, and the confusion comes from the fact that for objects, the *value being copied* is the reference. Know this cold; it's asked constantly and most people get it wrong.

Key points

- Primitives live on the **stack** (when local); objects live on the **heap**; the reference variable lives on the stack.
- Java passes a **copy of the variable's value**: for primitives, a copy of the value; for objects, a copy of the reference.
- So you **can** mutate the object a parameter points to, but you **cannot** reassign the caller's variable to point elsewhere.
- `final` on a parameter doesn't stop mutation — it only stops reassignment inside the method.
- Java has **no pointer arithmetic** and **no pass-by-reference** — unlike C++'s `&`.

Reference

Where	Holds	Lifetime	Notes
Stack	Local primitives, object references, method frames	Until the method returns	Fast, LIFO, per-thread, no GC
Heap	All objects and their instance fields	Until unreachable + GC	Shared across threads, managed by GC
Metaspace	Class metadata, method bytecode	Until classloader unloaded	Replaced PermGen in Java 8
String pool	Interned string literals	JVM lifetime (mostly)	Lives on the heap since Java 7

Code

The pass-by-value proof — predict the output before you scroll

```
static void changePrimitive(int n)   { n = 100; }
static void reassign(StringBuilder s){ s = new StringBuilder("new"); }
static void mutate(StringBuilder s)  { s.append(" world"); }

public static void main(String[] a) {
   int num = 10;
   changePrimitive(num);
   System.out.println(num);            // 10  <- unchanged

   StringBuilder sb = new StringBuilder("hello");
   reassign(sb);
   System.out.println(sb);             // hello <- still the original!

   mutate(sb);
   System.out.println(sb);             // hello world <- MUTATED

   // Same story with arrays and any object:
   static void swap(Integer x, Integer y) { Integer t = x; x = y; y = t; } // does nothing
}
```

```
Output:
10
hello
hello world
```

This is THE question. Say: "Java is always pass-by-value. For objects, the value passed is a copy of the reference — so I can mutate the object it points to, but reassigning the parameter has no effect on the caller's variable."

Why swap() can't work the obvious way in Java

```
// Doesn't work: primitives are copied
static void swap(int a, int b) { int t = a; a = b; b = t; }

// Doesn't work either: Integer is immutable, so even the mutation trick fails
static void swap(Integer a, Integer b) { Integer t = a; a = b; b = t; }

// What DOES work: mutate the holder, or return the values
static void swap(int[] arr, int i, int j) {
   int t = arr[i]; arr[i] = arr[j]; arr[j] = t;   // mutates shared array
}
static void swap(Point p) { int t = p.x; p.x = p.y; p.y = t; } // mutates object
```

SAY IT LIKE THIS IN THE INTERVIEW

"Java is strictly pass-by-value. When I pass an object, the value being copied is the reference — so mutating the object through that reference affects the original, but reassigning the parameter never changes the caller's variable. That's why a plain swap(x, y) method can't work."

Flashcards

Q. Is Java pass-by-value or pass-by-reference?
A. Always pass-by-value. For objects the value copied is the reference, which is why mutation of the object is visible but reassignment of the parameter is not.

Q. Why doesn't `void swap(int a, int b)` work in Java?
A. a and b are copies of the caller's values; swapping the copies has no effect. You must mutate a shared container/object or return the values.

Q. What's stored on the stack vs the heap?
A. Stack: method frames, local primitives and object references (per thread). Heap: all objects and their instance fields (shared, GC-managed).

Q. Why does passing a StringBuilder and appending work, but assigning a new one inside the method not work?
A. append() mutates the shared object through the copied reference. Assigning a new object only changes the local copy of the reference.

Q. Where do string literals live since Java 7?
A. On the heap, in the string pool. (Before Java 7 they were in PermGen.)

Q. Does `final` on a method parameter prevent modifying the object?
A. No — it only prevents reassigning the parameter. You can still mutate the object's state.

Quiz

Q1. What's the output? `static void f(String s){ s = s + "!"; }` `String t = "hi"; f(t);` `System.out.print(t);`

[ ] A. hi!
[x] B. hi
[ ] C. null
[ ] D. Compile error

Why: Two reasons: String is immutable (so `+` makes a new object) and Java is pass-by-value (so reassigning `s` inside doesn't touch `t`).

Q2. Which statement about Java memory is FALSE?

[ ] A. Local primitives live on the stack
[ ] B. Objects are always allocated on the heap
[ ] C. Each thread has its own stack but shares the heap
[x] D. The garbage collector destroys objects immediately when a method returns

Why: GC runs on its own schedule and only collects unreachable objects. A method returning doesn't trigger collection — objects are collected when no live references remain and the GC chooses to run.

Q3. `void m(final List<String> l) { l.add("x"); }` — what happens?

[ ] A. Compile error: final list can't be modified
[x] B. Compiles and runs; the list is mutated
[ ] C. Runtime UnsupportedOperationException
[ ] D. Compiles, but the change is discarded

Why: `final` only forbids reassigning `l` to another list. Mutating the referenced list is perfectly legal.

9. Packages, Access Modifiers & the static Keyword Core 30 min

Small topic, guaranteed questions — especially the access-modifier table and "what does static really mean". These are pure recall marks, so memorise the table.

Key points

- `static` members belong to the **class**, not to any instance — one copy shared by all objects, loaded when the class is initialised.
- Static methods **cannot** access instance members directly and **cannot** use `this` or `super`.
- Static methods **can** be overloaded, and **can** be inherited, but are **hidden** (not overridden) when redeclared in a subclass — resolved at compile time.
- **Static initialisation order**: static blocks/fields run once, top-to-bottom, when the class is first loaded; then instance blocks/fields; then the constructor.
- `main` is static precisely so the JVM can call it without creating an object.
- Packages group related classes and avoid name collisions; `import` is compile-time sugar only (doesn't load code or cost runtime memory).

Reference

Modifier	Same class	Same package	Subclass (other pkg)	Anywhere
private	Yes	No	No	No
default (package-private)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Code

Static: shared state, and the trap it creates

```
class Counter {
   static int count = 0;      // ONE copy shared by every instance
   int id;                    // one per object

   Counter() { id = ++count; }
   static int getCount() { return count; }        // static method
   // static void bad() { System.out.println(id); } // ERROR: can't touch instance
}

Counter a = new Counter();   // id=1
Counter b = new Counter();   // id=2
System.out.println(Counter.getCount());  // 2  <- call on the CLASS, not an object
```

```
Output:
2
```

Initialisation order — a favourite output-prediction question

```
class Init {
   static { System.out.println("1. static block"); }
   { System.out.println("2. instance block"); }
   Init() { System.out.println("3. constructor"); }

   public static void main(String[] a) {
       System.out.println("4. main start");
       new Init();
       new Init();          // static block does NOT run again
       System.out.println("5. main end");
   }
}
// Order: static block (once, at class load) -> main ->
//        per object: instance block -> constructor
```

Output:

```
1. static block
4. main start
2. instance block
3. constructor
2. instance block
3. constructor
5. main end
```

Static method hiding vs instance method overriding

```
class Parent { static void greet() { System.out.println("Parent"); } }
class Child  extends Parent {
   static void greet() { System.out.println("Child"); }   // HIDING, not overriding
}

Parent p = new Child();
p.greet();        // "Parent"  <- resolved by the REFERENCE type at compile time
new Child().greet(); // "Child"

// Contrast with instance methods:
class A { void show() { System.out.println("A"); } }
class B extends A { @Override void show() { System.out.println("B"); } }
A ref = new B();
ref.show();       // "B"  <- dynamic dispatch at runtime
```

Output:

```
Parent
Child
B
```

Packages & imports

```
// Declare at the top of the file, before imports
package com.parvez.project.service;

import java.util.List;              // single type
import java.util.*;                  // whole package (no runtime cost)
import static java.lang.Math.*;      // static import: use PI, max() directly
import java.sql.Date;                // disambiguate a name clash

double r = max(PI, 3);               // thanks to the static import

// Two classes with the same simple name -> use the fully-qualified name
java.util.Date utilDate = new java.util.Date();

// Compiling with a package structure:
// javac -d out src/com/parvez/App.java
// java  -cp out com.parvez.App
```

SAY IT LIKE THIS IN THE INTERVIEW

"static means the member belongs to the class rather than an instance — one shared copy, callable without an object, and it can't touch instance state or use this. Access modifiers go private → package-private → protected → public, widening at each step, and protected also grants access to subclasses in other packages."

Flashcards

Q. What does static mean on a field?
A. One copy belonging to the class, shared by all instances, initialised when the class is loaded — not per-object.

Q. Can a static method access instance variables or use this?
A. No. Static context has no instance; referring to instance members or this/super is a compile error.

Q. Can static methods be overridden?
A. No — they're hidden. Redeclaring a static method in a subclass resolves by the reference type at compile time, not by the runtime object.

Q. Order of execution for static block, instance block, constructor?
A. Static block first (once, at class load), then per object: instance initialiser block, then constructor body.

Q. Can you overload static methods? Can main be overloaded?
A. Yes to both — overloading depends on the parameter list, not staticness. Only public static void main(String[]) is the entry point.

Q. What's the difference between protected and package-private?
A. protected = package access PLUS subclasses in other packages. Package-private (no modifier) = same package only, no subclass exception.

Q. Why is main() static?
A. So the JVM can invoke it without instantiating the class — no object exists yet at startup.

Q. Does `import java.util.*;` cost memory or load all classes?
A. No. Imports are compile-time name resolution only; the JVM loads classes lazily on first use.

Quiz

Q1. Which access modifier allows access from a subclass in a DIFFERENT package?

[ ] A. private
[ ] B. default (no modifier)
[x] C. protected
[ ] D. None of them

Why: protected = package-private + subclasses (even in other packages). Note the subtlety: in another package, a subclass can only access it through its own type's reference.

Q2. What is the initialisation order for a subclass instantiation?

[ ] A. Child static → Parent static → Parent instance → Parent ctor → Child instance → Child ctor
[x] B. Parent static → Child static → Parent instance → Parent ctor → Child instance → Child ctor
[ ] C. Parent ctor → Child ctor → static blocks
[ ] D. Instance blocks → static blocks → constructors

Why: Static blocks run top-down as each class loads (parent first), then per object: parent instance initialisers + constructor, then child instance initialisers + constructor.

Q3. Can a static method be called using an object reference, e.g. `obj.staticMethod()`?

[x] A. Yes, but it's bad style and resolved by the declared type
[ ] B. No, compile error
[ ] C. Yes and it uses dynamic dispatch
[ ] D. Only if the method is final

Why: It compiles (with a warning in some IDEs) but is misleading — the call is resolved from the *declared* type, not the runtime object, so it looks polymorphic but isn't.

Q4. Why can't a static method be abstract?

[x] A. Because abstract requires overriding, and static methods are hidden not overridden

[ ] B. Because abstract methods are public

[ ] C. Because static methods are final

[ ] D. It can be

Why: abstract means 'subclasses must provide an implementation via dynamic dispatch'. Static methods have no dynamic dispatch, so the combination is a compile error.

10. Varargs, Enums & Command-Line Args Nice-to-have 20 min

Small, quick marks. Enums in particular come up more than people expect, because an enum can have fields, constructors and methods — it's a full class.

Key points

- Varargs `type... name` must be the **last** parameter and there can be only one; internally it's an array.
- Overloading with varargs can be ambiguous — avoid mixing `(int... x)` and `(int[] x)`.
- Enums are classes: they can have fields, constructors, methods, and can implement interfaces. Constructors are implicitly private.
- Enum constants are `public static final` instances; `values()` and `valueOf()` are compiler-generated.
- Enums are **safe for `==` comparison** (only one instance per constant) and are the best implementation of a thread-safe singleton.

Code

Varargs rules and gotchas

```
static int sum(int... nums) {      // nums IS an int[]
   int t = 0; for (int n : nums) t += n; return t;
}
sum();          // 0     - zero args allowed
sum(1, 2, 3);   // 6
sum(new int[]{1,2,3});  // 6 - you can pass an array directly

// static void bad(String a, int... n, int x) {}  // ERROR: varargs must be LAST
static void ok(String a, int x, int... n) {}      // fine

// Passing null is ambiguous with varargs:
// sum(null);      -> warning/ambiguity; cast it: sum((int[]) null)
```

Enums as real classes

```
enum Status {
   OPEN(1, "Open"), IN_PROGRESS(2, "Working"), CLOSED(3, "Done");

   private final int code;
   private final String label;

   Status(int code, String label) {   // implicitly private
       this.code = code; this.label = label;
   }
   public int getCode() { return code; }
   public String getLabel() { return label; }
}

Status s = Status.OPEN;
System.out.println(s.getCode() + " " + s.getLabel());   // 1 Open
System.out.println(Status.valueOf("CLOSED"));           // CLOSED
for (Status st : Status.values()) System.out.print(st + " ");  // OPEN IN_PROGRESS CLOSED
System.out.println(s.ordinal());                        // 0 (position - DON'T persist it)

// Enums in switch (exhaustive, no default needed for expressions)
String msg = switch (s) {
   case OPEN -> "New ticket";
   case IN_PROGRESS -> "Being worked on";
   case CLOSED -> "Finished";
};
```

```
Output:
1 Open
OPEN IN_PROGRESS CLOSED
0
```

Reading command-line arguments

```
// javac Args.java && java Args hello 42
public class Args {
   public static void main(String[] args) {
       System.out.println("count = " + args.length);   // 2
       for (int i = 0; i < args.length; i++)
           System.out.println("args[" + i + "] = " + args[i]);

       if (args.length > 1) {
           int n = Integer.parseInt(args[1]);          // "42" -> 42
       }
   }
}
```

```
Output:
count = 2
args[0] = hello
args[1] = 42
```

SAY IT LIKE THIS IN THE INTERVIEW

"Varargs is just an array parameter with a friendlier call site, and it must be last. Enums are full classes — constants are singletons, so == is safe, and they can carry fields, methods and even per-constant behaviour."

Flashcards

Q. Rules for varargs parameters?
A. Only one per method, must be the last parameter, internally it's an array, and it accepts zero arguments.

Q. Can an enum have a constructor and methods?
A. Yes — an enum is a class. Constructors are implicitly private, and constants can even override methods individually (constant-specific class bodies).

Q. Is == safe for comparing enums?
A. Yes — each constant is a single instance, so == is both correct and null-safe (unlike equals).

Q. What do values() and valueOf() do?
A. Compiler-generated: values() returns all constants as an array; valueOf(String) returns the constant with that exact name or throws IllegalArgumentException.

Q. Why is using ordinal() in a database a bad idea?
A. ordinal() is the declaration position; reordering or inserting a constant silently changes the persisted meaning. Store the name or an explicit code.

Q. Can an enum implement an interface?
A. Yes, but it can't extend a class (enums implicitly extend java.lang.Enum, and Java is single-inheritance).

Quiz

Q1. Which varargs declaration is legal?

[ ] A. void m(int... a, int... b)
[x] B. void m(String s, int... a)
[ ] C. void m(int... a, String s)
[ ] D. void m(int... a, int b)

Why: There can be only one varargs parameter and it must be the last one in the list.

Q2. Why is an enum the recommended way to write a Singleton in Java?

[ ] A. It's the shortest syntax

[x] B. The JVM guarantees only one instance, and it's safe against reflection and serialization attacks

[ ] C. Enums are faster

[ ] D. It allows lazy loading

Why: Joshua Bloch's 'Effective Java' recommends enum singletons: free serialization safety (no readResolve needed) and the JVM blocks reflective instantiation.

Q3. What does `Status.values()` return?

[ ] A. A String[] of names

[x] B. A Status[] of all constants in declaration order

[ ] C. An int[] of ordinals

[ ] D. A Map of name to constant

Why: The compiler adds a static values() method returning a fresh array of the constants in declaration order.

Day 2: OOP, Collections, Exceptions & Generics

The four pillars, the framework, and the error handling they'll drill you on

10 topics · 82 flashcards · 48 quiz questions · ~6.6 hours

> **Goal.** This is the day that decides the interview. OOP and Collections are where fresher rounds live or die — aim to explain every heading here out loud, in your own words, without notes.

Schedule

- Slot 1 (2h): OOP pillars, classes, constructors, this/super
- Slot 2 (2h): Overloading vs overriding, abstraction, abstract vs interface
- Slot 3 (2h): Object methods + equals/hashCode contract + exceptions
- Slot 4 (2h): Collections — List, Set, Map, HashMap internals
- Slot 5 (2h): Iterators, Comparable/Comparator, Generics + wildcards

1. The 4 Pillars of OOP (with real examples) Must-know 35 min

The opener is almost always "what are the four pillars of OOP?" — and the follow-up is "give me a real example of each." Textbook definitions alone sound rehearsed; a one-line concrete example each sounds like someone who's written code.

Key points

- **Encapsulation** — bundle data + behaviour, hide internals behind accessors. `private` fields with getters/setters, so you can add validation later without breaking callers.
- **Inheritance** — `extends`, IS-A relationship, code reuse. Single inheritance for classes in Java (one superclass), multiple inheritance of *type* via interfaces.
- **Polymorphism** — one interface, many forms. Compile-time (method overloading) and runtime (method overriding + dynamic dispatch).
- **Abstraction** — expose *what* an object does, hide *how*. Achieved with abstract classes and interfaces.
- If they ask for a 5th: **Association/Aggregation/Composition** — "has-a" relationships, with composition being the strong one (child can't exist without parent).

Reference

Pillar	One-liner	Java mechanism	Real example
Encapsulation	Hide the data, expose behaviour	private fields + getters/setters	BankAccount hides `balance`; deposit() validates
Inheritance	Child reuses parent's code	`extends` (IS-A)	SavingsAccount extends BankAccount
Polymorphism	Same call, different behaviour	Overloading (compile) / Overriding (runtime)	List l = new ArrayList<>() — code works for any List
Abstraction	What, not how	abstract class / interface	PaymentService interface; StripePayment implements it

Code

All four pillars in one small design

```
// ABSTRACTION: what a payment does, not how
interface PaymentService {
   boolean pay(double amount);        // implicitly public abstract
}

// INHERITANCE + implementation
abstract class CardPayment implements PaymentService {
   protected String cardNumber;       // protected: visible to subclasses
   public boolean pay(double amount) {
       if (!validate()) return false;   // shared behaviour
       return doCharge(amount);         // polymorphic: subclass decides how
   }
   private boolean validate() { return cardNumber != null && cardNumber.length() == 16; }
   protected abstract boolean doCharge(double amount);
}

class CreditCardPayment extends CardPayment {
   @Override protected boolean doCharge(double amount) {
       System.out.println("Charging credit card " + amount);
       return true;
   }
}
class DebitCardPayment extends CardPayment {
   @Override protected boolean doCharge(double amount) {
       System.out.println("Charging debit card " + amount);
       return true;
   }
}

// POLYMORPHISM: same call, different behaviour at runtime
PaymentService p = new CreditCardPayment();
p.pay(499.0);                     // -> "Charging credit card 499.0"
p = new DebitCardPayment();
p.pay(499.0);                     // -> "Charging debit card 499.0"

// ENCAPSULATION
class BankAccount {
   private double balance;                        // hidden
   public double getBalance() { return balance; }
   public void deposit(double amt) {              // behaviour controls state
       if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
       balance += amt;
   }
}
```

Output:
```
Charging credit card 499.0
Charging debit card 499.0
```

Composition over inheritance — the design answer that impresses

```
// Fragile: inheritance locks Engine behaviour into the class hierarchy
class Car extends Engine { }         // Car IS-A Engine? No. Wrong relationship.

// Better: composition - Car HAS-A Engine, swap it at will
class Car {
   private Engine engine;                         // has-a
   Car(Engine engine) { this.engine = engine; }  // inject it
   void drive() { engine.start(); }
}
interface Engine { void start(); }
class PetrolEngine implements Engine { public void start() { System.out.println("vroom"); } }
class ElectricEngine implements Engine { public void start() { System.out.println("whirr"); }
   }

new Car(new ElectricEngine()).drive();   // whirr  <- swap without touching Car
```

Output:
```
whirr
```

> "Prefer composition over inheritance" — inheritance breaks encapsulation and couples you to a parent's implementation. Composition + interfaces is what DI/Spring is built on.

SAY IT LIKE THIS IN THE INTERVIEW

"Encapsulation hides state behind behaviour, inheritance reuses code through an IS-A relationship, polymorphism lets the same call take many forms — overloading at compile time, overriding at runtime — and abstraction exposes what an object does via interfaces and abstract classes. In practice I favour composition over inheritance, because it keeps classes decoupled and testable."

Flashcards

Q. Name the 4 pillars of OOP with a one-line example each.
A. Encapsulation (private balance + deposit()), Inheritance (SavingsAccount extends BankAccount), Polymorphism (PaymentService p = new CreditCardPayment()), Abstraction (interface PaymentService).

Q. Why prefer composition over inheritance?
A. Inheritance couples the subclass to the superclass's implementation and breaks encapsulation. Composition (has-a) lets you swap behaviour at runtime and keeps classes independent.

Q. What's the difference between association, aggregation and composition?
A. Association is a loose 'uses-a'. Aggregation is a weak 'has-a' where parts outlive the whole (Car–Wheel). Composition is a strong 'has-a' where parts die with the whole (House–Room).

Q. Does Java support multiple inheritance?
A. Not for classes (one extends only). But a class can implement multiple interfaces, which gives multiple inheritance of TYPE — and since Java 8, of behaviour too via default methods.

Q. Compile-time vs runtime polymorphism?
A. Compile-time = method overloading (decided by the compiler from the argument types). Runtime = method overriding (decided by the JVM from the actual object type).

Q. What problem does encapsulation actually solve?
A. It protects invariants — validation lives in one place (setters/methods), internals can change without breaking callers, and fields can be made read-only.

Quiz

Q1. Which pillar is demonstrated by `List<String> l = new ArrayList<>();`?

[ ] A. Encapsulation
[ ] B. Inheritance
[x] C. Polymorphism (coding to the interface)
[ ] D. Abstraction only

Why: Programming to the interface is polymorphism: the variable's declared type is List, but calls dispatch to ArrayList's implementation. Swap in LinkedList and nothing changes.

Q2. Why is `class Car extends Engine` considered poor design?

[ ] A. Because Java forbids it
[x] B. Because Car is not an Engine — it HAS an Engine; inheritance models IS-A
[ ] C. Because Engine should be final
[ ] D. Because it's slower

Why: Inheritance encodes an IS-A relationship. Car has an Engine, so composition is the right model — and it lets you swap engine implementations.

Q3. Which is NOT one of the classic four pillars?

[ ] A. Encapsulation
[ ] B. Polymorphism
[x] C. Compilation
[ ] D. Abstraction

Why: The four are Encapsulation, Inheritance, Polymorphism, Abstraction. 'Compilation' isn't a pillar.

2. Constructors, this & super Core 30 min

Constructor chaining, `this()` vs `super()`, and "why did the compiler complain there's no default constructor" — all standard fresher territory.

Key points

- A constructor has **no return type** (not even void) and its name **must** match the class name.
- If you write **any** constructor, the compiler stops generating the default no-arg one — add it back explicitly if a framework needs it (JPA/Hibernate/Jackson do).
- `this()` calls another constructor **in the same class**; `super()` calls the parent's. Both must be the **first statement** — so you can't use both in one constructor.
- If you don't write `super()` or `this()`, the compiler inserts `super()` — a call to the parent's **no-arg** constructor. If the parent has none, you get a compile error.
- Constructors are **not inherited** and **cannot be overridden** (they're not methods). They also aren't `final`/`static`/`abstract`.
- **Constructor chaining** runs Object → parent → child, so parent state is always initialised first.

Code

Chaining with this() and super()

```
class Person {
   String name; int age;
   Person() { this("Unknown", 0); }                // -> delegates to the 2-arg ctor
   Person(String name) { this(name, 0); }          // -> delegates again
   Person(String name, int age) { this.name = name; this.age = age; }
}

class Employee extends Person {
   double salary;
   Employee(String name, int age, double salary) {
       super(name, age);      // MUST be first; initialises parent state
       this.salary = salary;
   }
   Employee(double salary) {
       this("Temp", 18, salary);   // delegate to the 3-arg ctor
   }
}
```

The 'no default constructor' error everyone hits once

```
class Parent {
   Parent(String x) { }      // only a 1-arg ctor -> NO implicit no-arg ctor
}
class Child extends Parent {
   Child() {
       // Implicit super(); inserted by the compiler -> COMPILE ERROR,
       // because Parent has no no-argument constructor.
   }
}
// Fix 1: call the parent's constructor explicitly
class Child extends Parent {
   Child() { super("default"); }
}
// Fix 2: add a no-arg constructor to Parent
class Parent {
   Parent() { }
   Parent(String x) { }
}
```

this vs super — quick reference

```
class Base { int v = 1; void show() { System.out.println("Base"); } }
class Derived extends Base {
   int v = 2;
   void show() { System.out.println("Derived"); }

   void demo() {
       System.out.println(v);          // 2   -> this.v (field shadowing)
       System.out.println(this.v);     // 2
       System.out.println(super.v);    // 1   -> parent's field
       show();                         // Derived
       this.show();                    // Derived
       super.show();                   // Base  <- escape the override
   }
   Derived() { super(); }              // parent ctor, must be line 1
   Derived(int x) { this(); }          // own ctor, must be line 1
}
```

Output:

```
2
2
1
Derived
Derived
Base
```

Fields are NOT polymorphic — field access is resolved by the reference type. Only instance methods are dynamically dispatched. That's a classic trap.

Copy constructor & private constructor

```
class Point {
   int x, y;
   Point(int x, int y) { this.x = x; this.y = y; }
   Point(Point other) { this(other.x, other.y); }   // copy constructor
}

class Utility {
   private Utility() { throw new AssertionError(); }  // prevent instantiation
}
// A private ctor also enables the Singleton and Factory patterns:
class Config {
   private static final Config INSTANCE = new Config();
   private Config() { }
   public static Config getInstance() { return INSTANCE; }
}
```

SAY IT LIKE THIS IN THE INTERVIEW

"Constructors have no return type and aren't inherited or overridable. this() and super() must be the first statement, so a constructor can only do one or the other. If I don't write one, the compiler inserts a call to super() — which is why a subclass fails to compile when the parent only has a parameterised constructor."

Flashcards

Q. Why can't this() and super() appear in the same constructor?
A. Both are required to be the very first statement, so they're mutually exclusive.

Q. Does the compiler always add a default constructor?
A. Only if you declare NO constructors at all. Write any constructor and the no-arg one disappears until you add it back.

Q. Are constructors inherited or overridable?
A. Neither. Constructors aren't members in that sense — a subclass must define its own and chain to a parent one with super().

Q. What is constructor chaining?
A. Each constructor delegates up the hierarchy (super) or across (this), guaranteeing Object → parent → child initialisation order.

Q. Are fields polymorphic like methods?
A. No. Field access is resolved at compile time by the *reference* type; only instance method calls use dynamic dispatch.

Q. How do you prevent a class from being instantiated?
A. Make the constructor private (optionally throwing AssertionError) — used for utility classes and singletons.

Q. Why do JPA entities and Jackson DTOs need a no-arg constructor?
A. Frameworks instantiate the object reflectively before populating fields, so they need a no-argument constructor (it can be protected/private for JPA).

Quiz

Q1. What happens if a class has only a parameterised constructor and its subclass constructor doesn't call super(...)?

[ ] A. Compiles fine
[x] B. Compile error — the implicit super() can't find a no-arg parent constructor
[ ] C. Runtime error
[ ] D. The parent constructor is skipped

> **Why:** The compiler inserts `super();` as the first statement of every constructor that doesn't start with this()/super(...). With no no-arg parent ctor, that's a compile-time error.

Q2. What is the output? `class A { A(){ System.out.print("A"); } }` `class B extends A { B(){ System.out.print("B"); } }` `new B();`

[x] A. AB
[ ] B. BA
[ ] C. B
[ ] D. A

> **Why:** The implicit super() runs A's constructor first, then B's body — so "AB". Parent state is always built before child state.

Q3. Can a constructor be private, and why would you want that?

[ ] A. No, never
[x] B. Yes — for singletons, factories and utility classes to control instantiation
[ ] C. Yes, but only in abstract classes
[ ] D. Yes, but subclasses can't exist

> **Why:** A private constructor prevents outside instantiation (singleton, static factory) and also prevents subclassing, since no subclass ctor could call super().

Q4. What is a copy constructor?

[ ] A. A constructor that clones via Object.clone()
[x] B. A constructor taking an object of the same class and copying its state
[ ] C. The compiler-generated constructor
[ ] D. A constructor annotated @Override

> **Why:** `Point(Point other) { this(other.x, other.y); }` — an explicit, controlled alternative to clone().

3. Overloading vs Overriding & Dynamic Dispatch Must-know 40 min

The most-asked OOP question after the four pillars. Learn the rules as a table, because interviewers check them one by one. The follow-up is always: "can you override a static/private/final method?"

Key points

- **Overloading** = same method name, different parameter list, **same or different class**. Resolved at **compile time** by the declared argument types.
- **Overriding** = same signature in a subclass. Resolved at **runtime** by the actual object type (dynamic method dispatch).
- Overriding rules: same name, same parameter list, return type must be the same or a **covariant** subtype; access **cannot be narrowed** (protected → public OK, public → protected not); checked exceptions **cannot be broadened**.
- You **cannot** override `final`, `static` or `private` methods. `private` methods aren't even visible to the subclass — it's a *new* method.
- Constructors can be overloaded but not overridden. Return type alone is NOT enough to overload.
- **A subclass method can't throw a broader checked exception** than the parent's; it may throw narrower ones or none. Unchecked exceptions are unrestricted.

Reference

Aspect	Overloading	Overriding
Where	Same class (or subclass)	Subclass of an inherited method
Signature	Must DIFFER (params)	Must be IDENTICAL
Return type	Can be anything	Same or covariant subtype
Binding	Compile time (static)	Runtime (dynamic dispatch)
Access modifier	Any	Cannot be more restrictive
Exceptions	Any	Cannot throw broader checked exceptions
private/static/final	Can be overloaded	Cannot be overridden
Polymorphism	Compile-time polymorphism	Runtime polymorphism

Code

Overriding rules that compile vs fail

```
class Parent {
   public Number getValue() { return 1; }
   protected void process() throws IOException { }
   public final void locked() { }
   public static void util() { }
   private void hidden() { }
}

class Child extends Parent {
   @Override public Integer getValue() { return 2; }   // OK: covariant return
   @Override public void process() { }                 // OK: no exception, wider access
   // @Override protected void process() { }           // ERROR: narrowing access
   // @Override public void process() throws Exception // ERROR: broader checked ex
   // @Override public void locked() { }               // ERROR: final
   // @Override public static void util() { } // ERROR: static -> hiding, not override

   // NOT an override - just a new method in Child (private methods aren't inherited)
   private void hidden() { }

   // Overloading inside the same class works on return-type-independent params:
   void log(String s) { }
   void log(String s, int n) { }
   // int log(String s) { }   // ERROR: same params -> not a valid overload
}

// Dynamic dispatch in action
Parent p = new Child();
System.out.println(p.getValue());   // 2  -> Child's version runs
```

Output:
```
2
```

The classic overload-resolution trap

```
class Overloads {
   static void f(Object o) { System.out.println("Object"); }
   static void f(String s) { System.out.println("String"); }
   static void f(Integer i) { System.out.println("Integer"); }
}

Object o = "hello";
Overloads.f(o);        // "Object"  <- chosen by the DECLARED type, not the runtime type
Overloads.f("hello");  // "String"
Overloads.f(null);     // "String"  <- the MOST SPECIFIC applicable type wins

// Nothing to do with overriding: overloading is a compile-time decision.
// Overriding would have picked the runtime method; overloading cannot.
```

Output:
```
Object
String
String
```

> With `f(null)`, if there were two unrelated specific types (String and Integer) it would be ambiguous and fail to compile.

@Override — why you should never skip it

```
class Child extends Parent {
   @Override public String toString() { return "Child"; }
   // If Parent had no toString(Object) and you wrote:
   // @Override public String tostring(String s) { }  -> COMPILE ERROR (typo caught!)
}
// Without @Override the typo version would silently become a NEW method
// that is never called -> the bug nobody finds in code review.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Overloading is compile-time polymorphism — the compiler picks the method from the declared argument types. Overriding is runtime polymorphism — the JVM picks it from the actual object type. When overriding, the signature is identical, the return type can be a covariant subtype, I can widen access but never narrow it, and I can't throw broader checked exceptions. And I always add @Override so the compiler catches my typos."

Flashcards

Q. Overloading vs overriding in one sentence each?
A. Overloading: same name, different parameters, resolved at compile time. Overriding: same signature in a subclass, resolved at runtime.

Q. Can you override a private method?
A. No. Private methods aren't inherited, so a same-signature method in the subclass is a brand-new method, not an override.

Q. Can you override a static method?
A. No — a static method with the same signature hides the parent's. The call is resolved by the declared type at compile time, not dynamically.

Q. Can you change the return type when overriding?
A. Yes, if it's a covariant (subtype) return — parent returns Number, child may return Integer. Primitives must match exactly.

Q. Can an overriding method throw a broader checked exception?
A. No. It may throw the same, a narrower checked exception, none, or any unchecked exception.

Q. Can an overriding method reduce visibility?
A. No. Public must stay public; protected can become public; you can never go more restrictive.

Q. Which is chosen for `Object o = "hi"; f(o);` with f(Object) and f(String)?
A. f(Object) — overloading is resolved from the declared (static) type of the argument, not the runtime type.

Q. What does @Override actually do?
A. It's a compile-time marker annotation: the compiler errors if the method doesn't actually override anything — catching typos and signature drift.

Quiz

Q1. Which method CANNOT be overridden?

[ ] A. A public instance method
[ ] B. A protected instance method
[x] C. A final method
[ ] D. A method in a non-final class

> **Why:** final methods cannot be overridden (by design — it locks the implementation). static and private methods also can't be overridden.

Q2. Is this valid overriding? Parent: `public void m() throws IOException {}` Child: `public void m() throws Exception {}`

[ ] A. Yes, subclasses can throw anything
[x] B. No — Exception is broader than IOException (a checked exception)
[ ] C. Yes, because Exception is unchecked
[ ] D. Only if the method is abstract

> **Why:** An overriding method may not declare broader checked exceptions. It could throw FileNotFoundException (narrower) or nothing at all.

Q3. What's the result of `Parent p = new Child(); p.m();` where Child overrides m()?

[ ] A. Parent's m()
[x] B. Child's m()
[ ] C. Compile error
[ ] D. Depends on the access modifier

Why: Dynamic method dispatch: the JVM uses the runtime object type (Child) for instance method calls, regardless of the reference type.

Q4. Can two methods differ only by return type?

[ ] A. Yes, that's valid overloading
[x] B. No — the parameter list must differ
[ ] C. Yes, if one returns a subtype
[ ] D. Yes, in Java 17+

Why: Overloading requires a different parameter list (type, count or order). Return type alone doesn't distinguish methods, so it's a compile error.

Q5. What is 'method hiding'?

[x] A. Declaring a static method in a subclass with the same signature as a parent's static method
[ ] B. Using private constructors
[ ] C. Overriding with @Override
[ ] D. Shadowing a local variable

Why: Static methods are hidden, not overridden: which one runs depends on the reference type at compile time.

4. Abstract Class vs Interface Must-know 40 min

Guaranteed question, and the interesting part is how much the answer changed after Java 8. If you only memorise "interfaces are 100% abstract" you'll sound like a 2013 textbook.

Key points

- **Abstract class** — can have instance state, constructors, concrete and abstract methods. Use it for an IS-A relationship with shared code.
- **Interface** — a contract. Since Java 8 it can have `default` and `static` methods with bodies; since Java 9, `private` methods too.
- A class `extends` **one** abstract class but `implements` **many** interfaces — that's how Java gets multiple inheritance of type.
- Interface fields are implicitly `public static final`; methods are implicitly `public abstract` (unless default/static/private).
- **Functional interface** = exactly one abstract method → can be a lambda target. Mark it `@FunctionalInterface`.
- **Marker interface** = no methods at all (Serializable, Cloneable, Remote) — used to tag classes for the JVM/frameworks. Mostly replaced by annotations now.

Reference

Feature	Abstract class	Interface
Keyword	abstract class / extends	interface / implements
Multiple inheritance	No (one class)	Yes (many interfaces)
Methods	abstract + concrete	abstract, default, static, private (Java 9+)
Fields	Any (instance state)	public static final constants only
Constructors	Yes	No
Access modifiers	Any	Members are implicitly public
State	Can hold object state	No instance state
When to use	Shared code + IS-A + template method	Capability/contract, decoupled design, DI

Code

Interface superpowers since Java 8

```
interface Vehicle {
   int MAX_SPEED = 120;              // implicitly public static final

   void start();                     // implicitly public abstract

   default void honk() {             // Java 8: implementation provided
       System.out.println("Beep!");
   }

   static boolean isFast(int speed) {// Java 8: static, called as Vehicle.isFast()
       return speed > MAX_SPEED;
   }

   private void log(String msg) {    // Java 9: helper for default methods
       System.out.println("[log] " + msg);
   }

   default void service() {          // default methods can use private helpers
       log("servicing");
       System.out.println("Service done");
   }
}

class Bike implements Vehicle {
   @Override public void start() { System.out.println("Bike started"); }
   @Override public void honk() { System.out.println("Ting ting!"); }  // may override
}

Bike b = new Bike();
b.start();                // Bike started
b.honk();                 // Ting ting!
b.service();              // [log] servicing / Service done
Vehicle.isFast(150);      // true
```

Output:

```
Bike started
Ting ting!
[log] servicing
Service done
```

Default method conflict — the diamond problem and its fix

```
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }

// class C implements A, B { }   // ERROR: inherits unrelated defaults for hello()

class C implements A, B {
   @Override public void hello() {
       A.super.hello();        // MUST disambiguate explicitly
       System.out.println("C");
   }
}
new C().hello();   // A / C

// Rule: a CLASS method always wins over an interface default.
//       Otherwise the most specific interface wins; ties must be resolved manually.
```

Output:

```
A
C
```

Template method pattern with an abstract class

```
abstract class DataProcessor {          // skeleton: fixed algorithm, variable steps
   public final void process() {        // final -> subclasses can't reorder it
       read();
       transform();                     // abstract -> subclass supplies it
       save();
   }
   private void read()   { System.out.println("reading"); }
   private void save()   { System.out.println("saving"); }
   protected abstract void transform();
}

class UpperCaseProcessor extends DataProcessor {
   @Override protected void transform() { System.out.println("UPPERCASING"); }
}
new UpperCaseProcessor().process();   // reading / UPPERCASING / saving
```

Output:
```
reading
UPPERCASING
saving
```

Functional interfaces & the built-in ones you must know

```
@FunctionalInterface      // compiler enforces: exactly ONE abstract method
interface Calculator {
   int compute(int a, int b);
   // default and static methods don't count toward the single abstract method
   default int twice(int a) { return compute(a, a); }
}

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;
System.out.println(add.compute(3, 4));   // 7

// java.util.function - memorise these four:
Predicate<String> isEmpty  = s -> s.isEmpty();      // T -> boolean
Function<String, Integer> len = String::length;     // T -> R
Consumer<String> printer  = System.out::println;    // T -> void
Supplier<Double> random   = Math::random;           // () -> T
// Plus: BiFunction<T,U,R>, UnaryOperator<T> (T->T), BinaryOperator<T> (T,T->T)
```

Output:
```
7
```

SAY IT LIKE THIS IN THE INTERVIEW

"An abstract class models an IS-A relationship with shared state and a constructor — I use it for template-method style skeletons. An interface is a capability contract; since Java 8 it can carry default and static implementations, and a class can implement many of them, which is how Java supports multiple inheritance of type. For DI and testability I program to interfaces."

Flashcards

Q. When would you choose an abstract class over an interface?
A. When you need shared instance state, a constructor, or a common skeleton of concrete methods for closely related classes (template method pattern).

Q. Can an interface have a constructor?
A. No — interfaces have no instance state and can't be instantiated; only classes have constructors.

Q. What is a default method and why was it introduced?
A. A method with a body in an interface (Java 8). It lets interfaces evolve (e.g. Stream added to Collection) without breaking every existing implementation.

Q. How do you resolve a default-method conflict from two interfaces?
A. Override the method in the class and call the one you want explicitly: `A.super.method()`.

Q. What is a functional interface?
A. An interface with exactly one abstract method — the target type for lambdas. Annotate with @FunctionalInterface so the compiler enforces it.

Q. Name the four core java.util.function interfaces.
A. Predicate<T> (T→boolean), Function<T,R> (T→R), Consumer<T> (T→void), Supplier<T> (()→T).

Q. What is a marker interface? Give two examples.
A. An interface with no methods used to tag a class: Serializable, Cloneable, Remote. Annotations have largely replaced them.

Q. Are interface methods public by default? Fields?
A. Methods are implicitly public (abstract/default/private variants aside). Fields are implicitly public static final — constants only.

Q. Can an interface extend multiple interfaces?
A. Yes — unlike classes, an interface can extend several interfaces.

Quiz

Q1. What is the default (implicit) modifier of a field declared in an interface?

[ ] A. private
[ ] B. protected
[x] C. public static final
[ ] D. package-private

> **Why:** Interface fields are implicitly public static final — they're constants, and you must initialise them at declaration.

Q2. A class implements two interfaces that both declare `default void hello()`. What happens?

[ ] A. It compiles and picks the first interface
[x] B. Compile error unless the class overrides hello()
[ ] C. Runtime error
[ ] D. The method is randomly chosen

> **Why:** The diamond conflict must be resolved explicitly: override hello() and delegate with `A.super.hello()` or `B.super.hello()`.

Q3. Which is true about a class that extends a base class and implements an interface, where both provide the same method?

[ ] A. The interface default wins
[x] B. The class (superclass) method wins — classes beat interfaces
[ ] C. Compile error
[ ] D. The most specific interface wins

> **Why:** The rule is: class wins over interface. A superclass method takes precedence over any inherited default method.

Q4. Can an interface contain a private method?

[ ] A. No, never
[x] B. Yes, since Java 9, as a helper for default methods
[ ] C. Yes, since Java 8
[ ] D. Only static private methods

> **Why:** Java 9 added private interface methods so default methods can share helper code without exposing it.

Q5. Which of these is a valid functional interface?

[ ] A. interface X { void a(); void b(); }
[x] B. interface X { void a(); default void b(){} static void c(){} }
[ ] C. interface X { }
[ ] D. interface X { void a(); void a(int i); }

Why: Exactly one abstract method. default and static methods don't count, so option 2 qualifies. Option 4 has two abstract methods.

5. Object Methods & the equals/hashCode Contract Must-know 40 min

`equals` and `hashCode` are the most technically-demanding fresher question there is. Learn the contract as five numbered rules and you'll never be caught out — and it explains why HashMap misbehaves when you get it wrong.

Key points

- Every class inherits from `Object`: `toString()`, `equals()`, `hashCode()`, `getClass()`, `clone()`, `notify()/wait()`, `finalize()` (deprecated since 9).
- **The contract**: if `a.equals(b)` then `a.hashCode() == b.hashCode()` — always. The reverse is NOT required (hash collisions are allowed).
- If you override `equals()`, you MUST override `hashCode()`, or HashMap/HashSet will lose your objects.
- `equals()` must be **reflexive, symmetric, transitive, consistent**, and `x.equals(null)` must be `false`.
- Use `Objects.equals(a, b)` and `Objects.hash(...)` — null-safe one-liners.
- **Never use a mutable field in hashCode/equals** — if the field changes after insertion into a HashSet, the object becomes unreachable.

Reference

Object method	Default behaviour	Why/when you override
toString()	ClassName@hexHash	Logging and debugging — always override for readability
equals(Object)	Reference equality (==)	When logical equality matters (value objects, DTOs)
hashCode()	Identity-based int	MUST override whenever equals is overridden
clone()	Shallow field copy	Rarely — prefer copy constructors; requires Cloneable
getClass()	Returns the runtime Class	final — cannot be overridden
finalize()	Called by GC before reclaim	Deprecated (Java 9) — use try-with-resources / Cleaner

Code

A correct equals + hashCode (Java 7+ style)

```
class Employee {
   private int id;
   private String name;

   @Override
   public boolean equals(Object o) {
       if (this == o) return true;              // 1. identity shortcut
       if (!(o instanceof Employee)) return false; // 2. type check (also handles null)
       Employee that = (Employee) o;
       return id == that.id && Objects.equals(name, that.name);
   }

   @Override
   public int hashCode() {
       return Objects.hash(id, name);           // same fields as equals!
   }

   @Override
   public String toString() {
       return "Employee{id=" + id + ", name='" + name + "'}";
   }
}
// Use `instanceof` (not getClass()) so subclasses can still be equal.
// Use `==` for primitives, Objects.equals() for references.
```

What breaks when you forget hashCode() — the HashSet bug

```
class Bad {
   int id;
   Bad(int id) { this.id = id; }
   @Override public boolean equals(Object o) {
       return o instanceof Bad && ((Bad) o).id == this.id;
   }
   // hashCode() NOT overridden -> uses Object's identity hash
}

Set<Bad> set = new HashSet<>();
set.add(new Bad(1));
System.out.println(set.contains(new Bad(1)));   // false!!!
// Two equal objects land in different buckets because their identity
// hashCodes differ, so HashSet never finds the match.

// With hashCode() implemented correctly -> true
```

Output:
```
false
```

> This is the canonical interview demo. Say it out loud: "equal objects must have equal hash codes, otherwise hash-based collections break."

Mutable-field trap: losing an object inside a HashSet

```
class Mutable {
   int id;
   Mutable(int id) { this.id = id; }
   @Override public int hashCode() { return Objects.hash(id); }
   @Override public boolean equals(Object o) {
       return o instanceof Mutable && ((Mutable) o).id == this.id;
   }
}

Set<Mutable> set = new HashSet<>();
Mutable m = new Mutable(1);
set.add(m);
m.id = 99;                          // mutating a field used in hashCode
System.out.println(set.contains(m));         // false - different bucket now
set.remove(m);                               // fails -> the entry is orphaned
// Lesson: use FINAL fields in equals/hashCode, or never mutate keys in a set.
```

Output:
```
false
```

Shallow vs deep copy (clone and copy constructors)

```
class Address { String city; Address(String c) { city = c; } }

class Person implements Cloneable {
   String name;
   Address addr;                // reference field -> shallow copy shares it!

   @Override protected Object clone() throws CloneNotSupportedException {
       return super.clone();     // shallow: addr is shared
   }
   // Deep copy version:
   Person deepCopy() {
       Person p = new Person();
       p.name = this.name;
       p.addr = new Address(this.addr.city);   // clone the nested object
       return p;
   }
}

Person p1 = new Person(); p1.addr = new Address("Hyd");
Person p2 = (Person) p1.clone();
p2.addr.city = "Blr";
System.out.println(p1.addr.city);   // Blr  <- shallow copy leaked the change!
```

Output:
```
Blr
```

> Shallow copy duplicates the top-level object but shares nested references. Deep copy recursively duplicates everything. Prefer copy constructors — clone()/Cloneable is widely considered broken.

SAY IT LIKE THIS IN THE INTERVIEW

"Every class inherits from Object. The key contract: if two objects are equal their hash codes must be equal, so I always override equals and hashCode together, using the same fields. I use Objects.equals and Objects.hash for null safety, instanceof for the type check, and I keep those fields immutable so an object can't go missing inside a HashMap."

Flashcards

Q. State the equals/hashCode contract.
A. If a.equals(b) then a.hashCode() == b.hashCode(). The converse isn't required (collisions are legal). Equal hashCodes don't imply equality.

Q. What breaks if you override equals but not hashCode?
A. Hash-based collections (HashMap, HashSet) put equal objects in different buckets, so contains/get/remove return false even though the object is present.

Q. List the 5 properties equals() must satisfy.
A. Reflexive (x.equals(x)), symmetric (x.equals(y) ⇒ y.equals(x)), transitive, consistent across calls, and x.equals(null) is false.

Q. Why use instanceof instead of getClass() in equals()?
A. getClass() breaks equality for subclasses and proxies (Hibernate/Spring use proxies); instanceof allows subtype equality. Either is acceptable if you're consistent, but instanceof is the common convention.

Q. Shallow copy vs deep copy?
A. Shallow duplicates the top-level object but shares nested references; deep recursively copies all referenced objects.

Q. Why is clone() considered broken?
A. Cloneable is a marker interface with no clone() method, clone() is protected, it bypasses constructors, and it does a shallow copy by default. Prefer copy constructors or factory methods.

Q. Is finalize() recommended?
A. No — deprecated since Java 9. It's unpredictable and slows GC. Use try-with-resources and AutoCloseable.

Q. Which Object methods are final?
A. getClass(), notify(), notifyAll(), wait() — they can't be overridden.

Quiz

Q1. You override equals() but not hashCode(). What is the result?

[ ] A. Compile error
[x] B. Hash-based collections misbehave — equal objects may not be found
[ ] C. Nothing; hashCode is optional
[ ] D. equals() stops working

> **Why:** It compiles, but you've broken the contract: two equal objects get different (identity) hash codes, so HashSet/HashMap lookups fail.

Q2. Which equals() property is violated here? `class A { int x; public boolean equals(Object o){ return o instanceof A && ((A)o).x == x; } }` `class B extends A { int y; public boolean equals(Object o){ return o instanceof B && ((B)o).y == y; } }`

[ ] A. Reflexive
[x] B. Symmetric
[ ] C. Transitive
[ ] D. Consistent

> **Why:** `a.equals(b)` can be true (b is an A, x matches) while `b.equals(a)` is false (a is not a B). Also a transitivity problem. Fix by using getClass() or `canEqual` — which is why many teams let the IDE generate equals.

Q3. What does `Object.hashCode()` return by default?

[ ] A. 0
[x] B. An identity-based hash (not necessarily the memory address)
[ ] C. The class name hash
[ ] D. A random number each call

> **Why:** The default is derived from the object's identity — typically the address converted to an int, though the spec allows other schemes. It stays stable for the object's lifetime.

Q4. After adding an object to a HashSet, you mutate a field used in hashCode(). What happens?

[ ] A. Nothing — the set updates the bucket
[x] B. The object becomes unfindable/removable — it's in the wrong bucket
[ ] C. ConcurrentModificationException
[ ] D. It's automatically rehashed

> **Why:** HashMap never rehashes existing keys. The entry sits in the bucket computed from the old hash, so lookups with the new hash miss it — a memory leak in practice.

Q5. What's the correct way to compare two objects that might be null?

[ ] A. a.equals(b)
[x] B. Objects.equals(a, b)
[ ] C. a == b
[ ] D. a.hashCode() == b.hashCode()

> **Why:** Objects.equals handles nulls on either side (returns true if both are null). a.equals(b) throws NPE when a is null.

6. Exception Handling Must-know 45 min

A whole round can be built on exceptions: the hierarchy, checked vs unchecked, and the `finally` / `return` gotchas. Learn the hierarchy diagram and the six gotcha outputs.

Key points

- `Throwable` → `Error` (JVM-level, don't catch) and `Exception` → `RuntimeException` (unchecked).
- **Checked** exceptions (IOException, SQLException, ClassNotFoundException) are verified at **compile time** — you must catch or declare `throws`.
- **Unchecked** (RuntimeException and its children: NPE, ArrayIndexOutOfBounds, ArithmeticException, IllegalArgumentException, ClassCastException, NumberFormatException, ConcurrentModificationException) are not checked by the compiler.
- `Error` (OutOfMemoryError, StackOverflowError, NoClassDefFoundError) is fatal — never catch it.
- `finally` runs even after a `return` — but a `return` inside `finally` **overrides** the earlier return.
- **try-with-resources** (Java 7) auto-closes anything `AutoCloseable`; resources are closed in reverse order, and suppressed exceptions are attached to the primary one.

Reference

Type	Checked at compile time?	Examples	Handle or declare?
Checked Exception	Yes	IOException, SQLException, FileNotFoundException, InterruptedException, ClassNotFoundException	Must catch or declare throws
Unchecked (RuntimeException)	No	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, ClassCastException, IllegalArgumentException, NumberFormatException	Optional — usually fix the bug instead
Error	No	OutOfMemoryError, StackOverflowError, NoClassDefFoundError	Do not catch

Code

The hierarchy, as text

```
Throwable
├── Error                    (unchecked, fatal - never catch)
│   ├── OutOfMemoryError
│   ├── StackOverflowError
│   └── NoClassDefFoundError
└── Exception
    ├── RuntimeException      (unchecked)
    │   ├── NullPointerException
    │   ├── ArithmeticException          <- / by zero (int)
    │   ├── ArrayIndexOutOfBoundsException
    │   ├── ClassCastException
    │   ├── IllegalArgumentException
    │   │   └── NumberFormatException    <- Integer.parseInt("abc")
    │   ├── IllegalStateException
    │   └── ConcurrentModificationException
    └── checked exceptions
        ├── IOException
        │   └── FileNotFoundException
        ├── SQLException
        ├── ClassNotFoundException
        ├── InterruptedException
        └── ParseException
```

The six outputs you must know by heart

```
// 1. finally overrides a return
static int f1() {
   try { return 1; }
   finally { return 2; }        // WARNING: never do this
}
System.out.println(f1());        // 2

// 2. finally runs even when an exception escapes
static void f2() {
   try { throw new RuntimeException("boom"); }
   finally { System.out.println("finally ran"); }
}
// prints "finally ran", then the exception propagates

// 3. return value is SNAPSHOTTED before finally mutates it
static int f3() {
   int x = 10;
   try { return x; }            // the value 10 is remembered here
   finally { x = 20; }          // too late
}
System.out.println(f3());        // 10

// 4. System.exit() is the one thing finally can't stop
try { System.exit(0); } finally { System.out.println("never printed"); }

// 5. catch order: subclass BEFORE superclass
try { ... }
catch (FileNotFoundException e) { }   // specific first
catch (IOException e) { }             // general after
// catch (IOException e){} catch (FileNotFoundException e){}  -> COMPILE ERROR

// 6. multi-catch (Java 7) - the parameter is implicitly final
catch (IOException | SQLException e) { log(e); }
```

```
Output:
2
10
```

try-with-resources — the modern way, and why it's better

```
// OLD: verbose, and a bug if close() itself throws
BufferedReader br = null;
try { br = new BufferedReader(new FileReader("a.txt")); }
finally { if (br != null) try { br.close(); } catch (IOException e) { } }

// NEW (Java 7+): anything AutoCloseable closes automatically, in reverse order
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"));
    Connection conn = DriverManager.getConnection(url)) {
   return br.readLine();
}   // conn closes first, then br - even if the body threw

// Java 9+: effectively-final resources don't need redeclaring
static void copy(BufferedReader in, PrintWriter out) throws IOException {
   try (in; out) { out.println(in.readLine()); }
}

// Write your own AutoCloseable
class MyResource implements AutoCloseable {
   public void close() { System.out.println("closed"); }
}
```

Suppressed exceptions: if both the body and close() throw, the close() exception is attached via addSuppressed() to the body's exception, so you don't lose it.

throw vs throws, and custom exceptions

```
// throws = DECLARATION (part of the signature, checked exceptions only)
public void readFile(String path) throws IOException {
   if (path == null) throw new IllegalArgumentException("path required"); // unchecked
   Files.readAllLines(Path.of(path));
}

// Custom CHECKED exception -> extends Exception
class InsufficientFundsException extends Exception {
   public InsufficientFundsException(String msg) { super(msg); }
}
// Custom UNCHECKED exception -> extends RuntimeException
class InvalidOrderException extends RuntimeException {
   private final String orderId;                    // carry context
   public InvalidOrderException(String orderId, String msg) {
       super(msg);
       this.orderId = orderId;
   }
}

// Exception chaining: always preserve the cause
try { repo.save(order); }
catch (SQLException e) {
   throw new OrderSaveException("Could not save order " + order.getId(), e);
   //                                                                  ^ cause
}
```

Five exception-handling anti-patterns they'll ask you to spot

```
// 1. Swallowing the exception
try { doWork(); } catch (Exception e) { /* nothing */ }        // worst

// 2. Catching Throwable / Error
try { doWork(); } catch (Throwable t) { }                      // hides OOM

// 3. Logging AND rethrowing (double reporting)
catch (IOException e) { log.error("fail", e); throw e; }       // log at the boundary only

// 4. Throwing away the cause
catch (SQLException e) { throw new ServiceException("failed"); }  // lost cause!

// 5. Using exceptions for normal control flow
try { int v = map.get(key); } catch (NullPointerException e) { }  // use containsKey()

// Preferred: catch specific, log at the boundary, preserve the cause,
// and let unchecked exceptions from bugs propagate.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Throwable splits into Error, which I never catch, and Exception. Exceptions are checked — the compiler forces me to catch or declare them — or unchecked RuntimeExceptions, which usually indicate a bug I should fix rather than catch. I use try-with-resources so resources always close, I catch specific types from most- to least-specific, and when I wrap an exception I always pass the original as the cause."

Flashcards

Q. Difference between checked and unchecked exceptions?
A. Checked (IOException, SQLException) are verified at compile time — catch or declare. Unchecked (RuntimeException and subclasses) aren't; they usually signal a programming error.

Q. What is the difference between Error and Exception?
A. Error is a serious JVM-level failure (OutOfMemoryError, StackOverflowError) you should not catch. Exception represents recoverable conditions an application can handle.

Q. Does finally always execute?
A. Yes, except when System.exit() is called, the JVM crashes, the thread is killed, or a daemon thread's finally when the JVM exits.

Q. What happens if both try and finally have a return?
A. The finally block's return wins and the try's return is discarded (and any exception in try is swallowed). Never return from finally.

Q. throw vs throws?
A. throw is the statement that actually throws an exception object; throws is the method declaration listing which checked exceptions may propagate.

Q. What is try-with-resources and what's the requirement?
A. A try that auto-closes resources declared in parentheses; they must implement AutoCloseable. Closed in reverse order, even if the body throws.

Q. What is a suppressed exception?
A. When the try body and close() both throw, the close() exception is attached to the primary one via addSuppressed() instead of replacing it.

Q. Can you catch multiple exception types in one block?
A. Yes (Java 7): `catch (IOException | SQLException e)`. The parameter is implicitly final, and the types can't be related by subclassing.

Q. Is NumberFormatException checked or unchecked?
A. Unchecked — it extends IllegalArgumentException, which extends RuntimeException.

Q. Why is catching Exception or Throwable bad practice?
A. It swallows errors you can't handle (like OutOfMemoryError) and hides bugs. Catch the most specific type you can actually recover from.

Quiz

Q1. What does this print? `static int m(){ try { return 1; } finally { return 2; } }`

[ ] A. 1
[x] B. 2
[ ] C. Compile error
[ ] D. 1 then 2

> **Why:** A return in finally overrides the try's return — and swallows any exception too. It's a well-known anti-pattern.

Q2. What is the output? `static int m(){ int x=10; try { return x; } finally { x=20; } }`

[x] A. 10
[ ] B. 20
[ ] C. 30
[ ] D. Compile error

> **Why:** The return value is evaluated and saved before finally runs, so changing x afterwards has no effect on what's returned.

Q3. Which is a checked exception?

[ ] A. NullPointerException
[x] B. IOException
[ ] C. ArithmeticException
[ ] D. IllegalArgumentException

> **Why:** IOException (and its subclasses) are checked. The other three are RuntimeExceptions, i.e. unchecked.

Q4. What is wrong with `catch (Exception e) { }` after `catch (IOException e) { }`?

[ ] A. Nothing, it's fine
[x] B. Nothing — but the reverse order is a compile error
[ ] C. Both are compile errors
[ ] D. It throws at runtime

> **Why:** Catch blocks must go from most specific to most general. `catch (IOException)` after `catch (Exception)` is unreachable → compile error.

Q5. In try-with-resources, if the body throws AND close() throws, which exception surfaces?

[ ] A. The close() exception, the body one is lost
[x] B. The body exception, with the close() exception attached as suppressed
[ ] C. Both are thrown
[ ] D. Neither — they cancel out

Why: The primary (body) exception propagates and the close() exception is added via addSuppressed(), so no information is lost.

Q6. Which statement about finally is FALSE?

[ ] A. It runs even if the try block returns
[ ] B. It runs even if an exception is thrown
[x] C. It runs even after System.exit(0)
[ ] D. It's used for cleanup like closing connections

Why: System.exit() terminates the JVM immediately; finally blocks do not run (that's why shutdown hooks exist).

7. Collections Framework: List & Set Must-know 45 min

The Collections Framework is the single most-tested area in Java interviews. Learn the interface hierarchy, then the implementation trade-offs: ArrayList vs LinkedList, HashSet vs TreeSet vs LinkedHashSet.

Key points

- `Collection` → `List`, `Set`, `Queue`. `Map` is separate (it does NOT extend Collection).
- **ArrayList** = dynamic array. Fast random access O(1), slow middle insert/delete O(n). Default capacity 10, grows ×1.5.
- **LinkedList** = doubly-linked. Fast insert/delete at the ends O(1), slow random access O(n). Also implements Deque.
- **Vector** = legacy, synchronized, grows ×2. Use `ArrayList` or `CopyOnWriteArrayList` instead.
- **HashSet** = hash table, no order, O(1). **LinkedHashSet** = insertion order preserved. **TreeSet** = sorted (red-black tree), O(log n).
- HashSet uses `hashCode()` to pick the bucket and `equals()` to detect duplicates — which is why the equals/hashCode contract matters.

Reference

Implementation	Order	get/contains	add/remove	Use when
ArrayList	Insertion	O(1) by index	O(n) in the middle, O(1) at end	Default list — random access, iteration
LinkedList	Insertion	O(n)	O(1) at ends (with a reference)	Frequent insert/delete at both ends; queue/deque
Vector	Insertion	O(1)	O(n)	Legacy — synchronized, avoid in new code
HashSet	None	O(1)	O(1)	Fast membership, unique elements, no order needed
LinkedHashSet	Insertion	O(1)	O(1)	Unique + predictable iteration order
TreeSet	Sorted (natural/Comparator)	O(log n)	O(log n)	Need sorted unique elements, range queries

Code

The hierarchy, as text you can redraw in the interview

```
Iterable
└── Collection
   ├── List          (ordered, indexed, allows duplicates)
   │   ├── ArrayList
   │   ├── LinkedList   (also implements Deque)
   │   └── Vector / Stack (legacy)
   ├── Set           (no duplicates)
   │   ├── HashSet
   │   │   └── LinkedHashSet
   │   └── SortedSet
   │       └── NavigableSet
   │           └── TreeSet
   └── Queue         (FIFO / priority)
       ├── PriorityQueue
       ├── ArrayDeque
       └── Deque (double-ended)
           ├── ArrayDeque
           └── LinkedList

Map  (SEPARATE hierarchy - does NOT extend Collection)
├── HashMap
│   └── LinkedHashMap
├── Hashtable (legacy, synchronized)
├── SortedMap -> NavigableMap -> TreeMap
└── ConcurrentHashMap
```

List: create, mutate, iterate

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Spring");
list.add(1, "JPA");          // insert at index -> [Java, JPA, Spring]
list.set(0, "Core Java");    // replace
list.get(0);                 // "Core Java" - O(1)
list.remove("JPA");          // by object
list.remove(0);              // by index (careful: remove(int) vs remove(Object))
list.contains("Spring");     // true (uses equals)
list.size();
list.isEmpty();
list.indexOf("Spring");
list.clear();

// Creating lists
List<String> fixed  = Arrays.asList("a","b","c");   // fixed-size, backed by array
List<String> immut  = List.of("a","b","c");         // Java 9, truly immutable, no nulls
List<String> mutable= new ArrayList<>(List.of("a","b"));
List<Integer> nums  = new ArrayList<>(Arrays.asList(3,1,2));
Collections.sort(nums);                              // [1,2,3]

// Three ways to iterate
for (int i = 0; i < list.size(); i++) { }            // need the index
for (String s : list) { }                            // enhanced for (cleanest)
list.forEach(s -> System.out.println(s));            // Java 8
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().equals("x")) it.remove(); }  // safe removal
```

Set: deduplication and how HashSet decides 'duplicate'

```
Set<String> set = new HashSet<>();
set.add("a"); set.add("b"); set.add("a");    // the second "a" is ignored
System.out.println(set.size());              // 2
System.out.println(set.add("a"));            // false -> already present

// Order guarantees
new HashSet<>(List.of(3,1,2));         // iteration order UNSPECIFIED
new LinkedHashSet<>(List.of(3,1,2));   // 3,1,2 (insertion order)
new TreeSet<>(List.of(3,1,2));         // 1,2,3 (sorted)

// TreeSet with a custom comparator + range queries
NavigableSet<Integer> ts = new TreeSet<>(List.of(10, 5, 20, 15));
ts.first();  ts.last();                // 5, 20
ts.lower(15);  ts.higher(15);          // 10, 20
ts.headSet(15); ts.tailSet(15);        // [5,10], [15,20]
ts.descendingSet();                    // reverse view

// Dedupe a list quickly:
List<String> unique = new ArrayList<>(new HashSet<>(duplicates));

// Custom objects in a HashSet -> must implement equals + hashCode
Set<Employee> emps = new HashSet<>();
emps.add(new Employee(1, "A"));
emps.add(new Employee(1, "A"));   // deduped ONLY if equals/hashCode are correct
```

```
Output:
2
false
```

Common coding patterns with collections

```
List<Integer> nums = List.of(5, 3, 8, 1, 9, 3);

// Frequency map (the most useful pattern in coding rounds)
Map<Integer, Long> freq = nums.stream()
       .collect(Collectors.groupingBy(n -> n, Collectors.counting()));
// {1=1, 3=2, 5=1, 8=1, 9=1}

// Remove duplicates while keeping order
List<Integer> deduped = nums.stream().distinct().toList();

// Find the first duplicate
Set<Integer> seen = new HashSet<>();
Integer dup = nums.stream().filter(n -> !seen.add(n)).findFirst().orElse(null); // 3

// Min / max / reverse
Collections.max(nums); Collections.min(nums); Collections.reverse(list);
Collections.frequency(nums, 3);   // 2

// Sorting by a field
employees.sort(Comparator.comparing(Employee::getName)
                         .thenComparing(Employee::getSalary, Comparator.reverseOrder()));

// Null-safe, defensive copies
List<String> safe = list == null ? List.of() : List.copyOf(list);
```

SAY IT LIKE THIS IN THE INTERVIEW

"Collection splits into List, Set and Queue, and Map sits in its own hierarchy. ArrayList is my default — backed by a resizable array with O(1) random access; LinkedList only wins for frequent insertions at both ends. For unique elements I use HashSet for speed, LinkedHashSet when I need insertion order, and TreeSet when I need them sorted. HashSet relies on hashCode to find the bucket and equals to confirm the duplicate, which is why I override them together."

Flashcards

Q. ArrayList vs LinkedList — when do you use each?
A. ArrayList (dynamic array): O(1) random access, O(n) insert/delete in the middle — the default. LinkedList (doubly-linked): O(1) insert/delete at the ends, O(n) access — good as a deque/queue.

Q. Does Map extend Collection?
A. No. Map has its own hierarchy and is not a Collection. (You bridge them with keySet(), values(), entrySet().)

Q. HashSet vs LinkedHashSet vs TreeSet?
A. HashSet: fastest, no order. LinkedHashSet: insertion order. TreeSet: sorted order, O(log n), supports range queries.

Q. How does HashSet detect a duplicate?
A. It uses hashCode() to pick the bucket, then equals() to check whether an equal element already exists in that bucket.

Q. What is the default capacity and growth factor of ArrayList?
A. Default capacity 10; when full it grows by ~1.5x (oldCapacity + oldCapacity >> 1) and copies the array.

Q. Vector vs ArrayList?
A. Vector is legacy and synchronizes every method (slower, grows 2x). ArrayList is not synchronized — use Collections.synchronizedList or CopyOnWriteArrayList if you need thread safety.

Q. How do you make a list immutable?
A. List.of(...) (Java 9) or Collections.unmodifiableList(...) / List.copyOf(...) for a defensive snapshot.

Q. Why does remove() behave oddly on a List<Integer>?
A. Overload ambiguity: remove(int) removes by index, remove(Object) removes by value. `list.remove(1)` removes index 1; you need `list.remove(Integer.valueOf(1))` to remove the value 1.

Q. What's the difference between Collection and Collections?
A. Collection is the root interface of the framework; Collections is a utility class with static helpers (sort, max, reverse, synchronizedList, unmodifiableList).

Quiz

Q1. You need fast lookup by index and mostly read. Which do you choose?

[ ] A. LinkedList
[x] B. ArrayList
[ ] C. TreeSet
[ ] D. HashSet

> **Why:** ArrayList gives O(1) random access by index and excellent cache locality for iteration — the right default for read-heavy, index-access workloads.

Q2. Which Set preserves insertion order?

[ ] A. HashSet
[ ] B. TreeSet
[x] C. LinkedHashSet
[ ] D. EnumSet

> **Why:** LinkedHashSet maintains a linked list of entries alongside the hash table, so iteration follows insertion order (at a small memory cost).

Q3. What is the time complexity of `contains()` on an ArrayList?

[ ] A. O(1)
[ ] B. O(log n)
[x] C. O(n)
[ ] D. O(n log n)

> **Why:** contains() does a linear scan calling equals() on each element → O(n). (HashSet.contains is O(1) — that's the usual reason to switch.)

Q4. What happens when you add a duplicate element to a HashSet?

[ ] A. It throws an exception
[ ] B. The old element is replaced

[x] C. add() returns false and the set is unchanged

[ ] D. It's added anyway

Why: Set.add returns false if the element is already present (as determined by hashCode + equals); the set is unchanged.

Q5. What does `List.of("a", "b")` return and what happens on add()?

[ ] A. A mutable ArrayList

[x] B. An immutable list; add() throws UnsupportedOperationException

[ ] C. A fixed-size list; add() silently fails

[ ] D. A synchronized list

Why: List.of (Java 9) returns a truly immutable list that also rejects nulls. add/remove/set throw UnsupportedOperationException.

8. Map & HashMap Internals Must-know 50 min

"Explain how HashMap works internally" is the most common Java collection question at any level. Get the story straight: hash → bucket → equals → treeify → resize.

Key points

- `HashMap` stores **Node<K,V>** objects in an array of **buckets**. Put: hash the key → index = `hash & (n-1)` → walk that bucket → `equals` to find an existing key.
- **Collision handling**: before Java 8, only chaining (linked list). Since Java 8, a bucket with **>8** nodes AND table size **≥64** is converted to a **red-black tree** (O(log n) instead of O(n)); it drops back to a list when it shrinks below 6.
- **Load factor 0.75**, default capacity 16. When `size > capacity * 0.75`, the table **doubles** and rehashes — that's the expensive operation to avoid by pre-sizing.
- HashMap allows **one null key** and multiple null values; `Hashtable` and `ConcurrentHashMap` allow **neither**.
- HashMap is **not thread-safe**. `Hashtable` is synchronized (legacy). `ConcurrentHashMap` is the modern choice (segment/-node-level locking, no nulls).
- **Keys must be immutable** — mutating a key after insertion makes the entry unreachable.

Reference

Map impl	Order	Null key?	Thread-safe?	Notes
HashMap	None	Yes (one)	No	Default. O(1) average get/put
LinkedHashMap	Insertion / access	Yes	No	Perfect for LRU caches (access-order + removeEldestEntry)
TreeMap	Sorted by key	No (NPE)	No	Red-black tree, O(log n), range queries
Hashtable	None	No	Yes (whole-map lock)	Legacy — use ConcurrentHashMap
ConcurrentHashMap	None	No	Yes (fine-grained)	High concurrency, no nulls, weakly consistent iterators

Code

How HashMap works, step by step (the answer to recite)

```
// 1. hash(key) -> spread the bits so bad hashCodes don't cluster
static final int hash(Object key) {
   int h;
   return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
// 2. index = hash & (n - 1)   <- fast modulo because n is a power of two
// 3. Walk the bucket:
//      - if a node has the same hash && (k == key || key.equals(k))  -> UPDATE value
//      - else append a new node (collision -> chaining)
// 4. If the bucket exceeds TREEIFY_THRESHOLD (8) and the table is >= 64
//      -> convert the chain to a red-black tree (worst case O(log n))
// 5. If ++size > threshold (capacity * 0.75) -> resize: new table 2x, rehash all

Map<String, Integer> map = new HashMap<>();   // capacity 16, load factor 0.75
map.put("a", 1);
map.put("a", 2);            // same key -> value replaced, put returns the old value (1)
map.get("a");               // 2
map.getOrDefault("zz", 0);  // 0  <- avoids NPE, very handy
map.putIfAbsent("b", 10);   // only if absent
map.computeIfAbsent("c", k -> k.length());     // lazy compute, great for groupings
map.merge("a", 5, Integer::sum);               // 7  <- counting pattern
map.get("missing");         // null (careful: also null if the value is null)
```

Iterating a Map — and the only correct way to remove during iteration

```
Map<String, Integer> map = new HashMap<>(Map.of("a",1,"b",2,"c",3));

// Best: entrySet (one lookup per entry, not two)
for (Map.Entry<String, Integer> e : map.entrySet()) {
   System.out.println(e.getKey() + " -> " + e.getValue());
}
map.forEach((k, v) -> System.out.println(k + " -> " + v));   // Java 8

// WRONG - ConcurrentModificationException
// for (String k : map.keySet()) { if (k.equals("a")) map.remove(k); }

// RIGHT - use the iterator's own remove()
Iterator<Map.Entry<String,Integer>> it = map.entrySet().iterator();
while (it.hasNext()) { if (it.next().getKey().equals("a")) it.remove(); }

// RIGHT (Java 8) - removeIf on the entry set
map.entrySet().removeIf(e -> e.getValue() < 2);
// or: map.keySet().removeIf(k -> k.startsWith("a"));
```

Pre-sizing and the LRU cache trick

```
// Pre-size to avoid rehashing: expected / 0.75 + 1
int expected = 1000;
Map<String, String> m = new HashMap<>((int) (expected / 0.75f) + 1);

// An LRU cache in 6 lines using LinkedHashMap's access-order mode
class LruCache<K, V> extends LinkedHashMap<K, V> {
   private final int max;
   LruCache(int max) {
       super((int) (max / 0.75f) + 1, 0.75f, true); // true = access order
       this.max = max;
   }
   @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
       return size() > max;      // evict the least-recently-used entry
   }
}
```

ConcurrentHashMap vs synchronizedMap vs Hashtable

```
Map<String,Integer> sync = Collections.synchronizedMap(new HashMap<>());
// Every method locks the whole map -> safe but a concurrency bottleneck.
// IMPORTANT: you must manually synchronize while iterating:
synchronized (sync) { for (String k : sync.keySet()) { } }

Map<String,Integer> chm = new ConcurrentHashMap<>();
// Locks only the affected bin (or uses CAS) -> readers never block.
// Iterators are weakly consistent: no ConcurrentModificationException.
chm.put("a", 1);
chm.compute("a", (k, v) -> v == null ? 1 : v + 1);   // atomic increment
chm.merge("hits", 1, Integer::sum);                  // atomic counter
// chm.put(null, 1);  -> NullPointerException (nulls are forbidden)

// Fail-safe iteration: CHM's iterator reflects some point in time, not live changes
```

SAY IT LIKE THIS IN THE INTERVIEW

"HashMap is an array of buckets indexed by hash(key) & (n-1). On a collision the entries are chained in a linked list, and since Java 8 a bucket with more than 8 entries is treeified into a red-black tree so the worst case stays O(log n). It resizes by doubling when size exceeds 0.75 × capacity. Keys must implement equals and hashCode properly and be immutable. For concurrency I'd use ConcurrentHashMap, not Hashtable — it locks per-bin and doesn't allow nulls."

Flashcards

Q. Explain HashMap's put() internals.
A. hash(key) → bucket index = hash & (n-1) → scan the bucket: if an equal key exists (same hash && equals) replace the value, else append a node. Treeify at >8 entries, resize (2x) past 0.75 load.

Q. How does HashMap handle collisions?
A. Separate chaining (linked list per bucket) since always; since Java 8 a bucket with >8 nodes and table ≥64 becomes a red-black tree, degrading back to a list below 6.

Q. Default capacity and load factor of HashMap?
A. Capacity 16, load factor 0.75 → resizes (doubles + rehashes) once the 13th entry is added.

Q. Why must HashMap keys be immutable?
A. The bucket is computed from the key's hash at insertion. Mutating the key changes its hash, so get/remove look in the wrong bucket and the entry is orphaned (a leak).

Q. HashMap vs Hashtable?
A. HashMap: unsynchronized, allows one null key and null values, modern. Hashtable: legacy, every method synchronized (slow), no nulls allowed.

Q. HashMap vs ConcurrentHashMap?
A. CHM is thread-safe with per-bin locking/CAS, doesn't allow nulls, and its iterators are weakly consistent (never throw ConcurrentModificationException). HashMap isn't thread-safe.

Q. How do you safely remove from a map while iterating?
A. Use iterator.remove(), or entrySet().removeIf(...) — never map.remove() inside an enhanced for loop (ConcurrentModificationException).

Q. How do you implement an LRU cache in Java?
A. Extend LinkedHashMap in access-order mode (constructor flag true) and override removeEldestEntry to evict past a max size. Or use Caffeine/Guava in production.

Q. Time complexity of HashMap get/put?
A. O(1) average, O(log n) worst case since Java 8 (treeified buckets); a badly-distributed hashCode degrades everything to one bucket.

Q. What does computeIfAbsent do?
A. If the key is absent (or maps to null), computes the value from the function, inserts it and returns it — atomic on ConcurrentHashMap. Perfect for building Map<K, List<V>> groupings.

Quiz

Q1. At how many colliding entries does Java 8+ HashMap treeify a bucket?

[ ] A. > 4

[x] B. > 8 (and table size ≥ 64)
[ ] C. > 16
[ ] D. > 32

Why: TREEIFY_THRESHOLD is 8, treeifying only kicks in when the table is at least MIN_TREEIFY_CAPACITY = 64; below 6 it de-treeifies back to a list.

Q2. Which collection allows a null key?

[ ] A. Hashtable
[ ] B. ConcurrentHashMap
[x] C. HashMap
[ ] D. TreeMap

Why: HashMap permits one null key (hashed to bucket 0) and any number of null values. Hashtable and ConcurrentHashMap throw NPE; TreeMap throws NPE with a natural-ordering comparator.

Q3. What happens if you modify a HashMap key after putting it in the map?

[ ] A. The map rehashes the entry
[x] B. The entry becomes unreachable — get/remove fail, causing a leak
[ ] C. ConcurrentModificationException
[ ] D. The key is auto-updated

Why: HashMap never rehashes existing entries. Mutating a key changes its hash, so lookups go to a different bucket and the old entry is stranded.

Q4. Which is the best way to count word frequencies?

[ ] A. map.put(w, map.containsKey(w) ? map.get(w)+1 : 1)
[x] B. map.merge(w, 1, Integer::sum)
[ ] C. map.putIfAbsent(w, 1)
[ ] D. map.compute(w, (k,v) -> v)

Why: merge() handles the missing-key case and the update in one call — concise and atomic on ConcurrentHashMap.

Q5. Why do you get ConcurrentModificationException when removing in a for-each over a map?

[ ] A. The map is immutable
[x] B. The enhanced for uses an iterator, and structural modification invalidates it (fail-fast)
[ ] C. You can't iterate a map
[ ] D. HashMap synchronizes internally

Why: Collection iterators are fail-fast: they track a modCount and throw ConcurrentModificationException when the collection is structurally modified outside the iterator. Use iterator.remove() or removeIf().

Q6. What is the default load factor and why 0.75?

[ ] A. 0.5, to save memory
[x] B. 0.75, a compromise between memory usage and collision frequency
[ ] C. 1.0, to resize as late as possible
[ ] D. 0.9, for speed

Why: 0.75 is the documented trade-off: higher saves memory but increases collisions; lower reduces collisions but wastes space and resizes sooner.

9. Iterators, Comparable vs Comparator Core 30 min

Two small but reliably-asked topics: why removing in a for-each throws, and how sorting actually decides order.

Key points

- **Iterator** works on any Collection. `ListIterator` adds bidirectional traversal, `set()` and `add()`.
- **Fail-fast** iterators (ArrayList, HashMap, HashSet) throw `ConcurrentModificationException` on structural modification during iteration — detected via an internal `modCount`.
- **Fail-safe** iterators (ConcurrentHashMap, CopyOnWriteArrayList) work on a snapshot — no exception, but you may not see live changes.
- **Comparable<T>** — `compareTo(T o)`, defines the *natural* ordering, lives **inside** the class. **Comparator<T>** — `compare(T a, T b)`, an external strategy, defined **outside**.
- Contract: `compareTo` must be consistent with `equals` for sorted collections to behave (return 0 for equal).
- `compare` returns negative / 0 / positive. Never write `return a - b` for ints — it **overflows**; use `Integer.compare(a, b)`.

Reference

Aspect	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o) — one arg	compare(T a, T b) — two args
Where	Inside the class (this vs o)	Outside, as a separate strategy object
Modifies the class?	Yes	No — good for third-party classes
Used by	Collections.sort(list), TreeSet/TreeMap natural order	Collections.sort(list, cmp), multiple orderings
Number of orderings	One natural ordering	Any number of orderings
Java 8 helpers	-	Comparator.comparing(), thenComparing(), reversed(), nullsLast()

Code

Fail-fast vs fail-safe — see the difference

```
List<String> list = new ArrayList<>(List.of("a","b","c"));

// FAIL-FAST: throws ConcurrentModificationException
// for (String s : list) { if (s.equals("b")) list.remove(s); }

// Correct ways to remove during iteration:
list.removeIf(s -> s.equals("b"));                    // Java 8, cleanest
Iterator<String> it = list.iterator();
while (it.hasNext()) if (it.next().equals("a")) it.remove();
list.removeAll(List.of("a", "b"));                    // bulk removal by value

// FAIL-SAFE: iterates a snapshot, no exception
List<String> cow = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : cow) { if (s.equals("b")) cow.remove(s); }   // OK

Map<String,Integer> chm = new ConcurrentHashMap<>();
for (String k : chm.keySet()) { chm.put("new", 1); }  // OK - weakly consistent
```

Comparable: natural ordering inside the class

```
class Student implements Comparable<Student> {
   String name; int marks;
   @Override public int compareTo(Student other) {
       return Integer.compare(this.marks, other.marks);   // ascending
       // return Integer.compare(other.marks, this.marks); // descending
   }
}

List<Student> students = new ArrayList<>();
Collections.sort(students);          // uses compareTo
students.sort(null);                 // same thing
// TreeSet<Student> / TreeMap<Student,...> also use it automatically
```

Comparator: external, composable, modern

```
// Old way: an anonymous class
Comparator<Student> byName = new Comparator<>() {
   @Override public int compare(Student a, Student b) {
       return a.name.compareTo(b.name);
   }
};

// Java 8 way - readable and chainable
Comparator<Student> byName   = Comparator.comparing(Student::getName);
Comparator<Student> byMarks  = Comparator.comparingInt(Student::getMarks);
Comparator<Student> combined = byMarks.reversed()          // marks desc
                                .thenComparing(Student::getName); // then name asc

students.sort(combined);
students.sort(Comparator.comparing(Student::getName, String.CASE_INSENSITIVE_ORDER));

// Null-safe ordering (a real-world must)
students.sort(Comparator.comparing(Student::getName,
             Comparator.nullsLast(String::compareTo)));

// NEVER do this - integer overflow for extreme values:
// (a, b) -> a.getMarks() - b.getMarks()
// DO this:
// (a, b) -> Integer.compare(a.getMarks(), b.getMarks())
```

SAY IT LIKE THIS IN THE INTERVIEW

"Comparable gives a class one natural ordering via compareTo; Comparator is an external strategy via compare, which is better when I need several orderings or can't modify the class. With Java 8 I compose comparators with comparing, thenComparing and reversed. For iteration, collection iterators are fail-fast — structural modification outside the iterator throws ConcurrentModificationException, so I use iterator.remove() or removeIf; concurrent collections are fail-safe instead."

Flashcards

Q. Comparable vs Comparator?
A. Comparable is internal natural ordering (compareTo, one per class). Comparator is an external comparison strategy (compare), allowing many orderings without touching the class.

Q. Fail-fast vs fail-safe iterators?
A. Fail-fast (ArrayList, HashMap) throw ConcurrentModificationException on concurrent structural modification. Fail-safe (ConcurrentHashMap, CopyOnWriteArrayList) iterate a snapshot without throwing.

Q. How do you remove elements while iterating a list?
A. Use iterator.remove(), or the Java 8 removeIf(). Modifying the list directly inside a for-each throws ConcurrentModificationException.

Q. What should compare() return?
A. A negative int if a < b, 0 if equal, positive if a > b — per the agreed ordering.

Q. Why is `a - b` a bad comparator for ints?
A. Integer overflow: for extreme values the subtraction wraps around and returns the wrong sign. Use Integer.compare(a, b).

Q. What does ListIterator add over Iterator?
A. Bidirectional traversal (hasPrevious/previous), plus add() and set() during iteration, and nextIndex/previousIndex.

Q. Should compareTo be consistent with equals?
A. Yes, ideally: compareTo returning 0 should mean equals returns true. Otherwise TreeSet may treat distinct objects as the same element.

Quiz

Q1. Which code throws ConcurrentModificationException?

[ ] A. list.removeIf(x -> x > 2)
[x] B. for (String s : list) list.remove(s)
[ ] C. iterator.remove() inside a while loop
[ ] D. map.entrySet().removeIf(...)

> **Why:** The enhanced for compiles to an Iterator; removing directly from the list bumps modCount and the iterator fails fast. removeIf and iterator.remove() are the safe forms.

Q2. You need to sort Employees by three different fields in different screens. Comparable or Comparator?

[ ] A. Comparable
[x] B. Comparator
[ ] C. Both required
[ ] D. Neither — use SQL

> **Why:** Comparable gives exactly one natural ordering. Multiple orderings are exactly what Comparator is for — define one per screen and pass it to sort().

Q3. Which is the safe way to compare two ints in a comparator?

[ ] A. a - b
[x] B. Integer.compare(a, b)
[ ] C. a < b ? -1 : 1
[ ] D. a == b ? 0 : 1

> **Why:** Integer.compare avoids overflow. (Note option 3 is also overflow-free but loses the three-way contract clarity; Integer.compare is idiomatic.)

Q4. Which collections have fail-safe iterators?

[ ] A. ArrayList and HashMap
[x] B. ConcurrentHashMap and CopyOnWriteArrayList
[ ] C. TreeSet and TreeMap
[ ] D. LinkedList and HashSet

> **Why:** Concurrent collections iterate over a snapshot/weakly consistent view, so concurrent modification doesn't throw.

10. Generics & Wildcards Core 40 min

Generics questions are usually one of two: "explain `? extends` vs `? super`" or "what is type erasure?" Nail PECS and you're covered.

Key points

- Generics give **compile-time type safety** and remove casts: `List<String>` instead of a raw `List`.
- **Type erasure**: the compiler removes generic type info at runtime. `List<String>` and `List<Integer>` are both just `List` to the JVM. That's why you can't do `new T()`, `instanceof List<String>`, or create generic arrays.
- `? extends T` = **upper bound** — T or a subtype. You can **read** as T but not add (producer).
- `? super T` = **lower bound** — T or a supertype. You can **add** a T but reads give Object (consumer).
- **PECS**: Producer `extends`, Consumer `super`. (Joshua Bloch's mnemonic — `Collections.copy(dest, src)` is the canonical example.)
- Bounded type params: `<T extends Comparable<T>>`. Multiple bounds: `<T extends A & B>` (class first, then interfaces).

Code

Why generics exist — the before/after

```
// BEFORE generics (Java 4): unchecked, cast at every use
List names = new ArrayList();
names.add("Parvez");
names.add(42);                          // compiles! runtime bug waiting
String s = (String) names.get(0);       // manual cast, ClassCastException risk

// WITH generics: the compiler catches it
List<String> safe = new ArrayList<>();
safe.add("Parvez");
// safe.add(42);                        // COMPILE ERROR
String s2 = safe.get(0);                // no cast needed

// Generic class
class Box<T> {
   private T value;
   public void set(T v) { this.value = v; }
   public T get() { return value; }
}
Box<Integer> bi = new Box<>();
bi.set(10); int v = bi.get();           // no cast, no boxing surprise

// Generic method (the <T> before the return type is the type parameter list)
static <T> void swap(T[] arr, int i, int j) {
   T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
}
static <T extends Comparable<T>> T max(List<T> list) {
   return list.stream().max(T::compareTo).orElseThrow();
}
```

Wildcards: ? extends vs ? super (PECS)

```
List<? extends Number> producers = new ArrayList<Integer>();
Number n = producers.get(0);        // OK: reading gives at least Number
// producers.add(1);                // COMPILE ERROR: could be a List<Double>!

List<? super Integer> consumers = new ArrayList<Number>();
consumers.add(42);                  // OK: any supertype can hold an Integer
// Integer i = consumers.get(0);    // COMPILE ERROR: only Object is guaranteed
Object o = consumers.get(0);        // OK

// PECS in the wild - Collections.copy:
public static <T> void copy(List<? super T> dest,   // dest CONSUMES T
                            List<? extends T> src) { // src PRODUCES T
   for (int i = 0; i < src.size(); i++) dest.set(i, src.get(i));
}

// Mnemonic: if you GET from it, use extends. If you PUT into it, use super.
// If you do both, use an exact type (no wildcard).

// Unbounded wildcard <?> = "some unknown type" - read as Object, can't add
static void printAll(List<?> list) { for (Object o : list) System.out.println(o); }
```

Type erasure, and what it stops you doing

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());   // true -> both are ArrayList

// Consequences of erasure (all COMPILE ERRORS):
// if (a instanceof List<String>) { }      // can't test a parameterized type
// T obj = new T();                        // can't instantiate a type parameter
// T[] arr = new T[10];                    // can't create a generic array
// class Foo extends Exception<T> { }      // generic throwables not allowed
// void m(List<String> l) & void m(List<Integer> l)  // same erasure -> clash

// What DOES survive: generic info on fields, method signatures and superclasses
// is kept in the class file and readable via reflection (that's how Jackson and
// Spring resolve List<Order> at runtime).

// Workarounds:
static <T> T create(Class<T> clazz) throws Exception {
   return clazz.getDeclaredConstructor().newInstance();   // pass the Class token
}
```

Output:
```
true
```

Raw types and unchecked warnings — what to avoid

```
List raw = new ArrayList();      // raw type: legal for backwards compatibility
raw.add("text"); raw.add(42);    // no type safety at all
List<String> typed = raw;        // unchecked conversion warning

// Mixing raw and generic causes heap pollution:
List<String> strings = new ArrayList<>();
List rawList = strings;
rawList.add(42);                 // warning, but allowed
// String s = strings.get(0);    // ClassCastException at runtime!

// Rules:
//  - Never use raw types in new code.
//  - Prefer List<Object> or List<?> if you truly don't know the element type.
//  - Annotate with @SuppressWarnings("unchecked") ONLY when you can prove safety,
//    and scope it to the smallest local variable possible.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Generics give compile-time type safety and eliminate casts. Java implements them with type erasure — the type parameters are gone at runtime, so List<String> and List<Integer> are the same class, which is why I

can't instantiate a type parameter or test instanceof on a parameterized type. For wildcards I follow PECS: use ? extends when I only read from the structure, ? super when I only write into it."

Flashcards

Q. What is type erasure?
A. The compiler strips generic type information from bytecode: List<String> becomes List at runtime. It enables backwards compatibility but blocks things like `new T()`, generic arrays and `instanceof List<String>`.

Q. What does PECS stand for and mean?
A. Producer extends, Consumer super. Use `? extends T` when the structure only produces (you read) T, and `? super T` when it only consumes (you write) T.

Q. Can you add to a `List<? extends Number>`?
A. No (except null) — the compiler can't know whether the actual list is List<Integer> or List<Double>, so adding could corrupt it.

Q. Can you add to a `List<? super Integer>`?
A. Yes — any supertype of Integer can hold an Integer. But reading only gives you Object.

Q. What is a raw type and why avoid it?
A. A generic type used without type arguments (List instead of List<T>). It exists for pre-Java-5 compatibility, disables type checking and can cause heap pollution.

Q. What is a bounded type parameter?
A. A type parameter with a constraint: `<T extends Comparable<T>>` (upper bound). Multiple bounds use & with the class first: `<T extends Number & Comparable<T>>`.

Q. Can you use primitives as type arguments?
A. No — generics require reference types. List<int> is illegal; use List<Integer> and rely on autoboxing.

Q. Are generic methods possible with static?
A. Yes: `static <T> void m(T t)`. The type parameter is declared on the method, so it's independent of the class's type parameters.

Quiz

Q1. You're writing `static double sum(Collection<? extends Number> nums)`. What does the wildcard buy you?

[ ] A. You can add Integers to the collection
[x] B. You can pass List<Integer>, List<Double> etc. and read them as Number
[ ] C. It makes it faster
[ ] D. It allows null elements

Why: `? extends Number` accepts any collection of a Number subtype and lets you read elements as Number — perfect for a method that only consumes/produces by reading.

Q2. Which signature matches PECS for a method that copies from src into dest?

[ ] A. copy(List<? extends T> dest, List<? super T> src)
[x] B. copy(List<? super T> dest, List<? extends T> src)
[ ] C. copy(List<T> dest, List<T> src)
[ ] D. copy(List<?> dest, List<?> src)

Why: dest receives elements (consumer → super); src provides them (producer → extends). This is exactly how java.util.Collections.copy is declared.

Q3. What is the runtime relationship between List<String> and List<Integer>?

[ ] A. Different classes
[x] B. The same class — generics are erased at runtime
[ ] C. Subtype/supertype
[ ] D. Unrelated but castable

Why: Type erasure means both are `java.util.ArrayList` at runtime; the compiler inserts the casts for you.

Q4. Why can't you write `new T()` inside a generic class?

[ ] A. Constructors can't be generic
[x] B. Erasure means T isn't known at runtime, so there's no constructor to call
[ ] C. T is always an interface
[ ] D. It's a bug in Java

Why: After erasure T is just Object, so the JVM has no idea which constructor to invoke. The standard workaround is passing a `Class<T>` token and using reflection or a Supplier<T>.

Q5. What is the 'diamond' operator <>?

[ ] A. A cast operator
[x] B. Type inference — `new ArrayList<>()` infers the type argument from the context
[ ] C. A wildcard shorthand
[ ] D. Only valid with anonymous classes

Why: Introduced in Java 7, the diamond lets the compiler infer type arguments from the assignment target, removing the duplication of `new ArrayList<String>()`.

Day 3: Modern Java: Streams, Concurrency, JDBC & Patterns

Java 8 → 17 features, threads, streams, persistence, design patterns

10 topics · 88 flashcards · 52 quiz questions · ~7.0 hours

> **Goal.** Walk in able to write a stream pipeline, explain synchronized vs volatile, and describe how a thread pool works. These three things alone cover most 'advanced Java' rounds.

Schedule

- Slot 1 (2h): Lambdas + Streams API (do every exercise)
- Slot 2 (1.5h): Optional + Date/Time API + Java 9–17 features
- Slot 3 (2.5h): Threads, synchronization, volatile, deadlock, wait/notify
- Slot 4 (2h): ExecutorService, concurrent collections, atomics
- Slot 5 (2h): JDBC, File I/O, serialization, annotations, design patterns

1. Lambdas & Method References Must-know 35 min

A lambda is an anonymous function you can pass around — but technically it's an **instance of a functional interface**. That distinction is what interviewers probe.

Key points

- Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`.
- A lambda can only target a **functional interface** (exactly one abstract method).
- Type inference: you can omit parameter types. One param with inferred type → parentheses optional (`s -> s.length()`).
- Lambdas can **capture** local variables, but only if they're **final or effectively final** (never reassigned).
- `this` inside a lambda refers to the **enclosing class** instance — unlike an anonymous inner class, where it refers to that class.
- **Method references**: `Class::staticMethod`, `object::instanceMethod`, `Class::instanceMethod`, `Class::new`.

Code

Lambda shapes, from verbose to minimal

```
// Anonymous inner class (the old way)
Comparator<String> oldWay = new Comparator<String>() {
   @Override public int compare(String a, String b) { return a.length() - b.length(); }
};

// Lambda, full syntax
Comparator<String> l1 = (String a, String b) -> { return Integer.compare(a.length(),
   b.length()); };

// Inferred types
Comparator<String> l2 = (a, b) -> Integer.compare(a.length(), b.length());

// Single expression -> no braces, no return
Comparator<String> l3 = (a, b) -> Integer.compare(a.length(), b.length());

// Method reference - because the body just delegates to an existing method
Comparator<String> l4 = Comparator.comparingInt(String::length);

// Runnable with no params
Runnable r = () -> System.out.println("running");
new Thread(() -> System.out.println("in a thread")).start();
```

Effectively final — the rule that trips everyone up

```
int factor = 2;                    // effectively final (never reassigned)
Function<Integer,Integer> times = n -> n * factor;   // OK: captures factor

// factor = 3;                     // uncomment -> COMPILE ERROR in the lambda above
// Local variables referenced from a lambda must be final or effectively final.
// Why? The lambda may outlive the stack frame, so Java copies the VALUE.

List<Integer> nums = new ArrayList<>();
Runnable add = () -> nums.add(1);  // OK - mutating the object is fine,
                                   // only REASSIGNING the reference is banned
```

The four kinds of method reference

```
// 1. Static method:        ClassName::staticMethod
Function<String, Integer> parser = Integer::parseInt;

// 2. Instance method of a particular object:  instance::method
String prefix = ">> ";
Function<String, String> addPrefix = prefix::concat;

// 3. Instance method of an arbitrary object of a given type: ClassName::method
Function<String, Integer> len   = String::length;      // s -> s.length()
BiFunction<String,String,Boolean> eq = String::equals; // (a,b) -> a.equals(b)

// 4. Constructor reference: ClassName::new
Supplier<List<String>> listFactory = ArrayList::new;
Function<String, BigDecimal> bdFactory = BigDecimal::new;

// Where you'll use them most: streams and sorting
list.stream().map(String::trim).filter(s -> !s.isEmpty()).forEach(System.out::println);
employees.sort(Comparator.comparing(Employee::getName));
```

this in a lambda vs an anonymous class

```
class Demo {
   String name = "Demo";
   Runnable asAnonymous() {
       return new Runnable() {
           String name = "Anonymous";
           public void run() { System.out.println(this.name); }  // "Anonymous"
       };
   }
   Runnable asLambda() {
       return () -> System.out.println(this.name);  // "Demo" - enclosing instance
   }
}
```

Output:
```
Anonymous
Demo
```

SAY IT LIKE THIS IN THE INTERVIEW

"A lambda is a concise instance of a functional interface — an interface with exactly one abstract method. Parameters types are inferred, and it can capture local variables as long as they're effectively final, because those values are copied. Method references are just shorthand when the lambda body only calls an existing method."

Flashcards

Q. What is a lambda expression in Java?
A. A concise way to provide an implementation of a functional interface — essentially an anonymous function passed as data.

Q. What must be true about the target type of a lambda?
A. It must be a functional interface: exactly one abstract method (default/static methods don't count).

Q. What does 'effectively final' mean?
A. A local variable that isn't declared final but is never reassigned after initialisation — lambdas and anonymous classes may capture those (and only those).

Q. Why must captured variables be effectively final?
A. The lambda may outlive the stack frame, so Java copies the value. If it could change, the copy would silently diverge from the original.

Q. What does `this` refer to inside a lambda?
A. The enclosing class instance — lambdas don't introduce a new scope. (Anonymous inner classes have their own `this`.)
Q. Name the four types of method reference.
A. ClassName::staticMethod, object::instanceMethod, ClassName::instanceMethod, ClassName::new.

Q. Can a lambda declare a checked exception?
A. Only if the functional interface's method declares it (e.g. Callable.call throws Exception). Otherwise you must handle it inside the body.

Quiz

Q1. Which of these is a valid functional interface target for a lambda?

[ ] A. interface A { void m(); void n(); }
[ ] B. abstract class B { abstract void m(); }
[x] C. interface C { int m(String s); }
[ ] D. interface D { }

> **Why:** C has exactly one abstract method. A has two, B is a class (not an interface), and D is a marker interface with none — none can be a lambda target.

Q2. What happens if you reassign a local variable after a lambda captures it?

[ ] A. Nothing — it just works
[x] B. Compile error: local variables referenced from a lambda must be final or effectively final
[ ] C. The lambda uses the new value
[ ] D. Runtime exception

> **Why:** Reassignment makes the variable non-final, and the compiler rejects the capture. Wrap mutable state in an object (or an AtomicInteger) instead.

Q3. `String::length` is which kind of method reference?

[ ] A. Static method reference
[ ] B. Instance method of a particular object
[x] C. Instance method of an arbitrary object of the type (s -> s.length())
[ ] D. Constructor reference

> **Why:** For `ClassName::instanceMethod`, the first parameter becomes the receiver: `(String s) -> s.length()`.

Q4. Is a lambda an object?

[ ] A. No, it's pure syntax sugar with no object
[x] B. Yes — it compiles to an instance of the functional interface (invokedynamic + a synthetic class)
[ ] C. It's a method pointer like in C
[ ] D. It's a String that gets evaluated

> **Why:** The JVM uses invokedynamic with a LambdaMetafactory to produce an instance of the target interface. Lambdas without captures are cached and reused.

2. Streams API Must-know 60 min

Streams are the single most practical Java 8 skill for interviews — they turn 15 lines of loop into 2, and they're a favourite 'refactor this code' exercise. Learn the pipeline shape and the top ~15 operations.

Key points

- A stream is **not a data structure** — it's a pipeline over a source. It doesn't store elements.
- Three parts: **source** → **intermediate operations** (lazy, return a Stream) → **terminal operation** (eager, produces a result).
- **Laziness**: nothing runs until a terminal operation is called (that's why a stream without one silently does nothing).
- Streams are **consumed once** — reusing a stream throws `IllegalStateException: stream has already been operated upon or closed`.
- Streams don't modify the source collection; they produce a new result.
- `parallelStream()` splits work across the ForkJoinPool — great for CPU-bound work on large data, bad for small collections, I/O, or anything with side effects / ordering requirements.

Reference

Category	Operations	Returns
Source	stream(), stream().of(), Arrays.stream(), Files.lines(), IntStream.range()	Stream
Filter / slice	filter, distinct, limit, skip, takeWhile, dropWhile	Stream
Map / flat	map, flatMap, mapToInt/Long/Double	Stream / IntStream
Sort / peek	sorted, sorted(comparator), peek (debug only)	Stream
Match (short-circuit)	anyMatch, allMatch, noneMatch, findFirst, findAny	boolean / Optional
Reduce	reduce, count, min, max, sum	Optional / value
Collect	collect(toList/toSet/toMap/groupingBy/joining)	Collection / Map / String

Code

Anatomy of a pipeline

```
List<Employee> emps = getEmployees();

List<String> result = emps.stream()                 // 1. SOURCE
   .filter(e -> e.getSalary() > 50000)            // 2. INTERMEDIATE (lazy)
   .sorted(Comparator.comparing(Employee::getSalary).reversed())
   .map(Employee::getName)                          //    transform
   .limit(10)
   .toList();                                       // 3. TERMINAL (triggers work)

// Nothing above executes until toList() is called - that's laziness.
// The source list is NEVER modified.
```

The 15 operations you'll actually use — with outputs

```
List<Integer> nums = List.of(5, 2, 9, 1, 5, 6, 3, 8);

nums.stream().filter(n -> n % 2 == 0).toList();          // [2, 6, 8]
nums.stream().map(n -> n * n).toList();                  // [25, 4, 81, 1, ...]
nums.stream().distinct().toList();                       // [5, 2, 9, 1, 6, 3, 8]
nums.stream().sorted().toList();                         // [1, 2, 3, 5, 5, 6, 8, 9]
nums.stream().sorted(Comparator.reverseOrder()).toList();// [9, 8, 6, 5, 5, 3, 2, 1]
nums.stream().limit(3).toList();                         // [5, 2, 9]
nums.stream().skip(2).toList();                          // [9, 1, 5, 6, 3, 8]

nums.stream().anyMatch(n -> n > 8);                      // true
nums.stream().allMatch(n -> n > 0);                      // true
nums.stream().noneMatch(n -> n < 0);                     // true
nums.stream().findFirst();                               // Optional[5]
nums.stream().count();                                   // 8
nums.stream().reduce(0, Integer::sum);                   // 39
nums.stream().mapToInt(Integer::intValue).sum();         // 39
nums.stream().max(Integer::compareTo);                   // Optional[9]

// flatMap: flatten nested collections (List<List<String>> -> Stream<String>)
List<List<String>> nested = List.of(List.of("a","b"), List.of("c"));
nested.stream().flatMap(List::stream).toList();          // [a, b, c]

// takeWhile / dropWhile (Java 9) on an ordered stream
List.of(1,2,3,4,1).stream().takeWhile(n -> n < 4).toList(); // [1,2,3]
```

Collectors — the terminal operations that build results

```
List<Employee> emps = getEmployees();

// toList / toSet / toCollection
emps.stream().map(Employee::getName).collect(Collectors.toList());
emps.stream().map(Employee::getName).collect(Collectors.toSet());   // dedupe

// joining - build a CSV
emps.stream().map(Employee::getName).collect(Collectors.joining(", "));
// "Parvez, Anil, Sneha"
Collectors.joining(", ", "[", "]");   // "[Parvez, Anil, Sneha]"

// groupingBy - the most powerful one
Map<String, List<Employee>> byDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment));

Map<String, Long> countByDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment,
                                               Collectors.counting()));

Map<String, Double> avgSalaryByDept =
   emps.stream().collect(Collectors.groupingBy(Employee::getDepartment,
                            Collectors.averagingDouble(Employee::getSalary)));

// partitioningBy - always 2 groups (true/false)
Map<Boolean, List<Employee>> rich =
   emps.stream().collect(Collectors.partitioningBy(e -> e.getSalary() > 100000));

// toMap - watch out for duplicate keys!
Map<Integer, String> idToName = emps.stream()
   .collect(Collectors.toMap(Employee::getId, Employee::getName,
                             (a, b) -> a));    // <-- merge function avoids the crash

// summarizingInt - count, sum, min, average, max in one pass
IntSummaryStatistics stats =
   emps.stream().collect(Collectors.summarizingInt(Employee::getSalary));
stats.getAverage(); stats.getMax(); stats.getCount();
```

Laziness and short-circuiting — prove it with peek

```
Stream.of(1, 2, 3, 4, 5)
     .peek(n -> System.out.println("processing " + n))
     .filter(n -> n > 2)
     .findFirst();
// Output: processing 1 / processing 2 / processing 3
// It STOPS at 3 because findFirst short-circuits - elements 4 and 5 are never
// touched. With an infinite stream this is the difference between terminating
// and hanging:

Stream.iterate(0, n -> n + 1)          // infinite source - fine, because
     .filter(n -> n > 100)            // ...the pipeline is lazy and
     .findFirst()                     // ...findFirst short-circuits
     .orElse(-1);                     // 101

// Without a terminal operation, peek prints NOTHING.
```

Output:

```
processing 1
processing 2
processing 3
```

Refactoring exercise: before and after (a real interview task)

```
// BEFORE: find the names of the 3 highest-paid engineers, sorted alphabetically
List<String> result = new ArrayList<>();
for (Employee e : employees) {
   if ("Engineering".equals(e.getDepartment()) && e.getSalary() > 60000) {
       result.add(e.getName().toUpperCase());
   }
}
Collections.sort(result);
if (result.size() > 3) result = result.subList(result.size() - 3, result.size());

// AFTER
List<String> result2 = employees.stream()
   .filter(e -> "Engineering".equals(e.getDepartment()))
   .filter(e -> e.getSalary() > 60000)
   .sorted(Comparator.comparing(Employee::getSalary).reversed())
   .limit(3)
   .map(e -> e.getName().toUpperCase())
   .sorted()
   .toList();
```

Primitive streams and parallel streams

```
// Primitive streams avoid boxing overhead and add maths helpers
IntStream.range(1, 5).toArray();                 // [1, 2, 3, 4]  (end EXCLUSIVE)
IntStream.rangeClosed(1, 5).toArray();           // [1, 2, 3, 4, 5]
IntStream.rangeClosed(1, 100).sum();             // 5050
IntStream.rangeClosed(1, 100).average();         // OptionalDouble[50.5]
List.of(1,2,3).stream().mapToInt(Integer::intValue).sum();   // 6

// Parallel: splits across ForkJoinPool.commonPool()
long count = IntStream.rangeClosed(1, 1_000_000).parallel().filter(isPrime).count();

// Rules for parallelStream:
//  - Great for large, CPU-bound, stateless, order-independent work
//  - Bad for small collections (overhead > benefit), I/O, or shared mutable state
//  - Avoid if you depend on encounter order (unless you use forEachOrdered)
//  - Never use it with stateful lambdas or side effects
```

SAY IT LIKE THIS IN THE INTERVIEW

"A stream is a lazy pipeline over a data source: a source, zero or more intermediate operations like filter and map, and one terminal operation that triggers execution and produces a result. It doesn't store data or modify the source, and it can only be consumed once. I reach for Collectors like groupingBy and joining to build results, and I only use parallelStream for large CPU-bound work because the overhead and ordering

caveats usually aren't worth it."

Flashcards

Q. Is a stream a data structure?
A. No — it's a view/pipeline over a source (collection, array, generator). It doesn't store elements and never modifies the source.

Q. What's the difference between intermediate and terminal operations?
A. Intermediate (filter, map, sorted) are lazy and return a new Stream. Terminal (collect, forEach, count, reduce) are eager and trigger the whole pipeline.

Q. What happens if you use a stream twice?
A. IllegalStateException: stream has already been operated upon or closed. Create a new stream from the source instead.

Q. map vs flatMap?
A. map is 1→1 (each element becomes one element). flatMap is 1→many and flattens the resulting streams into one (List<List<T>> → Stream<T>).

Q. How do you avoid the duplicate-key crash in Collectors.toMap?
A. Pass a merge function as the third argument: `toMap(keyFn, valueFn, (a, b) -> a)`.

Q. groupingBy vs partitioningBy?
A. groupingBy groups by any key function (n groups). partitioningBy splits by a predicate into exactly two groups keyed true/false.

Q. When would you use parallelStream()?
A. For large, CPU-bound, stateless work where order doesn't matter. Avoid for small data, I/O, side effects or shared state — the ForkJoinPool overhead usually outweighs the gain.

Q. Why use IntStream instead of Stream<Integer>?
A. Avoids boxing/unboxing overhead and provides sum/average/min/max/summaryStatistics directly.

Q. Give the alphabetic order of a typical pipeline.
A. source → filter → map → sorted → distinct → limit → collect.

Q. What does peek() do and when is it appropriate?
A. Performs an action on each element as they flow through — intended for debugging/logging (e.g. proving laziness). Using it for real side effects is discouraged.

Quiz

Q1. What is the result of `Stream.of(1,2,3).filter(n -> n > 1);` with no terminal operation?

[ ] A. [2, 3]
[x] B. Nothing happens — intermediate operations are lazy
[ ] C. Compile error
[ ] D. An empty stream is stored

> **Why:** Without a terminal operation the pipeline is never executed. This also means any peek() logging prints nothing.

Q2. Which of these is a TERMINAL operation?

[ ] A. filter
[ ] B. map
[x] C. collect
[ ] D. sorted

> **Why:** collect, forEach, reduce, count, min, max, anyMatch, findFirst and toList() are terminal. filter/map/sorted/distinct/limit are intermediate.

Q3. `List<List<String>> nested; nested.stream().flatMap(List::stream).count()` — what does flatMap do here?

[ ] A. Counts the inner lists
[x] B. Flattens all inner lists into a single stream of Strings, then counts them

[ ] C. Throws a ClassCastException
[ ] D. Counts distinct lists

> **Why:** flatMap maps each element to a stream and concatenates them, turning Stream<List<String>> into Stream<String>.

Q4. Which Collector gives you a Map<String, Long> of counts per key?

[ ] A. Collectors.toMap
[x] B. Collectors.groupingBy(keyFn, Collectors.counting())
[ ] C. Collectors.counting()
[ ] D. Collectors.partitioningBy

> **Why:** groupingBy produces the map; the downstream collector counting() computes each group's size.

Q5. What is a risk of parallelStream()?

[ ] A. It always produces wrong ordering and is never usable
[x] B. Shared mutable state, ordering and ForkJoinPool overhead can make it slower or incorrect
[ ] C. It requires a special JVM flag
[ ] D. It can't be used with filter

> **Why:** Parallel streams share the common ForkJoinPool, and side effects/ordering bugs are easy to introduce. Use them only for big, independent, CPU-bound workloads.

Q6. `emps.stream().collect(Collectors.toMap(Employee::getDept, Employee::getName))` throws on duplicate departments. Why?

[ ] A. Because the value mapper is wrong
[x] B. toMap throws IllegalStateException on duplicate keys — you need a merge function
[ ] C. Because streams can't produce maps
[ ] D. Because getName returns null

> **Why:** Supply `(a, b) -> a` (or your own merge logic) as the third argument to handle collisions — or use groupingBy if you want all values per key.

3. Optional & the Date/Time API Must-know 35 min

Two Java 8 additions that come up constantly because they replaced bad old habits: `Optional` for nulls, and `java.time` for `Date`/`Calendar` (which were genuinely awful).

Key points

- `Optional<T>` is a **container** that may or may not hold a value — it's designed as a **return type** to force callers to handle absence.
- **Never** call `get()` without checking — use `orElse`, `orElseGet`, `orElseThrow`, `ifPresent`, `map`, `filter`.
- `orElse(x)` **always evaluates** x; `orElseGet(supplier)` only when empty. Use orElseGet for expensive defaults.
- Anti-patterns: using Optional as a field or parameter, `Optional.of(null)` (throws), `isPresent()+get()` (same as a null check).
- `java.time` is **immutable and thread-safe** — unlike `java.util.Date` and `Calendar`, which are mutable (a real bug source).
- Key types: `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant` (machine time), `Duration`, `Period`, `DateTimeFormatter`.

Reference

Need	Use	Example
Date only (birthday)	LocalDate	LocalDate.of(1999, 5, 21)
Date + time, no zone	LocalDateTime	LocalDateTime.now()
Date + time + zone	ZonedDateTime	ZonedDateTime.now(ZoneId.of("Asia/Kolkata"))
Timestamp / machine time	Instant	Instant.now() — great for DB/audit fields
Elapsed machine time	Duration	Duration.between(start, end)
Elapsed calendar time	Period	Period.between(d1, d2) → years/months/days
Parsing / formatting	DateTimeFormatter	DateTimeFormatter.ofPattern("dd-MM-yyyy")

Code

Optional done right (and wrong)

```
Optional<String> name = findNameById(42);

// WRONG
// if (name.isPresent()) System.out.println(name.get());   // just a null check
// Optional.of(maybeNull);        // throws NPE if null!
// Optional<String> field;        // don't use Optional as a field

// RIGHT
name.ifPresent(System.out::println);
name.ifPresentOrElse(System.out::println, () -> System.out.println("not found")); // Java 9

String value = name.orElse("default");
String lazy  = name.orElseGet(() -> expensiveLookup());   // only called if empty
String boom  = name.orElseThrow(() -> new NotFoundException("no name"));

// Transform without unwrapping
int len = name.map(String::length).orElse(0);
String upper = name.map(String::toUpperCase).filter(s -> s.length() > 3).orElse("");

// Chain nested nullables (replaces nested if-null checks)
String city = user.flatMap(User::getAddress).map(Address::getCity).orElse("Unknown");

// Java 9+: turn an Optional into a Stream
List<String> present = optionals.stream().flatMap(Optional::stream).toList();
```

Optional in real service code

```
// Repository returns Optional -> no null leaks
public Optional<User> findByEmail(String email) {
   return users.stream().filter(u -> u.getEmail().equals(email)).findFirst();
}

// Service: handle absence explicitly
public User getOrCreate(String email) {
   return findByEmail(email).orElseGet(() -> createUser(email));
}

// Controller: 404 instead of NPE
@GetMapping("/{id}")
public ResponseEntity<User> get(@PathVariable Long id) {
   return service.findById(id)
           .map(ResponseEntity::ok)
           .orElseGet(() -> ResponseEntity.notFound().build());
}
```

java.time in practice

```
// Creating
LocalDate today       = LocalDate.now();
LocalDate independence= LocalDate.of(1947, Month.AUGUST, 15);
LocalDateTime meeting = LocalDateTime.of(2026, 10, 10, 14, 30);
Instant timestamp     = Instant.now();                       // UTC, machine time
ZonedDateTime ist     = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));

// Arithmetic (immutable - always returns a NEW object)
LocalDate nextWeek    = today.plusWeeks(1);
LocalDate lastMonth   = today.minusMonths(1);
LocalDate firstOfMonth= today.withDayOfMonth(1);

// Comparing
today.isBefore(independence);   // false
today.isAfter(independence);    // true
ChronoUnit.DAYS.between(independence, today);   // long

// Period vs Duration
Period p  = Period.between(LocalDate.of(2020,1,1), LocalDate.of(2023,3,15)); // P3Y2M14D
Duration d = Duration.between(startTime, endTime);   // seconds/nanos
d.toMinutes();

// Formatting and parsing (DateTimeFormatter is IMMUTABLE and thread-safe)
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm");
String text = meeting.format(fmt);                    // "10-10-2026 14:30"
LocalDateTime parsed = LocalDateTime.parse("10-10-2026 14:30", fmt);

// Legacy interop (you'll need this with old JDBC code)
Date legacy     = Date.from(Instant.now());
Instant fromOld = legacy.toInstant();
```

Why java.util.Date was replaced

```
// OLD API problems:
Date d = new Date(2026, 9, 10);      // deprecated: year is 1900-based, month is 0-based!
d.setMonth(15);                      // MUTABLE - a shared Date gets corrupted
Calendar cal = Calendar.getInstance();
cal.set(Calendar.MONTH, Calendar.OCTOBER);   // 0-based months, verbose, mutable
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");  // NOT thread-safe!

// NEW API: clear, immutable, thread-safe, fluent
LocalDate clean = LocalDate.of(2026, 10, 10);      // real year, real month
DateTimeFormatter safe = DateTimeFormatter.ofPattern("dd-MM-yyyy");  // shareable
```

SAY IT LIKE THIS IN THE INTERVIEW

"Optional is a container that may hold a value, designed as a return type so absence is handled explicitly — I use orElse, orElseGet, orElseThrow and map/flatMap chains instead of get(), and I never use it as a field or parameter. For dates I use java.time: it's immutable and thread-safe, unlike Date and Calendar, and DateTimeFormatter can be shared across threads where SimpleDateFormat can't."

Flashcards

Q. What problem does Optional solve?
A. It makes absence explicit in the type system, forcing callers to handle the empty case instead of hitting NullPointerException.

Q. orElse vs orElseGet?
A. orElse takes a value that is ALWAYS evaluated; orElseGet takes a Supplier evaluated only when the Optional is empty. Use orElseGet for expensive defaults.

Q. Optional.of vs Optional.ofNullable?
A. of(null) throws NullPointerException; ofNullable(null) returns Optional.empty(). Use ofNullable when the value may be null.

Q. Should Optional be used as a field or method parameter?
A. No — it's designed as a return type. As a field it's not Serializable and adds overhead; as a parameter it forces callers to wrap values.

Q. Why is java.time better than java.util.Date?
A. java.time is immutable and thread-safe, has a clear API (LocalDate/LocalDateTime/Instant/ZonedDateTime), real month numbers, and a thread-safe DateTimeFormatter.

Q. LocalDate vs Instant?
A. LocalDate is a calendar date with no time or zone (a birthday). Instant is a machine timestamp on the UTC timeline (audit fields, DB timestamps).

Q. Is SimpleDateFormat thread-safe?
A. No — sharing a static instance across threads corrupts results. DateTimeFormatter in java.time IS immutable and thread-safe.

Q. Period vs Duration?
A. Period measures calendar time (years, months, days). Duration measures exact machine time (seconds, nanos).

Quiz

Q1. What's the difference between orElse(compute()) and orElseGet(() -> compute())?

[ ] A. No difference
[x] B. orElse always evaluates compute(); orElseGet only evaluates it when the Optional is empty
[ ] C. orElseGet is deprecated
[ ] D. orElse throws if empty

> **Why:** orElse's argument is evaluated eagerly at call time. That's a real performance bug when the default requires a database call — use orElseGet.

Q2. `Optional.of(null)` results in:

[ ] A. Optional.empty()
[x] B. NullPointerException
[ ] C. Compile error
[ ] D. A warning only

> **Why:** of() requires a non-null value and throws NPE immediately. Use ofNullable() when the value might be null.

Q3. Which is immutable and thread-safe?

[ ] A. java.util.Date
[ ] B. java.util.Calendar
[x] C. LocalDateTime
[ ] D. SimpleDateFormat

> **Why:** All java.time types are immutable and thread-safe. Date and Calendar are mutable; SimpleDateFormat is not thread-safe.

Q4. How do you get the number of days between two LocalDate objects?

[ ] A. d2 - d1
[x] B. ChronoUnit.DAYS.between(d1, d2)
[ ] C. d1.minus(d2).getDays()
[ ] D. Duration.between(d1, d2).toDays()

> **Why:** ChronoUnit.DAYS.between(a, b) is the idiomatic call. (Duration doesn't work on LocalDate because there's no time component.)

Q5. What is the recommended use of Optional in an API?

[ ] A. As a field type to avoid nulls in entities
[x] B. As a method return type to signal 'may be absent'
[ ] C. As a parameter type
[ ] D. In constructors

> **Why:** Optional is a return-type convention. Fields make the class non-Serializable-friendly and add indirection; parameters force awkward wrapping.

4. Modern Java: 9 → 17 Features Edge 35 min

Most fresher candidates only know Java 8. Dropping `records`, `sealed`, `var`, and switch expressions into an answer is a cheap way to look current — and Java 17 is the current LTS, so your interview is probably on 11 or 17.

Key points

- **Java 9**: module system (JPMS), `List.of/Map.of/Set.of`, `Optional.ifPresentOrElse`, `Stream.takeWhile/dropWhile/ofNullable/iterate`, private interface methods, JShell.
- **Java 10**: `var` local variable type inference (locals only, must be initialised — not for fields, parameters or return types).
- **Java 11 (LTS)**: `var` in lambda params, HTTP Client (java.net.http), String `isBlank/strip/lines/repeat/indent`, `Files.readString/writeString`, single-file source launch.
- **Java 14/15**: **switch expressions** (`->`, `yield`), helpful NPE messages, **text blocks**, **records** (preview).
- **Java 16**: **records** (final), **pattern matching for instanceof**, Stream.toList().
- **Java 17 (LTS)**: **sealed classes** (`permits`), pattern matching for switch (preview), strong encapsulation of JDK internals.

Reference

Feature	Version	What it gives you
var	10	Less boilerplate for obvious local types
Records	16	Immutable data carriers — equals/hashCode/toString/accessors for free
Sealed classes	17	Restrict which classes may extend/implement — enables exhaustive pattern matching
Switch expressions	14	switch returns a value, no fall-through, exhaustive
Text blocks	15	Readable multi-line strings (JSON, SQL, HTML)
Pattern matching instanceof	16	Test and cast in one step
HTTP Client	11	Modern sync/async HTTP with HTTP/2 and WebSocket

Code

Records — the DTO killer (Java 16+)

```
// ONE line replaces a class with fields, getters, equals, hashCode, toString
public record UserDto(Long id, String name, String email) { }

UserDto u = new UserDto(1L, "Parvez", "p@example.com");
u.name();            // "Parvez"  (accessor, NOT getName())
u.equals(new UserDto(1L, "Parvez", "p@example.com"));   // true - value equality!
System.out.println(u);   // UserDto[id=1, name=Parvez, email=p@example.com]

// Records are implicitly final, and their fields are private final.
// You can still add methods, compact constructors and validation:
public record Range(int start, int end) {
   public Range {                       // compact constructor - validation
       if (start > end) throw new IllegalArgumentException("start > end");
   }
   public int length() { return end - start; }
}

// Where you use them: DTOs, API responses, Map keys, multi-value returns,
// and anywhere you'd otherwise write a Lombok @Data class.
```

Output:
```
UserDto[id=1, name=Parvez, email=p@example.com]
```

Sealed classes — controlled inheritance (Java 17+)

```
// Only the listed classes may extend Shape. Nothing else can.
public sealed interface Shape permits Circle, Square, Rectangle { }

record Circle(double radius) implements Shape { }
record Square(double side)   implements Shape { }
final class Rectangle implements Shape { }   // permitted subclasses must be
                                             // final, sealed, or non-sealed

// Because the hierarchy is closed, the compiler can prove exhaustiveness:
double area(Shape s) {
   return switch (s) {                    // NO default needed!
       case Circle c    -> Math.PI * c.radius() * c.radius();
       case Square sq   -> sq.side() * sq.side();
       case Rectangle r -> r.width() * r.height();
   };
}
```

Pattern matching, var, and text blocks

```
// instanceof + cast, OLD:
if (obj instanceof String) {
   String s = (String) obj;
   System.out.println(s.length());
}
// NEW (Java 16) - test and bind in one step:
if (obj instanceof String s && s.length() > 3) {
   System.out.println(s.toUpperCase());     // s is in scope here
}

// var - local type inference (Java 10)
var list = new ArrayList<String>();     // ArrayList<String>
var entry = map.entrySet().iterator().next();
// var x;          -> ERROR: needs an initialiser
// var x = null;   -> ERROR: can't infer
// Use it when the type is OBVIOUS from the right-hand side; otherwise spell
// it out - readability beats brevity.

// Text blocks (Java 15)
String query = """
   SELECT u.name, u.email
   FROM users u
   WHERE u.active = true
   ORDER BY u.name
   """;
```

Small but high-value additions

```
// Java 9: convenient immutable collections
List<String> l = List.of("a", "b");         // immutable, no nulls
Map<String,Integer> m = Map.of("a", 1, "b", 2);
Map<String,Integer> m5 = Map.ofEntries(Map.entry("a", 1), Map.entry("b", 2)); // >10 pairs
Set<Integer> s = Set.of(1, 2, 3);

// Java 9: Stream improvements
Stream.ofNullable(maybeNull);                       // empty stream if null
Stream.iterate(1, n -> n < 100, n -> n * 2).toList();  // has a predicate
List.of(1,2,3,4,5).stream().takeWhile(n -> n < 4).toList();  // [1,2,3]

// Java 11: String helpers
"  ".isBlank();          // true
"  hi  ".strip();        // "hi"    (Unicode-aware)
"hi\nthere".lines().toList();  // [hi, there]
"ab".repeat(3);          // "ababab"

// Java 11: run a single file without compiling
// java Hello.java

// Java 11: HTTP Client
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder()
       .uri(URI.create("https://api.example.com/users"))
       .header("Accept", "application/json")
       .GET().build();
HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode() + " " + res.body());
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'm comfortable on Java 11 and 17, the current LTS versions. Beyond Java 8 I use records for immutable DTOs instead of Lombok boilerplate, sealed interfaces with pattern-matching switch for closed hierarchies, var where the type is obvious, text blocks for SQL and JSON, and the Java 11 HTTP Client instead of HttpURLConnection."

Flashcards

Q. What is a record and when do you use it?
A. A transparent, immutable data carrier (Java 16): `record Point(int x, int y)`. The compiler generates a canonical constructor, accessors, equals, hashCode and toString. Use it for DTOs, API models and Map keys.

Q. What is a sealed class/interface?
A. Java 17: `sealed interface Shape permits Circle, Square` restricts which types may implement it. Permitted subclasses must be final, sealed or non-sealed — enabling exhaustiveness checks.

Q. Where can you use var?
A. Only for local variables with an initialiser (and lambda parameters since Java 11). Not fields, method parameters, return types, or with null.

Q. What does `instanceof String s` do?
A. Pattern matching (Java 16): tests the type and, if it matches, declares and assigns the variable in one step, scoping it to where the match is guaranteed.

Q. What is a text block?
A. A multi-line string literal delimited by triple quotes (Java 15) that preserves formatting and strips incidental indentation — ideal for SQL, JSON and HTML.

Q. Which Java versions are LTS?
A. 8, 11, 17 and 21. Most enterprises run 11 or 17.

Q. What replaced HttpURLConnection?
A. The java.net.http.HttpClient (Java 11) — supports HTTP/1.1, HTTP/2, synchronous and asynchronous calls, and WebSockets.

Q. What is JShell?
A. The Java REPL (Java 9) for quickly experimenting with snippets: type `jshell` at the terminal.

Quiz

Q1. Which is true about records?

[ ] A. They're mutable JavaBeans
[x] B. They're immutable; accessors are named after the field (u.name(), not getName())
[ ] C. They can be extended by subclasses
[ ] D. They only exist at compile time

> **Why:** Records are implicitly final with private final fields. Accessors use the field name directly, and equals/hashCode/toString are generated with value semantics.

Q2. Why does a sealed interface let you omit `default` in a switch expression?

[ ] A. Because switch never needs a default
[x] B. Because the compiler knows the complete set of permitted subtypes and can prove exhaustiveness
[ ] C. Because sealed types can't be switched on
[ ] D. Because records never fail

> **Why:** That's the whole point of sealing: a closed hierarchy makes exhaustive pattern matching checkable at compile time.

Q3. Where can `var` NOT be used?

[ ] A. Local variables with an initialiser
[ ] B. Enhanced-for loop variables
[x] C. Method parameters and fields
[ ] D. try-with-resources

> **Why:** var is restricted to local variable declarations (and lambda params in Java 11+). Fields and method signatures must be explicit.

Q4. Which Java version introduced the module system (JPMS)?

[ ] A. Java 8
[x] B. Java 9
[ ] C. Java 11
[ ] D. Java 17

> **Why:** Java 9 introduced Project Jigsaw modules (module-info.java). It's mostly relevant for library and JDK internals work, not daily app code.

Q5. What is the output of `"Java".repeat(2) + "\u0021"`?

[x] A. JavaJava!
[ ] B. Java!Java!
[ ] C. Java2!
[ ] D. Compile error

> **Why:** repeat(n) (Java 11) concatenates the string n times, giving "JavaJava", then "!" — though note \u0021 is processed by the compiler as a unicode escape, which still yields !.

5. Threads: Creation, Lifecycle & Basics Must-know 45 min

Multithreading is where fresher interviews get serious. Start with the lifecycle diagram and the two ways to create a thread — everything else builds on that.

Key points

- **Process** = a running program with its own memory. **Thread** = a unit of execution inside a process; threads share heap memory but have their own stack.
- Two ways to create: `extends Thread` or `implements Runnable` (preferred — you can still extend another class, and it separates the task from the runner).
- **Always call `start()`, never `run()`** — `run()` just executes on the current thread, no new thread is created.
- **Lifecycle**: NEW → (start) → RUNNABLE ⇄ RUNNING → (wait/sleep/join) → WAITING/TIMED_WAITING/BLOCKED → TERMINATED.
- `sleep()` holds the lock and pauses; `wait()` **releases** the lock and must be called inside `synchronized`; `yield()` is only a hint; `join()` waits for the thread to finish.
- A **daemon thread** is a background service thread — the JVM exits when only daemon threads remain.

Reference

Method	Releases lock?	Purpose	Class
sleep(ms)	No	Pause the current thread for a duration	Thread (static)
wait()	Yes	Wait until notify/notifyAll — must be in synchronized	Object (final)
notify()/notifyAll()	On exit of the block	Wake one/all waiting threads	Object (final)
join()	No	Wait for another thread to die	Thread
yield()	No	Hint: let another thread run (no guarantee)	Thread (static)
interrupt()	-	Signal a thread to stop blocking/stop work	Thread

Code

Two ways to create a thread — and the start() vs run() trap

```
// Way 1: extend Thread (simple, but you've used up your one inheritance)
class MyThread extends Thread {
   @Override public void run() { System.out.println("running in " + getName()); }
}
new MyThread().start();

// Way 2: implement Runnable (PREFERRED) - task decoupled from the thread
Runnable task = () -> System.out.println("running in " + Thread.currentThread().getName());
new Thread(task).start();
new Thread(task, "worker-1").start();     // named threads are easier to debug

// Way 3: Callable + Future when you need a RESULT or checked exceptions
ExecutorService pool = Executors.newSingleThreadExecutor();
Future<Integer> f = pool.submit(() -> 21 + 21);
Integer result = f.get();     // blocks until done -> 42

// THE TRAP
Thread t = new Thread(() -> System.out.println("hi"));
t.run();     // WRONG: runs on the MAIN thread, no concurrency
t.start();   // RIGHT: the JVM creates a new thread and calls run() there

// Calling start() twice -> IllegalThreadStateException
```

Thread lifecycle and states

```
//   new Thread()        --> NEW
//        .start()       --> RUNNABLE (waiting for the scheduler)
//   scheduled           --> RUNNING
//        .sleep(n) / .wait(timeout) / .join(timeout)  --> TIMED_WAITING
//        .wait()  / .join()                           --> WAITING
//   trying to enter a synchronized block held by another thread --> BLOCKED
//   run() returns       --> TERMINATED

Thread t = new Thread(() -> {
   try { Thread.sleep(1000); } catch (InterruptedException e) {
       Thread.currentThread().interrupt();   // restore the interrupt flag
   }
});
t.start();
System.out.println(t.getState());   // RUNNABLE or TIMED_WAITING
t.join();                           // main waits here until t finishes
System.out.println(t.getState());   // TERMINATED

// Daemon threads: JVM exits when only daemons remain
Thread daemon = new Thread(() -> { while (true) { /* background work */ } });
daemon.setDaemon(true);             // MUST be set before start()
daemon.start();
```

sleep vs wait — the #1 multithreading question

```
// sleep: static, on Thread, keeps the lock, wakes after the timeout
synchronized (lock) {
   Thread.sleep(1000);   // still HOLDS the lock - others stay blocked
}

// wait: on Object, releases the lock, needs notify to wake
synchronized (lock) {
   lock.wait();          // RELEASES the lock, other threads can enter
   // ... after notify() + re-acquiring the lock, continues here
}
// Another thread:
synchronized (lock) { lock.notify(); }   // or notifyAll()

// Rules:
//  - wait/notify/notifyAll must be called from INSIDE a synchronized block
//    on the same object -> IllegalMonitorStateException otherwise
//  - Always wait() in a LOOP (spurious wakeups are allowed):
synchronized (lock) {
   while (!condition) { lock.wait(); }
}
//  - Prefer notifyAll() over notify() unless you can prove one waiter suffices.
```

Race condition — see the bug before you fix it

```
class Counter {
   private int count = 0;
   public void increment() { count++; }   // NOT atomic! read -> add -> write
   public int get() { return count; }
}

Counter c = new Counter();
Runnable r = () -> { for (int i = 0; i < 10_000; i++) c.increment(); };
Thread t1 = new Thread(r), t2 = new Thread(r);
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println(c.get());   // Expected 20000, actual ~14000 (varies!)

// Fixes, in order of preference:
// 1. AtomicInteger   -> private final AtomicInteger c = new AtomicInteger();
// 2. synchronized    -> public synchronized void increment() { count++; }
// 3. Lock            -> lock.lock(); try { count++; } finally { lock.unlock(); }
// 4. LongAdder       -> best under very high contention
```

Output:
```
~14000 (non-deterministic)
```

SAY IT LIKE THIS IN THE INTERVIEW

"A thread is a lightweight unit of execution inside a process; threads share heap memory but each has its own stack. I implement Runnable and call start() — never run() directly, which would just execute on the calling thread. For lifecycle: NEW, RUNNABLE, RUNNING, BLOCKED/WAITING/TIMED_WAITING, TERMINATED. And the key difference: sleep() keeps the lock, wait() releases it and must be called inside a synchronized block."

Flashcards

Q. Thread vs Runnable — which should you use and why?
A. Implement Runnable. Java has single inheritance, so extending Thread wastes your one superclass, and Runnable separates the task from the thread running it (so it also works with executors).

Q. What happens if you call run() instead of start()?
A. No new thread is created — run() executes synchronously on the current thread. Only start() creates a thread; calling start() twice throws IllegalThreadStateException.

Q. sleep() vs wait()?
A. sleep is a Thread static method that pauses without releasing the lock. wait is an Object method that releases the monitor and must be called inside synchronized; it needs notify/notifyAll to resume.

Q. Why must wait() be inside a synchronized block?
A. Because wait() releases the monitor the thread owns — if it doesn't own it, you get IllegalMonitorStateException. Reacquisition is also required on wake.

Q. What is a daemon thread?
A. A background/service thread. The JVM terminates when only daemon threads remain — used for GC and background housekeeping.

Q. What is a race condition?
A. When the result depends on the unpredictable interleaving of threads accessing shared mutable state — e.g. count++ (read-modify-write) losing updates.

Q. Why should wait() be called in a loop?
A. Because of spurious wakeups (the JVM may wake a thread without notify) and because the condition may still be false when the lock is reacquired.

Q. What is a deadlock?
A. Two or more threads each hold a lock the other needs, so all block forever. Prevent it with consistent lock ordering, timeouts (tryLock), or by avoiding nested locks.

Q. notify vs notifyAll?
A. notify wakes one arbitrary waiting thread; notifyAll wakes all of them (they then compete for the lock). Prefer notifyAll unless exactly one waiter is guaranteed.

Quiz

Q1. You call `thread.run()` instead of `thread.start()`. What happens?

[ ] A. A new thread runs the code
[x] B. The code runs on the current (calling) thread — no concurrency
[ ] C. Compile error
[ ] D. IllegalThreadStateException

> **Why:** run() is just a method call. start() is what asks the JVM to spawn a new thread and then invoke run() on it.

Q2. Which releases the object's monitor lock?

[ ] A. Thread.sleep(100)
[x] B. obj.wait()
[ ] C. Thread.yield()
[ ] D. t.join()

> **Why:** wait() releases the monitor so other threads can enter the synchronized block. sleep, yield and join all hold the lock.

Q3. What is the thread state after calling start() but before the OS schedules it?

[ ] A. NEW
[x] B. RUNNABLE
[ ] C. RUNNING
[ ] D. BLOCKED

> **Why:** start() moves NEW → RUNNABLE, meaning 'ready to run and waiting for CPU'. Java's Thread.State has no separate RUNNING state.

Q4. What exception do you get from wait() outside a synchronized block?

[ ] A. InterruptedException
[x] B. IllegalMonitorStateException
[ ] C. IllegalStateException
[ ] D. No exception

> **Why:** The thread doesn't own the object's monitor, so wait() (and notify) throw IllegalMonitorStateException.

Q5. How do you correctly stop a running thread in modern Java?

[ ] A. Thread.stop()
[x] B. Cooperatively: set a volatile flag or call interrupt(), and let the thread exit its loop

[ ] C. System.exit(0)
[ ] D. Thread.suspend()

> **Why:** stop(), suspend() and resume() are deprecated and unsafe (they leave objects half-updated). Use interrupt() plus a checked flag, or Future.cancel(true).

Q6. What is the output order guarantee when starting two threads t1 and t2?

[ ] A. t1 always finishes first
[x] B. No guarantee — thread scheduling is up to the JVM/OS
[ ] C. They run simultaneously and interleave deterministically
[ ] D. main() finishes first

> **Why:** Never rely on thread ordering. Use join(), CountDownLatch, or an ExecutorService if you need coordination.

6. Synchronization, volatile & Deadlock Must-know 50 min

The heart of Java concurrency interviews: `synchronized`, `volatile`, what each guarantees, and how to avoid deadlock. Learn the memory-model reason for volatile — most candidates can't explain it.

Key points

- `synchronized` gives **mutual exclusion** AND **visibility** (happens-before). It can be a method or a block; it locks on an object monitor (or the Class object for static methods).
- `synchronized` is **reentrant** — a thread holding the lock can enter other synchronized blocks on the same monitor (that's why a synchronized method can call another).
- `volatile` guarantees **visibility and ordering** but **not atomicity** — so `volatile int i; i++` is still not thread-safe.
- Use volatile for a **flag or a single-writer value** (e.g. a stop flag, a safely-published config reference) where the write doesn't depend on the current value.
- **Deadlock** needs four conditions: mutual exclusion, hold-and-wait, no preemption, circular wait. Break any one — usually circular wait, via consistent lock ordering.
- **Livelock**: threads keep reacting to each other but make no progress. **Starvation**: a thread never gets CPU/lock access (often priority-related).

Reference

Aspect	synchronized	volatile
Mutual exclusion	Yes	No
Visibility	Yes	Yes
Atomicity	Yes (for the whole block)	No — only single reads/writes
Blocking	Yes (thread blocks)	No (never blocks)
Reentrant	Yes	N/A
Applies to	Methods and blocks	Fields only
Compiler reordering	Prevents it across the boundary	Prevents it around the volatile access
Use for	Compound actions (check-then-act, read-modify-write)	Flags, status, one-time publication

Code

synchronized: method, block, static — what's the lock?

```
class Counter {
   private int a = 0, b = 0;
   private static int staticCount = 0;
   private final Object lock = new Object();

   public synchronized void incA() { a++; }      // locks on `this`
   public void incB() { synchronized (this) { b++; } }   // same lock

   public void incBoth() { synchronized (lock) { a++; b++; } } // private lock
   // ^ PREFERRED: a private lock object can't be locked by outside code,
   //   which prevents accidental deadlocks and broken encapsulation.

   public static synchronized void incStatic() { staticCount++; }
   // locks on Counter.class (the Class object), NOT on any instance
   // equivalent to: synchronized (Counter.class) { ... }
}
```

volatile: what it fixes — and what it doesn't

```
// WITHOUT volatile, this loop may NEVER terminate:
// the reading thread caches `running` in a register and never sees the update.
class Worker implements Runnable {
   private volatile boolean running = true;   // <-- volatile fixes visibility
   public void run() { while (running) { /* do work */ } }
   public void stop() { running = false; }    // another thread calls this
}

// What volatile does NOT do:
class Bad {
   private volatile int count = 0;
   public void increment() { count++; }   // STILL BROKEN
   // count++ = read, add, write - three steps. Two threads can read the same
   // value and both write count+1, losing an update.
   // Fix: AtomicInteger, synchronized, or LongAdder.
}

// Volatile IS enough when: one thread writes, others only read, and the new
// value doesn't depend on the old one (a flag, a published immutable object).
class Config {
   private volatile Map<String,String> settings = Map.of();  // safe publication
   void reload() { settings = loadFromDisk(); }   // atomic reference write
   String get(String k) { return settings.get(k); }
}
```

Deadlock: reproduce it, then fix it

```
// DEADLOCK: two locks acquired in opposite orders
Object lockA = new Object(), lockB = new Object();

Thread t1 = new Thread(() -> {
   synchronized (lockA) {
       sleep(50);
       synchronized (lockB) { System.out.println("t1 got both"); }
   }
});
Thread t2 = new Thread(() -> {
   synchronized (lockB) {                 // <-- reverse order = circular wait
       sleep(50);
       synchronized (lockA) { System.out.println("t2 got both"); }
   }
});
t1.start(); t2.start();   // hangs forever

// FIX 1 (best): always acquire locks in a CONSISTENT global order (A then B).
// FIX 2: use ReentrantLock with tryLock + timeout
if (lockA.tryLock(1, TimeUnit.SECONDS)) {
   try {
       if (lockB.tryLock(1, TimeUnit.SECONDS)) {
           try { /* critical section */ } finally { lockB.unlock(); }
       }
   } finally { lockA.unlock(); }
}
// FIX 3: avoid nested locking altogether - restructure so one lock suffices.
// FIX 4: detect it - `jstack <pid>` prints "Found one Java-level deadlock".
```

Producer–Consumer with wait/notifyAll (a common whiteboard task)

```
class BlockingBuffer {
   private final Queue<Integer> queue = new LinkedList<>();
   private final int CAPACITY = 5;

   public synchronized void produce(int v) throws InterruptedException {
       while (queue.size() == CAPACITY) wait();   // loop, not if!
       queue.add(v);
       notifyAll();                               // wake the consumers
   }

   public synchronized int consume() throws InterruptedException {
       while (queue.isEmpty()) wait();
       int v = queue.remove();
       notifyAll();                               // wake the producers
       return v;
   }
}
// In real code you'd use BlockingQueue instead:
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(5);
q.put(1);        // blocks when full
int v = q.take();// blocks when empty - no wait/notify needed
```

ReentrantLock, ReadWriteLock and AtomicInteger

```
// ReentrantLock - more flexible than synchronized
private final ReentrantLock lock = new ReentrantLock(true);  // true = fair
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); }        // ALWAYS in finally

// tryLock avoids blocking forever
if (lock.tryLock(500, TimeUnit.MILLISECONDS)) { try { ... } finally { lock.unlock(); } }

// ReadWriteLock: many concurrent readers, exclusive writer
private final ReadWriteLock rw = new ReentrantReadWriteLock();
private final Map<String,String> cache = new HashMap<>();
String read(String k) {
   rw.readLock().lock(); try { return cache.get(k); } finally { rw.readLock().unlock(); }
}
void write(String k, String v) {
   rw.writeLock().lock(); try { cache.put(k, v); } finally { rw.writeLock().unlock(); }
}

// AtomicInteger - lock-free, CAS-based
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();            // atomic ++
counter.addAndGet(5);
counter.compareAndSet(10, 20);        // CAS: set to 20 only if currently 10
AtomicReference<User> ref = new AtomicReference<>();
LongAdder adder = new LongAdder();    // faster than AtomicInteger under high contention
adder.increment(); adder.sum();
```

SAY IT LIKE THIS IN THE INTERVIEW

"synchronized provides both mutual exclusion and the visibility guarantees of happens-before, and it's reentrant. volatile gives visibility and prevents reordering but not atomicity, so I use it for flags and safe publication, never for compound operations like i++. For deadlock, the practical fix is consistent lock ordering, plus tryLock with a timeout; in production I'd diagnose with jstack or a profiler."

Flashcards

Q. What does synchronized guarantee?
A. Mutual exclusion (only one thread in the block) and visibility/happens-before (changes are flushed to main memory and seen by the next thread to acquire the lock).

Q. Is synchronized reentrant?
A. Yes — a thread that holds a monitor can re-enter any synchronized block on that same monitor, which prevents self-deadlock when synchronized methods call each other.

Q. What does volatile guarantee?
A. Visibility (writes are immediately visible to other threads), ordering (no reordering across the volatile access), and atomicity of single reads/writes — but NOT compound operations.

Q. Is `volatile int i; i++` thread-safe?
A. No. i++ is read-modify-write. Use AtomicInteger, synchronized, or LongAdder.

Q. What lock does a static synchronized method use?
A. The Class object (Counter.class), not an instance — so it's independent of any instance-level synchronized method.

Q. What are the four conditions for deadlock?
A. Mutual exclusion, hold-and-wait, no preemption, circular wait. Break any one (usually circular wait via consistent ordering) to prevent deadlock.

Q. Deadlock vs livelock vs starvation?
A. Deadlock: blocked forever holding locks. Livelock: threads keep responding to each other but make no progress. Starvation: a thread never gets resources.

Q. synchronized vs ReentrantLock?
A. ReentrantLock adds tryLock with timeout, fairness policy, interruptible locking and multiple conditions — but you must unlock in a finally block. Prefer synchronized for simplicity unless you need those features.

Q. How do you detect a deadlock in production?
A. Take a thread dump: `jstack <pid>` (it reports 'Found one Java-level deadlock'), or use JConsole/VisualVM/YourKit.

Q. How does AtomicInteger work?
A. It uses CAS (compare-and-swap) CPU instructions in a loop — lock-free and generally faster than synchronized for simple counters.

Quiz

Q1. Which of these is thread-safe?

[ ] A. private volatile int count; void inc(){ count++; }
[x] B. private int count; synchronized void inc(){ count++; }
[ ] C. private int count; void inc(){ count++; }
[ ] D. private static int count; void inc(){ count++; }

> **Why:** Only synchronized makes read-modify-write atomic. volatile alone doesn't — that's exactly the misconception to avoid.

Q2. What object does `synchronized` lock on in an instance method?

[ ] A. The class object
[x] B. `this` (the current instance)
[ ] C. A new Object each time
[ ] D. The method itself

> **Why:** An instance synchronized method locks the receiver (`this`). A static synchronized method locks the Class object.

Q3. How do you prevent deadlock when two locks are needed?

[ ] A. Use more threads
[x] B. Always acquire locks in the same global order (and consider tryLock with a timeout)
[ ] C. Make all methods synchronized
[ ] D. Use volatile instead

> **Why:** Consistent acquisition order removes the circular-wait condition. tryLock with a timeout adds a safety net; a private lock object prevents external interference.

Q4. What is the difference between notify() and notifyAll()?

[ ] A. notify wakes all, notifyAll wakes one
[x] B. notify wakes one arbitrary waiting thread; notifyAll wakes all waiters
[ ] C. They're identical

[ ] D. notifyAll is deprecated

> **Why:** notify() picks one arbitrary waiter, which can deadlock if the wrong thread wakes. notifyAll is safer; default to it.

Q5. Does making a field volatile make all operations on it atomic?

[ ] A. Yes
[x] B. No — only single reads and writes; compound operations still need synchronization
[ ] C. Yes, for ints only
[ ] D. Only in synchronized methods

> **Why:** volatile = visibility + ordering. Atomicity of compound actions (i++, check-then-act) requires synchronized, a Lock, or an atomic class.

Q6. Reentrancy means:

[x] A. A thread can acquire the same lock it already holds without deadlocking
[ ] B. Locks can be shared between threads
[ ] C. A method can be called recursively without a stack overflow
[ ] D. Locks are automatically released

> **Why:** Java monitors are reentrant: the JVM tracks the owner and a hold count, so a thread can re-enter any synchronized block it already owns.

7. ExecutorService & Concurrent Collections Must-know 45 min

"Don't create threads manually — use a pool" is the modern answer, and interviewers check that you know why, plus which pool to pick.

Key points

- Creating threads is expensive (each reserves ~1MB of stack) and unbounded threads exhaust memory. A **pool reuses** a fixed number of threads.
- `Executors.newFixedThreadPool(n)` — bounded pool with an unbounded queue. `newCachedThreadPool()` — unbounded, dangerous under load. `newSingleThreadExecutor()` — sequential. `newScheduledThreadPool()` — delays/periodics.
- Prefer constructing `ThreadPoolExecutor` explicitly in production so you control the queue size and rejection policy (the Executors factory defaults to an unbounded queue → OOM risk).
- `submit()` returns a `Future`; `get()` blocks; `invokeAll` runs a batch; `shutdown()` stops accepting new work and lets queued tasks finish (`shutdownNow()` interrupts).
- `Callable<V>` returns a value and can throw checked exceptions; `Runnable` cannot.
- Concurrent collections: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` (ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue), `ConcurrentLinkedQueue`.

Reference

Tool	Use it for
ExecutorService	Managing a pool of worker threads instead of new Thread()
Future<T>	A handle on an async result (get, cancel, isDone)
CompletableFuture<T>	Composing async pipelines: thenApply, thenCompose, allOf
ConcurrentHashMap	Thread-safe map without locking the whole structure
BlockingQueue	Producer–consumer handoff (put/take block)
CountDownLatch	Wait for N tasks to finish (one-shot)
CyclicBarrier	Make N threads wait for each other at a barrier (reusable)
Semaphore	Limit concurrent access to a resource (permits)

Code

ExecutorService: the pattern you should write from memory

```
ExecutorService pool = Executors.newFixedThreadPool(4);

// Fire-and-forget
pool.execute(() -> System.out.println("task"));

// With a result
Future<Integer> f = pool.submit(() -> {
   TimeUnit.MILLISECONDS.sleep(200);
   return 42;
});
// ... do other work ...
Integer result = f.get();                 // blocks; throws ExecutionException
Integer safe   = f.get(1, TimeUnit.SECONDS);  // throws TimeoutException

// Batch
List<Callable<Integer>> tasks = List.of(() -> 1, () -> 2, () -> 3);
List<Future<Integer>> futures = pool.invokeAll(tasks);
for (Future<Integer> fu : futures) System.out.println(fu.get());

// ALWAYS shut down - otherwise the JVM never exits
pool.shutdown();
try {
   if (!pool.awaitTermination(60, TimeUnit.SECONDS)) pool.shutdownNow();
} catch (InterruptedException e) { pool.shutdownNow(); Thread.currentThread().interrupt(); }

// Or use try-with-resources (Java 19+): ExecutorService implements AutoCloseable
```

Explicit ThreadPoolExecutor — the production-grade answer

```
ThreadPoolExecutor executor = new ThreadPoolExecutor(
   4,                              // corePoolSize
   8,                              // maximumPoolSize
   60, TimeUnit.SECONDS,           // keep-alive for idle extra threads
   new ArrayBlockingQueue<>(100),  // BOUNDED queue -> backpressure, not OOM
   new CustomThreadFactory("api-worker-"),
   new ThreadPoolExecutor.CallerRunsPolicy()   // rejection policy
);

// Rejection policies:
//   AbortPolicy (default)  -> throws RejectedExecutionException
//   CallerRunsPolicy       -> the submitting thread runs the task (natural throttle)
//   DiscardPolicy          -> silently drops it
//   DiscardOldestPolicy    -> drops the oldest queued task

// Sizing a pool:
//   CPU-bound    -> threads ≈ number of cores
//   I/O-bound    -> threads ≈ cores * (1 + waitTime/computeTime)
int cores = Runtime.getRuntime().availableProcessors();
```

CompletableFuture — composing async work (Java 8+)

```
CompletableFuture<String> cf = CompletableFuture
       .supplyAsync(() -> fetchUser(id), pool)      // runs asynchronously
       .thenApply(user -> user.getName())           // transform the result
       .thenApply(String::toUpperCase)
       .exceptionally(ex -> "FALLBACK");            // recover from errors

String name = cf.get();

// Run several calls in parallel and wait for ALL of them
CompletableFuture<User> u  = CompletableFuture.supplyAsync(() -> getUser(id));
CompletableFuture<List<Order>> o = CompletableFuture.supplyAsync(() -> getOrders(id));
CompletableFuture<Void> all = CompletableFuture.allOf(u, o);
all.join();                                          // wait for both
User user = u.join();
List<Order> orders = o.join();

// wait for the FIRST to complete
CompletableFuture<Object> first = CompletableFuture.anyOf(u, o);

// Chaining dependent async calls (flatMap equivalent)
CompletableFuture<Order> order = CompletableFuture
       .supplyAsync(() -> getUser(id))
       .thenCompose(user -> getLatestOrder(user.getId()));   // returns a Future
```

Concurrent collections and synchronizers in action

```
// ConcurrentHashMap - lock-free reads, per-bin writes
ConcurrentHashMap<String, Integer> hits = new ConcurrentHashMap<>();
hits.merge("page", 1, Integer::sum);        // atomic increment
hits.computeIfAbsent("k", k -> expensiveLoad(k));
// Note:CHM iterators are weakly consistent (no CME), and nulls are forbidden

// BlockingQueue - the backbone of producer/consumer
BlockingQueue<String> q = new ArrayBlockingQueue<>(100);
q.put("job");            // blocks if full
String job = q.take();   // blocks if empty
q.offer("job", 1, TimeUnit.SECONDS);   // returns false on timeout
q.poll(1, TimeUnit.SECONDS);           // returns null on timeout

// CopyOnWriteArrayList - read-heavy, rare writes (listeners, observers)
List<Listener> listeners = new CopyOnWriteArrayList<>();
// Every write copies the whole array -> great reads, expensive writes

// CountDownLatch: start N workers, wait for all to finish
CountDownLatch latch = new CountDownLatch(3);
for (int i = 0; i < 3; i++)
   pool.submit(() -> { try { doWork(); } finally { latch.countDown(); } });
latch.await(10, TimeUnit.SECONDS);   // main resumes when the count hits 0

// CyclicBarrier: N threads wait for each other, then all proceed (reusable)
CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("all arrived"));

// Semaphore: allow only N concurrent accesses (connection limiter)
Semaphore sem = new Semaphore(5);
sem.acquire(); try { useResource(); } finally { sem.release(); }
```

SAY IT LIKE THIS IN THE INTERVIEW

"I never create threads directly in production — I submit tasks to an ExecutorService so threads are reused and the pool can be bounded. I construct ThreadPoolExecutor explicitly rather than using the Executors factories, because the default unbounded queue can cause an OOM under load. For results I use Future or CompletableFuture when I need to compose async work, and ConcurrentHashMap or a BlockingQueue instead of synchronizing collections by hand."

Flashcards

Q. Why use a thread pool instead of new Thread()?
A. Thread creation is expensive (~1MB stack each) and unbounded threads exhaust memory. A pool reuses a fixed set of workers, bounds concurrency and adds queueing, monitoring and rejection policies.

Q. Runnable vs Callable?
A. Runnable.run() returns void and can't throw checked exceptions. Callable.call() returns a value and can throw — submit it to an ExecutorService to get a Future.

Q. What is a Future?
A. A handle to an asynchronous computation: get() blocks for the result, cancel() attempts cancellation, isDone() polls. Exceptions surface as ExecutionException.

Q. shutdown vs shutdownNow?
A. shutdown() stops accepting new tasks and lets queued/running ones finish. shutdownNow() attempts to interrupt running tasks and returns the pending queue.

Q. What's wrong with Executors.newCachedThreadPool()?
A. It creates unbounded threads under bursty load — each with its own stack — which can OOM. Prefer a fixed pool backed by a bounded queue.

Q. What is CallerRunsPolicy?
A. A ThreadPoolExecutor rejection policy where the submitting thread runs the rejected task itself — which applies backpressure and slows the producer down.

Q. How do you size a thread pool?
A. CPU-bound: ≈ number of cores. I/O-bound: cores × (1 + wait/compute time). Measure and tune; the queue must stay bounded.

Q. CountDownLatch vs CyclicBarrier?
A. CountDownLatch is a one-shot countdown (N events → waiting threads proceed). CyclicBarrier makes N threads wait for each other and is reusable.

Q. What's a Semaphore used for?
A. Limiting concurrent access to a resource — acquire() takes a permit, release() gives it back. Think connection pools and rate limiting.

Q. BlockingQueue vs ConcurrentLinkedQueue?
A. BlockingQueue blocks on put/take (perfect for producer–consumer, supports bounds). ConcurrentLinkedQueue is non-blocking and unbounded.

Quiz

Q1. What is the main risk of `Executors.newCachedThreadPool()`?

[ ] A. It's too slow
[x] B. It creates unbounded threads under load, which can exhaust memory
[ ] C. It can't run Callables
[ ] D. It reuses only one thread

> **Why:** It has no upper bound and a synchronous handoff queue, so a burst of tasks creates a thread per task. In production, bound both the pool and its queue.

Q2. Which returns a value from an async task?

[ ] A. Runnable
[x] B. Callable
[ ] C. Thread
[ ] D. Executor

> **Why:** Callable<V>.call() returns V and may throw checked exceptions; Runnable returns void.

Q3. What happens if you forget to shut down an ExecutorService?

[ ] A. Nothing, GC cleans it up
[x] B. Its non-daemon worker threads keep the JVM alive — the app may never exit
[ ] C. It throws after 60 seconds
[ ] D. Tasks are silently dropped

Why: Pool threads are non-daemon and stay alive awaiting work. Always call shutdown() (in a finally or @PreDestroy).

Q4. Which is the safest way to increment a counter in ConcurrentHashMap?

[ ] A. map.put(k, map.get(k) + 1)
[x] B. map.merge(k, 1, Integer::sum)
[ ] C. synchronized (map) { map.put(k, map.get(k)+1); }
[ ] D. map.computeIfPresent

Why: merge (and compute/computeIfAbsent) perform the read-modify-write atomically. A manual get-then-put races with other threads.

Q5. What does CompletableFuture.thenCompose do vs thenApply?

[ ] A. They're the same
[x] B. thenApply transforms the value (map); thenCompose flattens a nested future (flatMap)
[ ] C. thenCompose runs synchronously
[ ] D. thenApply handles errors

Why: thenApply gives you CompletableFuture<CompletableFuture<T>> if your function returns a Future; thenCompose flattens that to CompletableFuture<T> — exactly map vs flatMap.

Q6. You need to run 5 independent API calls and wait for all before continuing. Which tool?

[ ] A. CountDownLatch only
[x] B. CompletableFuture.allOf(...).join()
[ ] C. A single thread
[ ] D. Semaphore

Why: allOf returns a future that completes when every given future completes; join() then blocks until they're all done.

8. File I/O & Serialization Core 30 min

Not deeply tested, but cheap marks: the NIO.2 API (`Files`/`Paths`) and the serialization rules (`Serializable`, `serialVersionUID`, `transient`).

Key points

- `java.io` (streams: `FileInputStream`, `BufferedReader`) is the classic API; **NIO.2** (`java.nio.file.Files`, `Paths`) is the modern one — prefer it.
- **Serialization**: converting an object to a byte stream. Implement `Serializable` (a marker interface).
- Declare `static final long serialVersionUID` — without it, any class change invalidates existing serialized data (`InvalidClassException`).
- `transient` fields are **skipped** during serialization (passwords, caches, non-serializable members).
- Static fields are not serialized (they belong to the class, not the object). Deserialization does **not** call the constructor.
- Serialization is considered fragile for persistence — JSON (Jackson) is preferred in modern apps.

Code

NIO.2 — the modern file API (Java 7+)

```
Path path = Paths.get("data", "users.txt");     // platform-independent
Path abs  = path.toAbsolutePath();

// Read
List<String> lines   = Files.readAllLines(path);          // small files
String content       = Files.readString(path);            // Java 11
try (Stream<String> s = Files.lines(path)) {              // large files, lazy
   s.filter(l -> l.contains("ERROR")).forEach(System.out::println);
}
try (BufferedReader br = Files.newBufferedReader(path)) {
   String line; while ((line = br.readLine()) != null) { }
}

// Write
Files.write(path, List.of("a", "b"));
Files.writeString(path, "hello");                          // Java 11
Files.write(path, data, StandardOpenOption.APPEND);

// Inspect and manipulate
Files.exists(path); Files.isDirectory(path); Files.size(path);
Files.copy(src, dst, StandardCopyOption.REPLACE_EXISTING);
Files.move(src, dst);
Files.deleteIfExists(path);
Files.createDirectories(Paths.get("a/b/c"));
Files.walk(Paths.get("src")).filter(p -> p.toString().endsWith(".java")).toList();

// Always close streams - or use try-with-resources (NIO.2 streams are lazily
// loaded, so an unclosed Files.lines() leaks a file handle).
```

Serialization: the rules

```
class User implements Serializable {
   private static final long serialVersionUID = 1L;   // RECOMMENDED

   private String name;
   private transient String password;    // NOT serialized
   private static int count;             // NOT serialized (static)

   // Constructors are NOT called during deserialization
}

// Write
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
   oos.writeObject(user);
}
// Read
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser"))) {
   User restored = (User) ois.readObject();   // ClassNotFoundException possible
   System.out.println(restored.getName());    // "Parvez"
   System.out.println(restored.getPassword());// null - transient was skipped
}

// Customize with writeObject/readObject for encryption or version migration:
private void writeObject(ObjectOutputStream o) throws IOException {
   o.defaultWriteObject();
   o.writeUTF(encrypt(password));
}
```

Output:
```
Parvez
null
```

Why serialization is falling out of favour

```
// Java serialization problems:
//  - Tightly coupled to the Java class (version-fragile, language-specific)
//  - A known attack surface - deserializing untrusted data is dangerous
//    (gadget chains -> RCE). Java 17 added deserialization filters.
//  - Binary, unreadable, hard to diff or debug

// Modern alternative: JSON with Jackson
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(user);        // serialize
User u      = mapper.readValue(json, User.class);     // deserialize
List<User> users = mapper.readValue(json,
       new TypeReference<List<User>>() {});          // generics need TypeReference

// That's exactly what Spring Boot uses for REST by default.
```

SAY IT LIKE THIS IN THE INTERVIEW

"For file work I use NIO.2 — Files and Paths — which is cleaner and throws better exceptions than the old io streams, and I always close streams with try-with-resources. Serialization converts an object to bytes via the Serializable marker; I declare serialVersionUID to keep compatibility across versions and mark sensitive or non-serializable fields transient. In modern apps I'd usually serialize to JSON with Jackson instead."

Flashcards

Q. What is serialVersionUID and why declare it?
A. A version stamp for a Serializable class. If it changes (or is absent and the class structure changes), deserializing old data throws InvalidClassException.

Q. What does transient do?
A. Marks a field to be skipped during serialization — used for passwords, derived data and non-serializable members.

Q. Is a static field serialized?
A. No — static fields belong to the class, not the object. Neither are transient-marked fields.

Q. Is the constructor called during deserialization?
A. No. The object is rebuilt from the byte stream without invoking any constructor.

Q. java.io vs java.nio.file?
A. java.io is the legacy stream API; NIO.2 (java.nio.file.Files/Paths) is modern, supports symbolic links, file attributes, directory walking and better exceptions.

Q. Why is Java serialization considered risky?
A. Deserializing untrusted data can execute attacker-controlled code via gadget chains. Prefer JSON and validate input; use serialization filters if you must.

Q. How do you read a large file without loading it all into memory?
A. Stream it: Files.lines(path) or a BufferedReader inside try-with-resources.

Quiz

Q1. Which field would NOT be serialized?

[ ] A. private String name
[x] B. private transient String token
[ ] C. private int age
[ ] D. private List<String> tags

> **Why:** transient fields are excluded from the serialized form (restored as their default value). Static fields are also excluded.

Q2. What exception is thrown if the class's serialVersionUID doesn't match the serialized data?

[ ] A. ClassNotFoundException
[x] B. InvalidClassException
[ ] C. NotSerializableException
[ ] D. IOException

> **Why:** InvalidClassException (an IOException subclass) signals a version/structure mismatch. NotSerializableException means the class doesn't implement Serializable.

Q3. What happens if you serialize an object with a non-serializable field?

[ ] A. It's silently skipped
[x] B. NotSerializableException at runtime (unless the field is transient)
[ ] C. Compile error
[ ] D. It's serialized as null

> **Why:** Every non-transient field must be serializable, otherwise the write fails at runtime with NotSerializableException.

Q4. Which is the modern way to read all lines of a file?

[ ] A. new BufferedReader(new FileReader(f))
[x] B. Files.readAllLines(Paths.get(f))
[ ] C. System.in
[ ] D. Files.read()

> **Why:** NIO.2's Files utility is concise and handles charset and resource management better. For big files, prefer Files.lines() as a stream.

9. JDBC — Connecting Java to a Database Must-know 45 min

Even in a Spring Boot interview they'll check that you know what happens underneath Hibernate: JDBC, PreparedStatement, transactions. And "why PreparedStatement over Statement" is a guaranteed question because it's about SQL injection.

Key points

- **7 steps**: load the driver (automatic since JDBC 4) → get a Connection → create a Statement → execute a query → process the ResultSet → close everything → handle SQLException.
- **PreparedStatement vs Statement**: precompiled, prevents SQL injection, faster for repeated execution, supports parameter binding and LOBs/setDate.
- **Always use try-with-resources** — leaked connections exhaust the pool fast.
- **Transactions**: `setAutoCommit(false)` → work → `commit()` / `rollback()` in the catch block. Default is auto-commit = true.
- `Statement.executeQuery()` → SELECT (returns ResultSet); `executeUpdate()` → INSERT/UPDATE/DELETE (returns the row count); `execute()` → any.
- **Connection pooling** (HikariCP is the Spring Boot default) reuses physical connections — opening one is expensive.

Code

A complete, correct JDBC query (memorise this shape)

```
String sql = "SELECT id, name, email FROM users WHERE dept = ? AND active = ?";

try (Connection conn = dataSource.getConnection();          // 1. connection
    PreparedStatement ps = conn.prepareStatement(sql)) {   // 2. statement

   ps.setString(1, "Engineering");    // 3. bind parameters (1-indexed!)
   ps.setBoolean(2, true);

   try (ResultSet rs = ps.executeQuery()) {                // 4. execute
       while (rs.next()) {                                 // 5. iterate
           long id    = rs.getLong("id");
           String name = rs.getString("name");
           String email = rs.getString("email");
           System.out.printf("%d %s %s%n", id, name, email);
       }
   }
} catch (SQLException e) {                                  // 6. handle
   throw new DataAccessException("query failed", e);
}   // 7. everything auto-closed by try-with-resources
```

SQL injection — why PreparedStatement matters

```
String userInput = "' OR '1'='1";

// VULNERABLE: string concatenation
String bad = "SELECT * FROM users WHERE name = '" + userInput + "'";
// -> SELECT * FROM users WHERE name = '' OR '1'='1'   -> returns EVERY row!
// Worse: input like "'; DROP TABLE users; --"

// SAFE: parameter binding. The driver sends the SQL and the values separately,
// so the input can never be parsed as code.
String good = "SELECT * FROM users WHERE name = ?";
PreparedStatement ps = conn.prepareStatement(good);
ps.setString(1, userInput);      // treated strictly as DATA
```

Transactions — the classic money-transfer question

```
public void transfer(long fromId, long toId, BigDecimal amount) throws SQLException {
   try (Connection conn = dataSource.getConnection()) {
       conn.setAutoCommit(false);              // 1. begin transaction
       try {
           debit(conn, fromId, amount);        // 2. work
           credit(conn, toId, amount);
           conn.commit();                      // 3. commit on success
       } catch (SQLException e) {
           conn.rollback();                    // 4. undo on ANY failure
           throw e;
       } finally {
           conn.setAutoCommit(true);           // 5. restore for pooled reuse
       }
   }
}
// ACID: Atomicity (all or nothing), Consistency (invariants hold),
// Isolation (concurrent transactions don't interfere), Durability (committed = persisted)

// Isolation levels (weakest -> strongest), set with:
// conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
//   READ_UNCOMMITTED -> dirty reads possible
//   READ_COMMITTED   -> default in Postgres/Oracle; no dirty reads
//   REPEATABLE_READ  -> same row reads give the same values (MySQL default)
//   SERIALIZABLE     -> fully isolated, slowest
```

INSERT with a generated key, and batching

```
// Get the auto-generated primary key back
String sql = "INSERT INTO users(name, email) VALUES (?, ?)";
try (PreparedStatement ps = conn.prepareStatement(sql,
                            Statement.RETURN_GENERATED_KEYS)) {
   ps.setString(1, "Parvez");
   ps.setString(2, "p@example.com");
   int rows = ps.executeUpdate();            // number of affected rows
   try (ResultSet keys = ps.getGeneratedKeys()) {
       if (keys.next()) long newId = keys.getLong(1);
   }
}

// Batch insert - one round trip instead of N
try (PreparedStatement ps = conn.prepareStatement(sql)) {
   for (User u : users) {
       ps.setString(1, u.getName());
       ps.setString(2, u.getEmail());
       ps.addBatch();                        // queue it
       if (++i % 500 == 0) ps.executeBatch(); // flush periodically
   }
   ps.executeBatch();                        // flush the remainder
}
```

Connection pooling with HikariCP

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
config.setUsername("root");
config.setPassword("secret");
config.setMaximumPoolSize(10);          // cap concurrent connections
config.setMinimumIdle(2);
config.setConnectionTimeout(30_000);    // fail fast rather than hang
config.setIdleTimeout(600_000);
config.setAutoCommit(false);

HikariDataSource ds = new HikariDataSource(config);
try (Connection c = ds.getConnection()) { /* returns to the pool on close */ }

// Why: opening a TCP connection + authenticating costs ~10-50ms.
// A pool keeps them warm and hands out the same physical connections.
// Spring Boot auto-configures HikariCP - it's the default since Boot 2.
```

SAY IT LIKE THIS IN THE INTERVIEW

"JDBC is the layer under JPA: get a connection from the pool, prepare a statement, bind parameters, execute, process the ResultSet and close everything in try-with-resources. I always use PreparedStatement — it's precompiled, faster on repeat, and critically it prevents SQL injection because the values are sent separately from the SQL. For writes I turn off auto-commit, commit on success and roll back on any exception."

Flashcards

Q. What are the 7 JDBC steps?
A. Register the driver (automatic since JDBC 4), open a Connection, create a Statement/PreparedStatement, execute the query, process the ResultSet, close the resources, handle SQLException.

Q. Statement vs PreparedStatement?
A. PreparedStatement is precompiled, allows parameter binding, prevents SQL injection, handles dates/LOBs cleanly, and is faster when executed repeatedly. Prefer it always.

Q. How does PreparedStatement prevent SQL injection?
A. The SQL is compiled first with placeholders; user values are then sent as data and can never be parsed as SQL syntax.

Q. executeQuery vs executeUpdate vs execute?
A. executeQuery → SELECT, returns a ResultSet. executeUpdate → INSERT/UPDATE/DELETE, returns the affected row count. execute → any statement, returns a boolean indicating whether a ResultSet was produced.

Q. How do you manage a JDBC transaction?
A. setAutoCommit(false), do the work, commit() on success, rollback() in the catch block, and restore auto-commit in a finally.

Q. What is connection pooling and why?
A. A pool (HikariCP) keeps physical DB connections open and reuses them, avoiding the ~10-50ms cost of establishing and authenticating each connection. Spring Boot uses HikariCP by default.

Q. Are JDBC parameters 0-indexed or 1-indexed?
A. 1-indexed — ps.setString(1, value) refers to the first `?` placeholder.

Q. Name the four ACID properties.
A. Atomicity (all-or-nothing), Consistency (invariants preserved), Isolation (transactions don't interfere), Durability (committed data survives crashes).

Q. What are the transaction isolation levels?
A. READ_UNCOMMITTED (dirty reads), READ_COMMITTED (default for Postgres/Oracle), REPEATABLE_READ (MySQL default), SERIALIZABLE (fully isolated, slowest).

Q. How do you get the generated key after an INSERT?
A. Prepare with Statement.RETURN_GENERATED_KEYS, then read ps.getGeneratedKeys().

Quiz

Q1. Why should you use PreparedStatement instead of Statement?

[ ] A. It's the only way to run SELECT
[x] B. It's precompiled, faster on reuse, and prevents SQL injection via parameter binding
[ ] C. Statement is deprecated
[ ] D. It automatically commits transactions

Why: All of the above benefits — and the security one is the headline. Never build SQL by concatenating user input.

Q2. What does executeUpdate() return?

[ ] A. A ResultSet
[x] B. The number of rows affected
[ ] C. A boolean
[ ] D. The generated key

Why: executeUpdate returns an int row count for INSERT/UPDATE/DELETE. Use executeQuery for SELECT and getGeneratedKeys for keys.

Q3. What is the default value of autoCommit, and what does it mean?

[ ] A. false — you must always commit
[x] B. true — each statement is committed immediately as its own transaction
[ ] C. It depends on the driver
[ ] D. null

Why: By default each SQL statement commits as soon as it executes. For multi-statement units of work (like a transfer) you must set autoCommit(false) and manage commit/rollback yourself.

Q4. Which ResultSet method moves the cursor to the next row?

[x] A. rs.next()
[ ] B. rs.moveNext()
[ ] C. rs.hasNext()
[ ] D. rs.forward()

Why: rs.next() returns false when there are no more rows — the standard while (rs.next()) loop.

Q5. Why is a connection pool important?

[ ] A. It makes SQL faster
[x] B. Opening a real DB connection is expensive (TCP + auth), so connections are created once and reused
[ ] C. JDBC requires one
[ ] D. It prevents SQL injection

Why: Pooling avoids per-request connection setup cost and bounds the number of concurrent DB connections, which protects the database.

10. Design Patterns You Must Know Must-know 40 min

For a fresher they'll ask for 2–3 patterns, almost always including Singleton. Learn to write Singleton three ways and explain the reflection/serialization attacks.

Key points

- **Creational**: Singleton, Factory, Builder, (Prototype).
- **Structural**: Adapter, Decorator, Facade, Proxy.
- **Behavioural**: Strategy, Observer, Template Method, Command, Iterator.
- **Singleton**: private constructor + static instance. Must handle reflection, serialization and cloning.
- The **enum singleton** (Joshua Bloch) is the recommended implementation — JVM-guaranteed uniqueness and serialization safety.
- **Builder** shines with many optional parameters (and Lombok's `@Builder` generates it).
- Spring itself is patterns: DI (Factory), `@Transactional` (Proxy), `JdbcTemplate` (Template Method), events (Observer).

Code

Singleton — five implementations, from fragile to bulletproof

```
// 1. Eager: simple, thread-safe, created even if unused
class Eager {
   private static final Eager INSTANCE = new Eager();
   private Eager() { }
   public static Eager getInstance() { return INSTANCE; }
}

// 2. Lazy synchronized: thread-safe but every call pays for the lock
class LazySync {
   private static LazySync instance;
   private LazySync() { }
   public static synchronized LazySync getInstance() {
       if (instance == null) instance = new LazySync();
       return instance;
   }
}

// 3. Double-checked locking: lazy + fast; `volatile` is MANDATORY (Java 5+)
class DCL {
   private static volatile DCL instance;
   private DCL() {
       if (instance != null) throw new IllegalStateException("Already created");
   }
   public static DCL getInstance() {
       if (instance == null) {                      // first check (no lock)
           synchronized (DCL.class) {
               if (instance == null) instance = new DCL();  // second check
           }
       }
       return instance;
   }
}

// 4. Holder idiom (Bill Pugh): lazy, lock-free, thread-safe - class init is atomic
class Holder {
   private Holder() { }
   private static class Inner { static final Holder INSTANCE = new Holder(); }
   public static Holder getInstance() { return Inner.INSTANCE; }
}

// 5. ENUM - the recommended one (Effective Java, Joshua Bloch)
enum EnumSingleton {
   INSTANCE;
   public void doWork() { }
}
// Free serialization safety (no readResolve needed), reflection-proof, one instance.

// Breaking a naive singleton (the follow-up question):
//  - Reflection: setAccessible(true) on the private ctor  -> guard by throwing
//  - Serialization: deserialize creates a NEW object      -> implement readResolve()
//  - Cloning: clone() creates a copy                      -> don't implement Cloneable
protected Object readResolve() { return INSTANCE; }
```

Builder — for objects with many optional fields

```
public class User {
   private final String name;      // required
   private final String email;     // required
   private final int age;          // optional
   private final String phone;     // optional

   private User(Builder b) { this.name = b.name; this.email = b.email;
                             this.age = b.age; this.phone = b.phone; }

   public static class Builder {
       private final String name;
       private final String email;
       private int age; private String phone;

       public Builder(String name, String email) {
           this.name = name; this.email = email;
       }
       public Builder age(int age)         { this.age = age; return this; }
       public Builder phone(String phone)  { this.phone = phone; return this; }
       public User build() {
           if (email == null || !email.contains("@"))
               throw new IllegalStateException("invalid email");
           return new User(this);
       }
   }
}

User u = new User.Builder("Parvez", "p@example.com")
            .age(24)
            .phone("99999")
            .build();
// vs. telescoping constructors: new User("Parvez","p@x.com", 0, null)

// Lombok equivalent: @Builder on the class -> User.builder().name("x").build()
```

Factory, Strategy and Template Method — the three you'll use most

```
// FACTORY: hide the instantiation logic behind a common interface
interface Notification { void send(String msg); }
class EmailNotification implements Notification { public void send(String m){ } }
class SmsNotification   implements Notification { public void send(String m){ } }

class NotificationFactory {
   public static Notification of(String type) {
       return switch (type.toLowerCase()) {
           case "email" -> new EmailNotification();
           case "sms"   -> new SmsNotification();
           default -> throw new IllegalArgumentException("unknown: " + type);
       };
   }
}

// STRATEGY: swap an algorithm at runtime (this is what DI gives you for free)
interface PaymentStrategy { void pay(double amount); }
class Checkout {
   private final PaymentStrategy strategy;      // injected
   Checkout(PaymentStrategy s) { this.strategy = s; }
   void checkout(double amt) { strategy.pay(amt); }
}
new Checkout(new CardPayment()).checkout(100);

// TEMPLATE METHOD: fixed algorithm, subclass supplies the steps
abstract class ReportGenerator {
   public final String generate() {        // final -> the skeleton can't change
       return header() + body() + footer();
   }
   protected abstract String body();
   protected String header() { return "--- Report ---\n"; }
   protected String footer() { return "--- End ---\n"; }
}
```

Patterns Spring uses — great discussion material

```
// Singleton   - Spring beans are singletons by scope (one per container)
// Factory     - BeanFactory / ApplicationContext create beans for you
// Proxy       - @Transactional, @Cacheable, @Async are AOP proxies
// Template    - JdbcTemplate, RestTemplate, RabbitTemplate (boilerplate removed)
// Observer    - ApplicationEventPublisher / @EventListener
// MVC         - DispatcherServlet + @Controller + ViewResolver
// DI/IoC      - the container injects dependencies instead of you new-ing them
// Builder     - RestClient.builder(), ResponseEntity.ok().body(x).build()
// Front Ctrl  - DispatcherServlet handles every request and routes it

// Being able to say "Spring's @Transactional is a proxy-based implementation of
// the Decorator/Proxy pattern" is a strong, concrete answer.
```

SAY IT LIKE THIS IN THE INTERVIEW

"I know the classic GoF patterns by category — creational like Singleton, Factory and Builder; structural like Adapter, Decorator and Proxy; behavioural like Strategy, Observer and Template Method. For Singleton I'd normally use the enum implementation: the JVM guarantees a single instance and it's safe against reflection and serialization. And I can point at where Spring uses each — @Transactional is a proxy, JdbcTemplate is template method, events are observer."

Flashcards

Q. What is the Singleton pattern and how do you implement it safely?
A. One instance per JVM: private constructor + a static accessor. The enum implementation is preferred (serialization- and reflection-safe); alternatives are the holder idiom and double-checked locking with volatile.

Q. How can a Singleton be broken?
A. Reflection (setAccessible on the private constructor), serialization (a new instance on deserialize — fix with readResolve), and cloning (don't implement Cloneable). Multiple classloaders also create one per loader.

Q. Why is volatile needed in double-checked locking?
A. Without it, another thread can see a partially constructed object due to instruction reordering (the reference published before the constructor finished). volatile establishes the happens-before needed.

Q. When is the Builder pattern better than constructors?
A. With many optional parameters — it avoids telescoping constructors, keeps objects immutable, and lets you validate in build().

Q. Factory vs Abstract Factory?
A. Factory creates one type of object based on input (NotificationFactory.of("email")). Abstract Factory is a factory of factories, producing families of related objects.

Q. What is the Strategy pattern used for?
A. Encapsulating interchangeable algorithms behind an interface so you can swap behaviour at runtime — the basis of dependency injection.

Q. Template Method vs Strategy?
A. Template Method uses inheritance: a fixed algorithm in an abstract class with abstract steps. Strategy uses composition: an injected interface. Strategy is usually preferred (more flexible, testable).

Q. Which Spring feature is an example of the Proxy pattern?
A. @Transactional (and @Cacheable, @Async) — Spring wraps the bean in an AOP proxy that adds behaviour around the method call.

Q. What is the DAO pattern?
A. Data Access Object — an interface + implementation that hides persistence details (SQL/JPA) from the service layer, so the business logic is decoupled from the database.

Quiz

Q1. Which Singleton implementation is recommended in Effective Java?

[ ] A. Lazy synchronized
[ ] B. Double-checked locking
[x] C. Enum singleton

[ ] D. Public static field

> **Why:** The enum singleton is concise, provides free serialization safety, and the JVM guarantees a single instance — no readResolve or reflection guards needed.

Q2. Why is `volatile` required in double-checked locking?

[ ] A. For performance
[x] B. To prevent another thread from seeing a partially constructed instance due to reordering
[ ] C. It isn't required
[ ] D. To make the lock reentrant

> **Why:** Since Java 5, volatile's happens-before guarantee makes the safe publication work. Without it, DCL is broken.

Q3. Which pattern would you use to create objects of related families without specifying concrete classes?

[ ] A. Singleton
[x] B. Abstract Factory
[ ] C. Observer
[ ] D. Adapter

> **Why:** Abstract Factory produces families of related objects (e.g. a UI kit factory returning matching Button + Checkbox for a given OS theme).

Q4. Which pattern removes the need for if/else chains when selecting an algorithm?

[x] A. Strategy
[ ] B. Singleton
[ ] C. Facade
[ ] D. Decorator

> **Why:** Strategy encapsulates each algorithm behind a common interface and lets you inject the one you want — replacing conditional branching with polymorphism.

Q5. JdbcTemplate is an example of which pattern?

[ ] A. Singleton
[x] B. Template Method
[ ] C. Observer
[ ] D. Builder

> **Why:** It fixes the algorithm (get connection, create statement, execute, handle exceptions, close) and lets you supply just the varying part (row mapping / SQL).

Day 4: Full Stack: Spring Boot, JPA, REST & SQL

The framework stack — IoC, Boot, REST, persistence, SQL

10 topics · 95 flashcards · 50 quiz questions · ~6.8 hours

> **Goal.** Be able to explain your project's architecture end to end: how a request travels from the client through a controller, service and repository to the database — and back.

Schedule

- Slot 1 (2h): Spring Core — IoC, DI, beans, scopes, annotations
- Slot 2 (2h): Spring Boot — auto-config, starters, properties, profiles
- Slot 3 (2h): REST controllers, DTOs, validation, exception handling
- Slot 4 (2.5h): JPA/Hibernate — entities, relationships, repositories, transactions
- Slot 5 (1.5h): SQL revision + project walkthrough out loud

1. Spring Core: IoC, DI & Beans Must-know 45 min

Two questions open almost every Spring interview: "What is IoC/DI?" and "What is a bean and what are its scopes?" Answer both crisply and the rest of the round goes smoothly.

Key points

- **IoC (Inversion of Control)**: you stop creating and wiring objects yourself; the container does it. "Don't call us, we'll call you."
- **DI (Dependency Injection)**: the mechanism — dependencies are *pushed into* a class rather than created inside it. Three types: **constructor** (recommended), setter, field.
- **Bean** = any object managed by the Spring container (instantiated, wired, and destroyed by it). The container is the `ApplicationContext`.
- Prefer **constructor injection** — it enables `final` fields, makes dependencies explicit, is immutable, and is easy to unit-test without Spring.
- Stereotypes: `@Component` (generic), `@Service` (business logic), `@Repository` (data access, translates DB exceptions), `@Controller`/`@RestController` (web).
- **Bean scopes**: singleton (default, one per container), prototype (new each request), request, session, application (web-only last three).

Reference

Annotation	Layer	What it does
@Component	Any	Generic Spring-managed bean
@Service	Service	Business logic — @Component with semantic meaning
@Repository	DAO	Data access + automatic exception translation to DataAccessException
@Controller	Web	MVC controller returning a view
@RestController	Web	@Controller + @ResponseBody → returns JSON
@Configuration	Config	Declares @Bean factory methods
@Autowired	Any	Injects a dependency (constructor, setter or field)
@Qualifier	Any	Disambiguates when multiple candidates exist

Code

IoC/DI: the before and after that explains everything

```
// WITHOUT Spring: you create and wire everything yourself (tightly coupled)
class OrderService {
   private OrderRepository repo = new JdbcOrderRepository();  // hard-coded!
   private PaymentClient pay    = new StripePaymentClient();  // hard-coded!
}
// Problems: can't swap implementations, can't unit test in isolation,
//           construction logic duplicated everywhere.

// WITH Spring: the container injects what's needed (loosely coupled)
@Service
public class OrderService {
   private final OrderRepository repo;
   private final PaymentClient pay;

   public OrderService(OrderRepository repo, PaymentClient pay) {  // constructor DI
       this.repo = repo;
       this.pay  = pay;
   }
}
// Spring sees the constructor, finds a bean of each type, and injects them.
// Fields can be final -> immutable and thread-safe by default.

@Repository
public class JpaOrderRepository implements OrderRepository { /* JPA impl */ }

// Swapping the implementation is now a one-line config change, and unit tests
// can pass a mock: new OrderService(mockRepo, mockPay);
```

Bean scopes in one place

```
// singleton (DEFAULT) - one instance shared by the whole container
@Service                                  // implicitly @Scope("singleton")
class StatsService { }

// prototype - a brand-new instance every time it's injected or fetched
@Component
@Scope("prototype")
class ReportBuilder { }

// web-aware scopes
@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST, proxyMode = ScopedProxyMode.TARGET_CLASS)
class RequestContext { }       // one per HTTP request

@Component
@Scope("session")
class ShoppingCart { }         // one per user session

// Defining beans manually with @Configuration + @Bean
@Configuration
class AppConfig {
   @Bean
   public ObjectMapper objectMapper() {
       return new ObjectMapper()
               .registerModule(new JavaTimeModule())
               .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
   }

   @Bean
   @Profile("dev")                  // only active when the dev profile is on
   public DataSource devDataSource() { /* H2 */ }

   @Bean
   @Profile("prod")
   public DataSource prodDataSource() { /* HikariCP + Postgres */ }
}

// Bean lifecycle: instantiate -> populate properties -> setBeanName ->
// BeanPostProcessors -> @PostConstruct init -> READY -> @PreDestroy on shutdown
@PostConstruct  public void init()   { /* after wiring */ }
@PreDestroy     public void cleanup(){ /* before removal */ }
```

Constructor vs field vs setter injection — and the @Autowired trap

```
// BEST: constructor injection (Spring 4.3+ doesn't even need @Autowired
// when there's a single constructor)
@Service
class GoodService {
   private final Repo repo;
   public GoodService(Repo repo) { this.repo = repo; }   // final, testable, explicit
}

// AVOID: field injection
@Service
class MehService {
   @Autowired private Repo repo;    // can't be final, hard to unit test,
                                    // hides dependencies, allows circular refs
}

// Resolving ambiguity when two beans implement the same interface
@Repository("jdbcRepo")  class JdbcRepo implements Repo { }
@Repository("jpaRepo")   class JpaRepo  implements Repo { }

@Service
class Uses {
   private final Repo repo;
   public Uses(@Qualifier("jpaRepo") Repo repo) { this.repo = repo; }
}
// Or mark one as @Primary: @Primary @Repository class JpaRepo implements Repo { }
```

What the container actually does on startup

```
@SpringBootApplication
public class Application {
   public static void main(String[] args) {
       // 1. Creates the ApplicationContext (the IoC container)
       // 2. Scans for @Component/@Service/@Repository/@Controller under this package
       // 3. Instantiates each bean definition (eagerly, by default)
       // 4. Injects dependencies (wiring)
       // 5. Calls @PostConstruct / InitializingBean
       // 6. Starts the embedded Tomcat on port 8080
       ApplicationContext ctx = SpringApplication.run(Application.class, args);

       // You can then pull beans out manually (useful in main()/tests):
       OrderService svc = ctx.getBean(OrderService.class);
   }
}

// Common startup failure to recognise:
// "No qualifying bean of type 'X' available"
//   -> the class isn't in a scanned package, or it lacks a stereotype annotation.
// "NoUniqueBeanDefinitionException"
//   -> two implementations; use @Qualifier or @Primary.
```

SAY IT LIKE THIS IN THE INTERVIEW

"IoC means the framework controls object creation and wiring instead of my code doing `new` everywhere. DI is how it's delivered — Spring injects dependencies, and I prefer constructor injection because it keeps fields final, makes dependencies explicit and makes unit testing trivial without Spring. A bean is any object managed by the container, singleton-scoped by default."

Flashcards

Q. What is IoC (Inversion of Control)?
A. The principle that object creation and lifecycle management is handled by a framework/container rather than your own code — 'don't call us, we'll call you'.

Q. What is Dependency Injection?
A. The implementation of IoC: dependencies are supplied to a class from the outside (constructor, setter or field) instead of being created inside it.

Q. Why prefer constructor injection?
A. Dependencies can be final/immutable, they're explicit and mandatory, circular dependencies fail fast, and you can unit test with plain mocks — no Spring context or reflection needed.

Q. What is a Spring bean?
A. Any object instantiated, assembled and managed by the Spring IoC container, defined via a stereotype annotation or an @Bean method.

Q. List the bean scopes.
A. singleton (default, one per container), prototype (new per injection), and for web apps: request, session, application, websocket.

Q. @Component vs @Service vs @Repository vs @Controller?
A. All are @Component specialisations. @Service marks business logic, @Repository marks data access and adds exception translation, @Controller/@RestController handle web requests.

Q. How do you resolve two beans of the same type?
A. Use @Qualifier("beanName") at the injection point, or mark one @Primary.

Q. @PostConstruct vs @PreDestroy?
A. @PostConstruct runs after dependency injection completes (init); @PreDestroy runs just before the bean is removed/container shuts down (cleanup).

Q. What is ApplicationContext?
A. The Spring IoC container interface: it holds bean definitions, manages their lifecycle, wires dependencies and provides features like events, i18n and resource loading.

Q. Is a singleton bean thread-safe?
A. Spring only guarantees one instance — it does NOT make it thread-safe. Keep singleton beans stateless (no mutable instance fields), or use synchronization/atomics.

Quiz

Q1. What is the relationship between IoC and DI?

[ ] A. They're the same thing
[x] B. IoC is the principle (container controls creation); DI is one way to implement it (dependencies are injected)
[ ] C. DI is the principle, IoC is the implementation
[ ] D. IoC is Spring-only, DI is Java EE-only

> **Why:** IoC is the broad design principle of inverting control; DI is the specific technique of supplying dependencies from outside.

Q2. Which injection type is recommended and why?

[ ] A. Field injection — least code
[x] B. Constructor injection — immutability, explicit dependencies, testability
[ ] C. Setter injection — most flexible
[ ] D. All are equally good

> **Why:** Constructor injection allows final fields, guarantees a fully-initialised object, and lets you instantiate the class in a unit test with no container.

Q3. What is the default bean scope in Spring?

[ ] A. prototype
[x] B. singleton
[ ] C. request
[ ] D. session

> **Why:** Singleton: one shared instance per Spring container. Important caveat: Spring does not make it thread-safe — keep singleton beans stateless.

Q4. You get `NoUniqueBeanDefinitionException`. What's the fix?

[ ] A. Delete one of the classes
[x] B. Use @Qualifier to name the bean you want, or mark one @Primary
[ ] C. Add @Scope("prototype")

[ ] D. Restart the server

Why: Two beans satisfy the same type. Disambiguate at the injection point with @Qualifier, or designate a default with @Primary.

Q5. Why is `@Repository` more than just a marker?

[ ] A. It's identical to @Component
[x] B. It also enables persistence exception translation (vendor exceptions → Spring DataAccessException)
[ ] C. It creates the database schema
[ ] D. It makes methods transactional automatically

Why: @Repository triggers a BeanPostProcessor that wraps the bean and translates technology-specific exceptions (e.g. SQLException) into Spring's consistent DataAccessException hierarchy. It does NOT add transactions.

Q6. What does @SpringBootApplication combine?

[x] A. @Configuration + @EnableAutoConfiguration + @ComponentScan
[ ] B. @Service + @Repository + @Controller
[ ] C. @Bean + @Autowired
[ ] D. @EntityScan + @EnableJpaRepositories

Why: Those three annotations: it marks the class as a config source, enables Boot's auto-configuration, and triggers component scanning of the current package and below.

2. Spring Boot: Auto-configuration, Starters & Config Must-know 40 min

"What's the difference between Spring and Spring Boot?" — the answer is auto-configuration, starters, embedded servers and opinionated defaults. That's it, and it's a free question if you've rehearsed it.

Key points

- **Spring Boot = Spring + conventions**: auto-configuration, starter dependencies, embedded server, externalised config, production-ready actuator endpoints.
- **Auto-configuration**: Boot inspects the classpath and your beans, then configures sensible defaults. It's all conditional (`@ConditionalOnClass`, `@ConditionalOnMissingBean`).
- **Your beans always win** — auto-configuration backs off when you define your own (`@ConditionalOnMissingBean`).
- **Starters** are curated dependency bundles: `spring-boot-starter-web`, `-data-jpa`, `-security`, `-test`, `-validation`.
- Config lives in `application.properties` or `application.yml`; bind it with `@Value` or (better) `@ConfigurationProperties`.
- **Profiles** (`@Profile`, `spring.profiles.active`) separate dev/test/prod configuration.

Reference

Concern	Plain Spring	Spring Boot
Setup	Lots of XML/Java config by hand	Auto-configured from the classpath
Dependencies	You pick versions for everything	Starters bundle compatible versions
Server	Deploy a WAR to external Tomcat	Embedded Tomcat/Jetty/Undertow, java -jar
Config	Manual property plumbing	application.yml + @ConfigurationProperties
Monitoring	DIY	Spring Boot Actuator (/actuator/health, /metrics)

Code

A minimal Boot app and its project layout

```
@SpringBootApplication       // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class Application {
   public static void main(String[] args) {
       SpringApplication.run(Application.class, args);
   }
}

// Typical package structure (component scan starts at the root package):
// src/main/java/com/parvez/app/
//   Application.java          <- @SpringBootApplication (MUST be at the root)
//   controller/OrderController.java
//   service/OrderService.java
//   repository/OrderRepository.java
//   model/Order.java
//   dto/OrderRequest.java
//   exception/GlobalExceptionHandler.java
//   config/AppConfig.java
// src/main/resources/
//   application.yml
//   static/  templates/  data.sql
// src/test/java/...        <- JUnit tests
// pom.xml (Maven) or build.gradle (Gradle)
```

application.yml — the file you'll be asked about

```
server:
 port: 8081

spring:
 application:
   name: order-service
 profiles:
   active: dev
 datasource:
   url: jdbc:postgresql://localhost:5432/orders
   username: ${DB_USER}          # from an environment variable
   password: ${DB_PASS}
   driver-class-name: org.postgresql.Driver
 jpa:
   hibernate:
     ddl-auto: update           # none | validate | update | create | create-drop
   show-sql: true
   properties:
     hibernate:
       format_sql: true
   open-in-view: false          # IMPORTANT: avoid lazy-loading in the view layer

 hikari:
   maximum-pool-size: 10

logging:
 level:
   root: INFO
   com.parvez: DEBUG

---
spring:
 config:
   activate:
     on-profile: dev
 datasource:
   url: jdbc:h2:mem:testdb      # in-memory DB for local dev
```

Reading configuration: @Value vs @ConfigurationProperties

```
@Component
public class SimpleConfig {
   @Value("${app.name}")                    // single value
   private String appName;
   @Value("${app.timeout:5000}")            // with a DEFAULT if absent
   private int timeout;
   @Value("${JAVA_HOME}")                   // system env vars work too
   private String javaHome;
}

// BETTER for groups of related properties: type-safe, validated, IDE-completable
@ConfigurationProperties(prefix = "app.mail")     // needs @EnableConfigurationProperties
@Validated
public record MailProperties(
       @NotBlank String host,
       int port,
       @NotBlank String from,
       Duration timeout
) { }

// app.yml:
// app:
//   mail:
//     host: smtp.example.com
//     port: 587
//     from: noreply@example.com
//     timeout: 5s

// Profiles: activate per environment
// java -jar app.jar --spring.profiles.active=prod
// SPRING_PROFILES_ACTIVE=prod docker run ...
// Or per-bean: @Profile("dev") on a @Bean method
```

Starters and how auto-configuration decides

```
<!-- pom.xml: one starter pulls in a coherent, version-aligned set of libs -->
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId> <!-- Tomcat + Jackson + Spring MVC -->
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-data-jpa</artifactId><!-- Hibernate + Spring Data -->
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
   <groupId>com.h2database</groupId>
   <artifactId>h2</artifactId>
   <scope>runtime</scope>
</dependency>
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-test</artifactId>    <!-- JUnit5 + Mockito + AssertJ -->
   <scope>test</scope>
</dependency>

// How auto-config works (simplified):
@Configuration
@ConditionalOnClass(DataSource.class)          // only if the driver is on the classpath
@ConditionalOnMissingBean(DataSource.class)    // and I haven't defined my own
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
   @Bean
   public DataSource dataSource(DataSourceProperties props) { /* build one */ }
}
// See what Boot decided: add --debug, or hit /actuator/conditions.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Spring Boot is Spring with opinionated defaults. Its auto-configuration inspects the classpath and conditionally configures beans — Tomcat because spring-boot-starter-web is present, a DataSource because a JDBC driver is — but it backs off the moment I define my own bean. Starters bundle compatible dependency versions, the app runs as a plain JAR with an embedded server, and config is externalised into application.yml bound with @ConfigurationProperties and separated by profiles."

Flashcards

Q. Spring vs Spring Boot?
A. Boot adds auto-configuration, starter dependencies, an embedded server, externalised config and Actuator — so you write far less boilerplate and run with `java -jar`.

Q. What is auto-configuration?
A. Boot inspects the classpath and existing beans, then applies @Conditional configuration (e.g. @ConditionalOnClass, @ConditionalOnMissingBean) to set up sensible defaults.

Q. How do you override an auto-configured bean?
A. Just define your own bean — auto-configuration classes use @ConditionalOnMissingBean and back off automatically.

Q. What is a Spring Boot starter?
A. A curated Maven/Gradle dependency bundle that brings in all the libraries needed for a feature with compatible versions (e.g. spring-boot-starter-web, -data-jpa, -security, -test).

Q. @Value vs @ConfigurationProperties?
A. @Value injects one property (good for single values). @ConfigurationProperties binds a whole typed, validated group — better for related settings and supports records.

Q. How do you separate dev and prod config?
A. Profiles: @Profile on beans, spring.profiles.active in config or as a runtime arg, plus profile-specific files (application-dev.yml) or --- documents in one YAML file.

Q. What is Spring Boot Actuator?
A. Production monitoring endpoints: /actuator/health, /info, /metrics, /loggers, /env, /beans, /conditions — usually secured and partially exposed.

Q. What does @SpringBootApplication combine?
A. @Configuration + @EnableAutoConfiguration + @ComponentScan.

Q. Why keep the main class at the root package?
A. @ComponentScan scans the package of the main class and everything below it — a class outside that tree simply won't be picked up (a very common 'bean not found' bug).

Q. What does spring.jpa.hibernate.ddl-auto=update do?
A. Hibernate updates the schema to match entities on startup. Fine for dev; in production prefer `validate` or `none` and manage migrations with Flyway/Liquibase.

Quiz

Q1. How does Spring Boot decide whether to configure a DataSource?

[ ] A. It always creates one
[x] B. Conditional annotations — e.g. @ConditionalOnClass(DataSource) and @ConditionalOnMissingBean — based on the classpath and your beans
[ ] C. You must always configure it manually
[ ] D. It reads a properties file only

> **Why:** Auto-configuration classes are loaded from META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports and guarded by @Conditional* annotations.

Q2. You define your own DataSource bean. What does Boot do with its auto-configured one?

[ ] A. Creates both, causing a conflict
[x] B. Backs off — @ConditionalOnMissingBean means yours wins
[ ] C. Throws an exception
[ ] D. Randomly picks one

> **Why:** That's the escape hatch that makes auto-configuration usable: conventions first, but your explicit definitions always take precedence.

Q3. Which dependency gives you an embedded Tomcat?

[ ] A. spring-boot-starter-data-jpa
[x] B. spring-boot-starter-web
[ ] C. spring-boot-starter-security
[ ] D. spring-core

> **Why:** spring-boot-starter-web pulls in spring-webmvc plus spring-boot-starter-tomcat, so the app can run standalone as a JAR.

Q4. How do you bind a group of properties to a typed object?

[ ] A. @Value on each field
[x] B. @ConfigurationProperties(prefix = "...")
[ ] C. @Autowired
[ ] D. @PropertySource only

> **Why:** @ConfigurationProperties binds and converts a whole prefix into a bean (works nicely with records), and supports validation with @Validated.

Q5. Which Actuator endpoint is typically used by orchestrators to check if the app is up?

[ ] A. /actuator/env
[x] B. /actuator/health

[ ] C. /actuator/beans

[ ] D. /actuator/shutdown

Why: /actuator/health is the standard liveness/readiness probe used by Kubernetes, load balancers and monitoring tools.

3. Spring MVC & REST Controllers Must-know 50 min

This is what you'll actually write on the job, so interviews go deep: mapping annotations, path/query/body parameters, status codes, validation, and global exception handling.

Key points

- **Request flow**: Client → DispatcherServlet (front controller) → HandlerMapping → Controller → Service → Repository → DB, then back through HttpMessageConverters (Jackson) → JSON response.
- `@RestController` = `@Controller` + `@ResponseBody` (return values are serialized to JSON, not resolved as views).
- `@PathVariable` = from the URL path; `@RequestParam` = query string; `@RequestBody` = JSON body; `@RequestHeader` = headers.
- **DTO pattern**: never expose entities directly — map to a request/response DTO so you control the API contract and avoid leaking internals or creating infinite JSON loops.
- **Validation**: `@Valid` + Jakarta Bean Validation annotations (`@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min`) and a `@ExceptionHandler` for `MethodArgumentNotValidException`.
- Return `ResponseEntity<T>` when you need control over status codes/headers; return the DTO directly for the simple 200 case.

Reference

Annotation	Maps	Example
@GetMapping	HTTP GET (read)	@GetMapping("/users/{id}")
@PostMapping	HTTP POST (create)	@PostMapping("/users") — returns 201
@PutMapping	HTTP PUT (full update)	@PutMapping("/users/{id}")
@PatchMapping	HTTP PATCH (partial update)	@PatchMapping("/users/{id}")
@DeleteMapping	HTTP DELETE	@DeleteMapping("/users/{id}") — 204
@PathVariable	URL segment	/users/{id} → Long id
@RequestParam	Query string	?page=0&size=20
@RequestBody	JSON request body	Deserialized to a DTO
@Valid	Triggers validation	On @RequestBody or @ModelAttribute

Code

A complete REST controller (the shape to memorise)

```
@RestController
@RequestMapping("/api/users")
@RequiredArgsConstructor          // Lombok: generates the constructor for DI
public class UserController {

   private final UserService service;      // constructor injection

   @GetMapping
   public List<UserResponse> getAll(
           @RequestParam(defaultValue = "0") int page,
           @RequestParam(defaultValue = "20") int size) {
       return service.findAll(page, size);
   }

   @GetMapping("/{id}")
   public ResponseEntity<UserResponse> getById(@PathVariable Long id) {
       return service.findById(id)
               .map(ResponseEntity::ok)
               .orElseGet(() -> ResponseEntity.notFound().build());
   }

   @PostMapping
   public ResponseEntity<UserResponse> create(@Valid @RequestBody UserRequest req) {
       UserResponse created = service.create(req);
       URI location = ServletUriComponentsBuilder
               .fromCurrentRequest().path("/{id}")
               .buildAndExpand(created.id()).toUri();
       return ResponseEntity.created(location).body(created);   // 201 + Location
   }

   @PutMapping("/{id}")
   public UserResponse update(@PathVariable Long id,
                              @Valid @RequestBody UserRequest req) {
       return service.update(id, req);
   }

   @DeleteMapping("/{id}")
   @ResponseStatus(HttpStatus.NO_CONTENT)     // 204
   public void delete(@PathVariable Long id) { service.delete(id); }
}
```

DTOs, validation and why not to expose entities

```
// Request DTO - what the client may send, with validation rules
public record UserRequest(
       @NotBlank(message = "name is required") String name,
       @NotBlank @Email(message = "must be a valid email") String email,
       @Min(value = 18, message = "must be 18+") int age
) { }

// Response DTO - what the client receives (no password, no internal ids)
public record UserResponse(Long id, String name, String email, int age) {
   public static UserResponse from(User u) {
       return new UserResponse(u.getId(), u.getName(), u.getEmail(), u.getAge());
   }
}

// Why DTOs instead of returning the entity?
//  1. Security: never leak password hashes or internal fields by accident
//  2. Stability: changing the DB schema doesn't break the API contract
//  3. Loops: bidirectional JPA relations cause infinite JSON recursion
//     (User -> orders -> user -> orders -> ...) - @JsonIgnore is a band-aid,
//     DTOs are the fix
//  4. Validation and shaping (per-endpoint views of the same data)

// Mapping: write it by hand (as above), or use MapStruct / ModelMapper
@Mapper(componentModel = "spring")     // MapStruct generates the implementation
public interface UserMapper {
   UserResponse toResponse(User user);
   User toEntity(UserRequest request);
}
```

Global exception handling with @ControllerAdvice

```
@RestControllerAdvice           // applies to every @RestController
public class GlobalExceptionHandler {

   @ExceptionHandler(ResourceNotFoundException.class)
   @ResponseStatus(HttpStatus.NOT_FOUND)                // 404
   public ApiError handleNotFound(ResourceNotFoundException ex, WebRequest req) {
       return new ApiError(Instant.now(), 404, "Not Found",
                           ex.getMessage(), req.getDescription(false));
   }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   @ResponseStatus(HttpStatus.BAD_REQUEST)              // 400
   public ApiError handleValidation(MethodArgumentNotValidException ex) {
       Map<String, String> errors = new HashMap<>();
       ex.getBindingResult().getFieldErrors()
         .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
       return new ApiError(Instant.now(), 400, "Validation Failed",
                           errors.toString(), "/api");
   }

   @ExceptionHandler(Exception.class)                   // catch-all safety net
   @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)    // 500
   public ApiError handleGeneric(Exception ex) {
       log.error("Unexpected error", ex);               // log, don't leak stack traces
       return new ApiError(Instant.now(), 500, "Internal Server Error",
                           "Something went wrong", "/api");
   }
}

public record ApiError(Instant timestamp, int status,
                       String error, String message, String path) { }
```

Status codes and the request pipeline

```
// The codes you must know:
//   200 OK            - successful GET/PUT
//   201 Created       - successful POST (+ Location header)
//   204 No Content    - successful DELETE
//   400 Bad Request   - invalid input / validation failure
//   401 Unauthorized  - not authenticated (no/invalid credentials)
//   403 Forbidden     - authenticated but not permitted
//   404 Not Found     - resource doesn't exist
//   409 Conflict      - state conflict (duplicate email etc.)
//   422 Unprocessable - syntactically valid but semantically wrong
//   500 Internal      - unexpected server error

// REQUEST PIPELINE (DispatcherServlet flow):
// 1. Client sends HTTP request
// 2. DispatcherServlet receives it (the single front controller)
// 3. HandlerMapping finds the controller method for the URL+verb
// 4. Argument resolvers bind @PathVariable / @RequestParam / @RequestBody
//    (Jackson converts JSON -> object via HttpMessageConverter)
// 5. Interceptors (@Aspect, filters, security) run around the handler
// 6. Controller -> Service (business logic, @Transactional) -> Repository
// 7. Return value converted to JSON by Jackson
// 8. Response sent with the appropriate status code

// Filter vs Interceptor vs AOP:
//   Filter    - servlet-level, runs before DispatcherServlet (Spring Security)
//   Interceptor - Spring MVC, pre/post handler (logging, auth checks)
//   AOP       - method-level, around advice (@Transactional, @Cacheable)
```

SAY IT LIKE THIS IN THE INTERVIEW

"A request hits the DispatcherServlet, which uses a HandlerMapping to find the controller method, binds parameters via HttpMessageConverters, runs the service layer and returns a value that Jackson serializes to JSON. I use constructor injection, return DTOs rather than entities so the API contract is stable and internal

fields never leak, validate incoming bodies with @Valid, and centralise error responses in a @RestControllerAdvice so the client always gets a consistent JSON error shape."

Flashcards

Q. What is DispatcherServlet?
A. Spring MVC's front controller: it receives every HTTP request and routes it to the right handler via HandlerMapping, then renders/converts the result.

Q. @Controller vs @RestController?
A. @RestController = @Controller + @ResponseBody. It serializes return values to JSON instead of resolving a view name.

Q. @PathVariable vs @RequestParam vs @RequestBody?
A. @PathVariable from the URL path (/users/{id}); @RequestParam from the query string (?page=0); @RequestBody from the JSON body, deserialized into an object.

Q. Why use DTOs instead of returning JPA entities?
A. They protect internal/sensitive fields, decouple the API from the DB schema, let you shape data per endpoint, and avoid infinite JSON recursion from bidirectional relationships.

Q. How do you validate a request body?
A. Annotate the DTO fields with Jakarta validation (@NotNull, @NotBlank, @Email, @Size) and mark the parameter @Valid; handle MethodArgumentNotValidException in a @ControllerAdvice.

Q. What status code for a successful POST?
A. 201 Created, ideally with a Location header pointing at the new resource (ResponseEntity.created(uri).body(dto)).

Q. What does @ControllerAdvice do?
A. It's a global, cross-cutting handler: @ExceptionHandler methods in it apply to all controllers, centralising error-to-HTTP-response mapping.

Q. Filter vs Interceptor?
A. Filters are servlet-level and run before DispatcherServlet (used by Spring Security). Interceptors are Spring MVC-level and run around the handler, with access to the handler/model.

Q. What makes a REST API 'RESTful'?
A. Statelessness, resource-based URLs (nouns, not verbs), correct HTTP verbs, proper status codes, HATEOAS (optional) and a uniform interface — commonly JSON over HTTP.

Q. What is the difference between PUT and PATCH?
A. PUT replaces the whole resource; PATCH applies a partial update.

Quiz

Q1. Which annotation extracts a value from `GET /users/42` where the method is mapped as `/users/{id}`?

[ ] A. @RequestParam
[x] B. @PathVariable
[ ] C. @RequestBody
[ ] D. @RequestHeader

Why: @PathVariable binds the {id} template variable from the URL path. @RequestParam would read ?id=42.

Q2. What is the correct response for creating a resource?

[ ] A. 200 OK
[x] B. 201 Created with a Location header
[ ] C. 204 No Content
[ ] D. 301 Moved Permanently

Why: 201 Created signals a new resource; including the Location header lets the client fetch it. 204 is for DELETE with no body.

Q3. Why should you avoid returning JPA entities directly from a controller?

[ ] A. It's slower

[x] B. It leaks internal fields, couples the API to the DB schema, and can cause infinite recursion with bidirectional relations
[ ] C. Jackson can't serialize entities
[ ] D. It breaks transactions

Why: Entities carry fields you don't want public and relationships that Jackson will happily recurse into. DTOs give you a deliberate, stable contract.

Q4. Where should you handle validation failures?

[ ] A. Inside the try/catch in every controller method
[x] B. In a @ControllerAdvice with @ExceptionHandler(MethodArgumentNotValidException.class)
[ ] C. In the service layer only
[ ] D. Let them propagate to the client as stack traces

Why: Centralising it gives consistent error JSON across the API and keeps controller methods clean — you return a field → message map of the violations.

Q5. How does JSON get converted to a Java object in @RequestBody?

[ ] A. Manually in the controller
[x] B. By an HttpMessageConverter — Jackson's MappingJackson2HttpMessageConverter
[ ] C. By the DispatcherServlet's view resolver
[ ] D. Java's built-in serializer

Why: Spring selects an HttpMessageConverter based on the Content-Type; for application/json it's Jackson, which is on the classpath via spring-boot-starter-web.

Q6. Which is true about REST statelessness?

[ ] A. The server stores session data per client
[x] B. Each request must carry all the information needed; the server keeps no client session state
[ ] C. REST requires cookies
[ ] D. It only applies to GET

Why: Statelessness is a REST constraint: no server-side session between requests. That's why JWTs (self-contained tokens) are a natural fit for REST auth.

4. JPA & Hibernate Must-know 60 min

The heaviest full-stack topic: entities, relationships, lazy vs eager, the N+1 problem, transactions and caching. If your interview mentions "your project used Hibernate", expect all of these.

Key points

- **JPA** is the specification (interfaces: `EntityManager`, annotations). **Hibernate** is the most common implementation. Spring Data JPA sits on top and generates repositories for you.
- An **entity** is a POJO mapped to a table: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`.
- **Entity lifecycle**: Transient (new) → Managed/Persistent (inside a session) → Detached (session closed) → Removed.
- **Fetch types**: `@ManyToOne`/`@OneToOne` default to **EAGER**; `@OneToMany`/`@ManyToMany` default to **LAZY**. Always be explicit.
- **LazyInitializationException** happens when you touch a lazy collection outside an open session/transaction — fix with a fetch join, `@EntityGraph`, a DTO projection, or keeping the transaction open.
- **N+1 problem**: 1 query for the parents + N queries for each child. Fix with `JOIN FETCH`, `@EntityGraph`, or `@BatchSize`.

Reference

Annotation	Purpose	Notes
@Entity	Marks a class as a DB-mapped entity	Needs a no-arg constructor
@Table(name=...)	Maps to a specific table	Optional — defaults to the class name
@Id	Primary key	Required on every entity
@GeneratedValue	Key generation strategy	IDENTITY, SEQUENCE, AUTO (TABLE is discouraged)
@Column	Column mapping	name, nullable, unique, length
@OneToMany	One parent → many children	mappedBy names the owning side; LAZY by default
@ManyToOne	Many children → one parent	EAGER by default — set fetch = LAZY explicitly
@JoinColumn	Declares the FK column	Placed on the owning side
@Transactional	Declares a transaction	Use on the service layer, not the controller

Code

Entities and relationships, done properly

```
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)   // auto-increment
   private Long id;

   @Column(nullable = false, length = 100)
   private String name;

   @Column(unique = true, nullable = false)
   private String email;

   @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
   private List<Order> orders = new ArrayList<>();      // LAZY by default

   public void addOrder(Order o) {                      // keep both sides in sync
       orders.add(o);
       o.setUser(this);
   }
   // no-arg constructor (required), getters, setters
}

@Entity
@Table(name = "orders")
public class Order {
   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @ManyToOne(fetch = FetchType.LAZY)          // EAGER by default - override it!
   @JoinColumn(name = "user_id")               // the FK column in `orders`
   private User user;

   @Enumerated(EnumType.STRING)                // store the name, not the ordinal!
   private OrderStatus status;

   @Column(name = "created_at", updatable = false)
   private LocalDateTime createdAt;

   @PrePersist
   protected void onCreate() { createdAt = LocalDateTime.now(); }
}

// Cascade types:
//   ALL          - propagate every operation
//   PERSIST      - save parent -> save children
//   MERGE        - update parent -> update children
//   REMOVE       - delete parent -> delete children
//   orphanRemoval = true  - deleting a child from the collection DELETES the row
```

Spring Data JPA repositories — queries without SQL

```
// No implementation needed - Spring generates it from the method name
public interface UserRepository extends JpaRepository<User, Long> {

   // Derived queries (the method name IS the query):
   Optional<User> findByEmail(String email);
   List<User> findByNameContainingIgnoreCase(String name);
   List<User> findByAgeGreaterThanEqualOrderByNameAsc(int age);
   boolean existsByEmail(String email);
   long countByStatus(OrderStatus status);
   void deleteByEmail(String email);

   // Custom JPQL - use @Param for named parameters
   @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
   Optional<User> findByIdWithOrders(@Param("id") Long id);

   @Query("SELECT new com.app.dto.UserSummary(u.name, COUNT(o)) " +
          "FROM User u LEFT JOIN u.orders o GROUP BY u.name")
   List<UserSummary> findUserSummaries();      // DTO projection - no entity overhead

   // Native SQL when you need DB-specific features
   @Query(value = "SELECT * FROM users WHERE age > :age",
          nativeQuery = true)
   List<User> findAdultsNative(@Param("age") int age);

   // Modifying queries need @Modifying + @Transactional
   @Modifying
   @Query("UPDATE User u SET u.name = :name WHERE u.id = :id")
   int updateName(@Param("id") Long id, @Param("name") String name);

   // Paging and sorting come free
   Page<User> findAll(Pageable pageable);
}

// Using it:
Pageable page = PageRequest.of(0, 20, Sort.by("name").ascending());
Page<User> result = repo.findAll(page);
result.getContent(); result.getTotalPages(); result.getTotalElements();
```

The N+1 problem — recognise it, then fix it

```
// PROBLEM: 1 query for users + 1 query per user for their orders = N+1
List<User> users = userRepository.findAll();
for (User u : users) {
   System.out.println(u.getOrders().size());    // lazy load -> a query EACH time
}
// SQL log:  SELECT * FROM users;  SELECT * FROM orders WHERE user_id=1;
//           SELECT * FROM orders WHERE user_id=2;  ... (100 users -> 101 queries!)

// FIX 1: JOIN FETCH in JPQL (one query, orders populated already)
@Query("SELECT DISTINCT u FROM User u LEFT JOIN FETCH u.orders")
List<User> findAllWithOrders();

// FIX 2: @EntityGraph (declarative, no JPQL string)
@EntityGraph(attributePaths = {"orders"})
List<User> findAll();

// FIX 3: @BatchSize - fetch children in batches (e.g. 25 at a time)
@OneToMany(mappedBy = "user")
@BatchSize(size = 25)
private List<Order> orders;

// FIX 4: a DTO projection when you only need a few fields (fastest)
@Query("SELECT new com.app.dto.UserRow(u.id, u.name, SIZE(u.orders)) FROM User u")
List<UserRow> findUserRows();

// Spot it in logs: enable spring.jpa.show-sql=true or
// logging.level.org.hibernate.SQL=DEBUG and look for repeated similar queries.
```

Transactions, dirty checking and caching

```
@Service
@RequiredArgsConstructor
public class OrderService {
   private final OrderRepository repo;

   // @Transactional on the SERVICE layer (not the controller, not the repo)
   @Transactional
   public Order placeOrder(OrderRequest req) {
       Order o = new Order(req);          // managed entity
       o.setTotal(computeTotal(req));
       repo.save(o);                      // not strictly needed: dirty checking
       return o;
   }                                      // commit + flush on successful exit

   @Transactional(readOnly = true)        // optimisation: no dirty checking, no flush
   public List<Order> findAll() { return repo.findAll(); }

   @Transactional(rollbackFor = Exception.class)   // checked exceptions too
   public void cancel(Long id) throws CancellationException { /* ... */ }

   @Transactional(propagation = Propagation.REQUIRES_NEW)   // independent tx
   public void audit(String action) { /* always committed, even if the caller rolls back */ }
}

// DIRTY CHECKING: inside a transaction, Hibernate snapshots loaded entities and
// automatically UPDATEs any whose state changed at flush time — no save() needed.

// PROPAGATION: REQUIRED (default, join existing or create), REQUIRES_NEW,
// SUPPORTS, MANDATORY, NEVER, NOT_SUPPORTED, NESTED

// ISOLATION: DEFAULT, READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE
@Transactional(isolation = Isolation.READ_COMMITTED)

// CACHING:
//   1st level - the persistence context (per transaction/session), always on
//   2nd level - shared across sessions; enable with a provider (Ehcache/Caffeine)
//               @Cacheable + @org.hibernate.annotations.Cache(usage = READ_WRITE)
//   Query cache - caches result sets; use sparingly

// IMPORTANT: @Transactional doesn't work on private/self-invoked methods —
// it needs the Spring proxy, so calls must come from outside the bean.
```

LazyInitializationException — what causes it and the fixes

```
// CAUSE: the transaction/session closed before you touched the lazy collection
@Transactional
public User loadUser(Long id) {
   return repo.findById(id).orElseThrow();   // tx ends here, session closes
}
User u = service.loadUser(1L);
u.getOrders().size();     // LazyInitializationException: no Session

// FIX 1 (best): fetch what you need inside the transaction (JOIN FETCH / EntityGraph)
// FIX 2: use a DTO projection - map to the DTO before the tx ends
// FIX 3: @Transactional on the caller so the session spans the whole use case
// FIX 4 (AVOID): spring.jpa.open-in-view=true keeps the session open for the
//        whole web request - it hides the bug and holds DB connections longer.
//        Spring Boot warns about this on startup; set it to false.
```

SAY IT LIKE THIS IN THE INTERVIEW

"JPA is the specification and Hibernate is the implementation; Spring Data JPA generates repositories from interface method names. I map entities with @Entity/@Id/@GeneratedValue, set @ManyToOne to LAZY rather than accepting the EAGER default, and watch for the N+1 problem — which I fix with JOIN FETCH or @EntityGraph. Transactions belong on the service layer: Hibernate's dirty checking means I don't need to call save() on managed entities, and the commit happens when the transactional method exits."

Flashcards

Q. JPA vs Hibernate vs Spring Data JPA?
A. JPA is the specification (interfaces and annotations). Hibernate is the implementation. Spring Data JPA is a layer on top that generates repository implementations from interfaces.

Q. What are the entity lifecycle states?
A. Transient/New (not persisted), Managed/Persistent (attached to a session), Detached (session closed), Removed (scheduled for deletion).

Q. Lazy vs Eager fetching — what are the defaults?
A. @OneToMany and @ManyToMany default to LAZY; @ManyToOne and @OneToOne default to EAGER. Always declare fetch explicitly, and prefer LAZY.

Q. What is the N+1 problem and how do you fix it?
A. One query for the parents plus one per child collection. Fix with JOIN FETCH, @EntityGraph, @BatchSize, or a DTO projection.

Q. What is LazyInitializationException?
A. Accessing a lazy association after the session/transaction closed. Fix by fetching inside the transaction, using a DTO projection, or widening @Transactional — not by enabling open-in-view.

Q. Where should @Transactional go?
A. On the service layer (the unit of work), not on controllers or repositories. Use readOnly = true for queries. It won't apply to private or self-invoked methods because it's proxy-based.

Q. What is dirty checking?
A. Hibernate snapshots managed entities and automatically issues UPDATEs at flush time for any that changed — you don't need to call save() on a managed entity.

Q. GenerationType.IDENTITY vs SEQUENCE?
A. IDENTITY uses DB auto-increment (blocks JDBC batching). SEQUENCE uses a DB sequence and allows batching — generally preferred for performance with Hibernate.

Q. First-level vs second-level cache?
A. First-level is the persistence context (per transaction, always enabled). Second-level is shared across sessions and needs a provider like Ehcache/Caffeine plus @Cacheable.

Q. Why @Enumerated(EnumType.STRING)?
A. It stores the enum NAME instead of the ordinal. Ordinals break silently when you reorder or insert enum constants.

Q. What does mappedBy mean?
A. It marks the inverse (non-owning) side of a bidirectional relationship and names the field on the owning side that holds the foreign key.

Q. orphanRemoval vs cascade = REMOVE?
A. cascade REMOVE deletes children when the parent is deleted. orphanRemoval also deletes a child when it's removed from the parent's collection.

Quiz

Q1. What is the default fetch type of @ManyToOne?

[ ] A. LAZY
[x] B. EAGER
[ ] C. Depends on the DB
[ ] D. None

Why: @ManyToOne and @OneToOne default to EAGER (a common performance surprise). @OneToMany and @ManyToMany default to LAZY. Always specify it explicitly.

Q2. You run `findAll()` on 100 users and then loop over each user's orders. How many queries?

[ ] A. 1
[ ] B. 2
[x] C. 101 (the N+1 problem)
[ ] D. 100

Why: One query for the users plus one lazy load per user. Fix with JOIN FETCH, @EntityGraph, @BatchSize or a DTO projection.

Q3. Where is the best place for @Transactional?

[ ] A. On the controller
[x] B. On the service layer (the unit of work)
[ ] C. On the entity
[ ] D. On the DTO

> **Why:** The service method usually represents one business transaction. Controllers may fan out to several services, so the transaction boundary belongs in the service.

Q4. Why might @Transactional appear not to work?

[ ] A. It's deprecated
[x] B. The method is private, final, or called from within the same bean (self-invocation) — bypassing the proxy
[ ] C. The database doesn't support it
[ ] D. You imported the wrong package

> **Why:** Spring's transactions are AOP proxy-based, so only external calls through the proxy are intercepted. Also check you're using jakarta.transaction.Transactional or org.springframework.transaction.annotation.Transactional.

Q5. What is dirty checking?

[ ] A. Validating entities before insert
[x] B. Hibernate automatically detecting changed managed entities and issuing UPDATEs at flush time
[ ] C. Checking for SQL injection
[ ] D. Validating the schema

> **Why:** Because managed entities are tracked in the persistence context, mutating them inside a transaction is enough — an explicit save() is only needed for new (transient) entities.

Q6. How do you fetch a parent with its children in ONE query?

[ ] A. Set fetch = EAGER
[x] B. JOIN FETCH in JPQL or @EntityGraph
[ ] C. Call size() first
[ ] D. Use native SQL only

> **Why:** EAGER just hides the problem (it still issues queries, and always). JOIN FETCH or @EntityGraph retrieve parents and children together in a single SQL statement.

Q7. Which GenerationType allows JDBC batch inserts?

[ ] A. IDENTITY
[x] B. SEQUENCE
[ ] C. TABLE
[ ] D. AUTO

> **Why:** IDENTITY requires an immediate INSERT to obtain the key, which disables batching. SEQUENCE (or a pooled optimizer) lets Hibernate batch inserts efficiently.

5. SQL Essentials for Interviews Must-know 45 min

You will be asked to write SQL — usually a join, a group-by with having, and a 'second highest salary'-style query. Practise these until they're muscle memory.

Key points

- **Execution order**: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. (Knowing this explains why column aliases work in ORDER BY but not WHERE.)
- **JOINs**: INNER (matching rows), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS. LEFT JOIN + `WHERE right.id IS NULL` finds unmatched rows.
- `WHERE` filters rows **before** grouping; `HAVING` filters groups **after**.
- **Indexes** speed up reads but slow writes. A composite index respects the leftmost-prefix rule (index on (a,b) helps `WHERE a=?` but not `WHERE b=?`).
- **Normalization**: 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency). Denormalize deliberately for read performance.
- `NVL/COALESCE`, `CASE`, `DISTINCT`, `UNION` vs `UNION ALL` (UNION dedupes and is slower).

Code

The queries you'll actually be asked to write

```
-- Second highest salary (the classic)
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Or with a window function / LIMIT
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Top earner per department
SELECT dept, name, salary FROM (
   SELECT e.*, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn
   FROM employees e
) t WHERE rn = 1;

-- Departments with more than 5 employees, by average salary
SELECT d.name, COUNT(e.id) AS headcount, AVG(e.salary) AS avg_salary
FROM departments d
JOIN employees e ON e.dept_id = d.id
WHERE e.active = true
GROUP BY d.name
HAVING COUNT(e.id) > 5
ORDER BY avg_salary DESC;

-- Employees with no department (LEFT JOIN ... IS NULL)
SELECT e.name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;

-- Duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;

-- Employees earning more than their manager
SELECT e.name FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- Pagination
SELECT * FROM orders ORDER BY created_at DESC LIMIT 20 OFFSET 40;
```

JOINs explained with a picture in words

```
-- Sample data
-- employees: (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cara', NULL)
-- departments: (10, 'Eng'), (20, 'Sales'), (30, 'HR')

-- INNER JOIN: only rows that match in BOTH -> Ana/Eng, Ben/Sales  (2 rows)
SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.dept_id = d.id;

-- LEFT JOIN: ALL left rows, NULLs where there's no match -> also Cara/NULL (3 rows)
SELECT e.name, d.name FROM employees e LEFT JOIN departments d ON e.dept_id = d.id;

-- RIGHT JOIN: all right rows -> also NULL/HR  (3 rows)
-- FULL OUTER JOIN: everything from both sides (not supported in MySQL!)

-- CROSS JOIN: cartesian product (3 x 3 = 9 rows) - usually a mistake
-- SELF JOIN: join a table to itself (employee -> manager)

-- JOIN vs WHERE filtering:
--   ON   -> condition applied while joining (affects which rows join)
--   WHERE-> applied after the join (a WHERE on a LEFT-joined column
--           effectively turns it into an INNER JOIN!)
```

Indexes, EXPLAIN and the things that kill performance

```
-- Create indexes
CREATE INDEX idx_emp_dept ON employees(dept_id);
CREATE UNIQUE INDEX idx_user_email ON users(email);
CREATE INDEX idx_order_date ON orders(created_at DESC);

-- Composite index: column order matters (leftmost prefix rule)
CREATE INDEX idx_name_age ON employees(last_name, age);
-- Helps:  WHERE last_name = 'X'                      -> yes
--         WHERE last_name = 'X' AND age = 30         -> yes
--         WHERE age = 30                             -> NO (skips the prefix)

-- See how the DB will run your query
EXPLAIN SELECT * FROM employees WHERE dept_id = 10;

-- Common index killers:
--   WHERE YEAR(created_at) = 2024        -> a function on the column blocks the index
--     FIX: WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'
--   WHERE name LIKE '%son'               -> a leading wildcard can't use the index
--   Implicit type conversion, OR across unindexed columns, SELECT * on huge tables

-- Best practices:
--   - Index foreign keys (they're your join and filter columns)
--   - Index the columns in WHERE/JOIN/ORDER BY
--   - Don't over-index: every index slows INSERT/UPDATE/DELETE
--   - Prefer covering indexes for hot queries
```

Useful patterns: CASE, COALESCE, EXISTS, window functions

```
-- CASE (SQL's if/else)
SELECT name,
      CASE WHEN salary > 100000 THEN 'High'
           WHEN salary >  50000 THEN 'Mid'
           ELSE 'Low' END AS band
FROM employees;

-- COALESCE: first non-null value (default handling)
SELECT name, COALESCE(bonus, 0) AS bonus FROM employees;

-- EXISTS is usually faster than IN for large subqueries
SELECT name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.id);
-- EXISTS stops at the first match; IN materialises the whole result set.

-- Window functions (PostgreSQL/MySQL 8+/SQL Server)
SELECT name, dept, salary,
      RANK()       OVER (PARTITION BY dept ORDER BY salary DESC) AS rank_in_dept,
      SUM(salary)  OVER (PARTITION BY dept) AS dept_total,
      LAG(salary)  OVER (ORDER BY salary) AS previous_salary
FROM employees;

-- UPSERT (insert or update on conflict)
INSERT INTO users(email, name) VALUES ('a@b.c', 'Ana')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name;

-- Soft delete pattern + partial index
ALTER TABLE orders ADD COLUMN deleted_at TIMESTAMP NULL;
CREATE INDEX idx_orders_live ON orders(id) WHERE deleted_at IS NULL;
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'm comfortable writing joins, group-by with having, subqueries and window functions. I know WHERE filters before grouping and HAVING after, that indexes speed reads but cost writes, and that a LEFT JOIN plus IS NULL finds unmatched rows. For slow queries I read the EXPLAIN plan first and look for full scans, functions on indexed columns and leading wildcards."

Flashcards

Q. WHERE vs HAVING?
A. WHERE filters individual rows before grouping (can't use aggregates). HAVING filters groups after GROUP BY (can use COUNT/SUM/AVG).

Q. What's the SQL query execution order?
A. FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. (Which is why a SELECT alias works in ORDER BY but not WHERE.)

Q. INNER vs LEFT JOIN?
A. INNER returns only matching rows from both tables. LEFT returns every row from the left table, with NULLs where there's no match.

Q. How do you find rows with no match in another table?
A. LEFT JOIN and then `WHERE right_table.id IS NULL`.

Q. UNION vs UNION ALL?
A. UNION removes duplicates (extra sort/dedupe cost); UNION ALL keeps everything and is faster.

Q. What is the Nth-highest-salary pattern?
A. SELECT DISTINCT salary ORDER BY salary DESC LIMIT 1 OFFSET N-1, or a subquery with MAX(salary) < (SELECT MAX(...)), or DENSE_RANK() over the ordered salaries.

Q. How do indexes work and what's the trade-off?
A. An index (usually a B-tree) lets the DB find rows without a full scan: much faster reads, but every INSERT/UPDATE/DELETE must also maintain it, and it costs storage.

Q. What is the leftmost-prefix rule?
A. A composite index on (a, b) can serve queries filtering on a, or a+b, but not b alone — the leading column must be used.

Q. Why is `WHERE YEAR(date) = 2024` slow?
A. Applying a function to an indexed column prevents index usage; the DB must evaluate it per row. Use a range comparison instead.

Q. EXISTS vs IN?
A. EXISTS stops scanning at the first match and is generally better for large subqueries; IN materialises the full result set first.

Q. What are the normal forms?
A. 1NF: atomic column values, no repeating groups. 2NF: 1NF + no partial dependency on part of a composite key. 3NF: 2NF + no transitive dependency on non-key columns.

Q. What is a deadlock in a database?
A. Two transactions each hold a lock the other needs. The DB detects it and rolls one back (that's the deadlock error you see in logs) — prevent it with consistent lock ordering and short transactions.

Quiz

Q1. Which clause filters after GROUP BY?

[ ] A. WHERE
[x] B. HAVING
[ ] C. ORDER BY
[ ] D. LIMIT

> **Why:** HAVING filters groups and can reference aggregates (HAVING COUNT(*) > 5). WHERE filters rows before grouping.

Q2. You want all customers even if they have no orders. Which join?

[ ] A. INNER JOIN
[x] B. LEFT JOIN
[ ] C. CROSS JOIN
[ ] D. RIGHT JOIN only

> **Why:** LEFT JOIN keeps every row from the left (customers) and fills NULLs for missing matches (no orders).

Q3. What is the second highest salary query pattern?

[ ] A. SELECT salary FROM emp ORDER BY salary DESC LIMIT 2
[x] B. SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp)
[ ] C. SELECT salary FROM emp WHERE rownum = 2
[ ] D. SELECT MIN(salary) FROM emp

> **Why:** The subquery excludes the maximum, so MAX of the remainder is the second highest. LIMIT 1 OFFSET 1 with DISTINCT is the other common form.

Q4. Does adding an index always make queries faster?

[ ] A. Yes, always
[x] B. No — it speeds reads but slows writes and consumes storage, and a badly-chosen index may not be used at all
[ ] C. Only for primary keys
[ ] D. Yes, if you index every column

> **Why:** Every write must update each index. Index the columns you actually filter, join and sort on — and verify with EXPLAIN.

Q5. Which of these prevents an index from being used?

[ ] A. WHERE status = 'A'
[x] B. WHERE UPPER(name) = 'ANA' (a function on the indexed column)
[ ] C. WHERE id = 42
[ ] D. WHERE created_at >= '2024-01-01'

Why: Wrapping the column in a function makes the predicate non-sargable. Store a normalized column, use a functional index, or rewrite as a range.

Q6. What does `COUNT(*)` vs `COUNT(col)` return?

[ ] A. Identical results always
[x] B. COUNT(*) counts all rows; COUNT(col) counts only non-NULL values of that column
[ ] C. COUNT(*) is slower and deprecated
[ ] D. COUNT(col) counts distinct values

Why: A classic trick question — COUNT(column) skips NULLs. Use COUNT(DISTINCT col) if you want unique values.

6. Maven, Build & Project Structure Core 25 min

Low difficulty, high frequency: "walk me through your pom.xml" and "what's Maven's build lifecycle?" — easy marks most candidates fumble.

Key points

- Maven uses a **POM** (`pom.xml`) with GAV coordinates: groupId, artifactId, version.
- **Standard layout**: `src/main/java`, `src/main/resources`, `src/test/java`, `target/` for build output.
- **Lifecycle phases**: `validate → compile → test → package → verify → install → deploy` (running a later phase runs all earlier ones).
- **Scope**: `compile` (default, everywhere), `provided` (JDK/container supplies it), `runtime`, `test`, `import`.
- **Dependency management**: Spring Boot's parent POM (or `spring-boot-dependencies` BOM) pins compatible versions so you don't specify them.
- Gradle is the alternative — faster, Groovy/Kotlin DSL, used by Android; same concepts, different syntax.

Code

A Spring Boot pom.xml, annotated

```
<project>
 <modelVersion>4.0.0</modelVersion>

 <parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>3.2.5</version>          <!-- manages ALL dependency versions -->
   <relativePath/>
 </parent>

 <groupId>com.parvez</groupId>        <!-- your org, reverse domain -->
 <artifactId>order-service</artifactId><!-- the project name -->
 <version>0.0.1-SNAPSHOT</version>    <!-- SNAPSHOT = in development -->
 <packaging>jar</packaging>

 <properties>
   <java.version>17</java.version>
 </properties>

 <dependencies>
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-web</artifactId>
     <!-- no <version> - inherited from the parent -->
   </dependency>
   <dependency>
     <groupId>org.projectlombok</groupId>
     <artifactId>lombok</artifactId>
     <scope>provided</scope>          <!-- compile-time only, not in the JAR -->
   </dependency>
   <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-test</artifactId>
     <scope>test</scope>              <!-- test classpath only -->
   </dependency>
 </dependencies>

 <build>
   <plugins>
     <plugin>
       <groupId>org.springframework.boot</groupId>
       <artifactId>spring-boot-maven-plugin</artifactId>
       <!-- builds an executable "fat jar" with the embedded server -->
     </plugin>
   </plugins>
 </build>
</project>
```

Commands you'll actually run

```
mvn clean                # delete target/
mvn compile              # compile main sources
mvn test                 # run unit tests
mvn package              # build the JAR (runs tests too)
mvn package -DskipTests  # build without running tests
mvn spring-boot:run      # run the app directly (dev)
mvn clean install        # build + install into ~/.m2 for other projects
mvn dependency:tree      # see where a transitive dependency comes from
mvn versions:display-dependency-updates

java -jar target/order-service-0.0.1-SNAPSHOT.jar
java -jar app.jar --spring.profiles.active=prod

# Gradle equivalents
./gradlew bootRun
./gradlew build
./gradlew test
```

Lombok — the annotations to name-drop (and understand)

```
@Data                    // getters + setters + equals + hashCode + toString
@Getter @Setter          // just the accessors
@NoArgsConstructor       // JPA needs this
@AllArgsConstructor
@RequiredArgsConstructor  // one ctor for final/@NonNull fields -> constructor DI!
@Builder                 // User.builder().name("x").build()
@Slf4j                   // adds a static `log` field
@ToString(exclude = "password")
@EqualsAndHashCode(of = "id")   // use the business key, not every field

@Slf4j
@Service
@RequiredArgsConstructor      // <-- the standard Spring DI pattern
public class OrderService {
   private final OrderRepository repo;   // injected via the generated ctor

   public void place(Order o) {
       log.info("placing order {}", o.getId());
   }
}

// Caveat: Lombok is compile-time magic - it needs the IDE plugin and an
// annotation-processor entry. Records (Java 16) cover many of its use cases
// for immutable data, so prefer records for DTOs and Lombok for entities.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Maven uses a POM with GAV coordinates and a fixed lifecycle: validate, compile, test, package, verify, install, deploy. The Spring Boot parent POM manages dependency versions so I don't have to align them myself. I scope dependencies properly — test for JUnit/Mockito, provided for Lombok — and the spring-boot-maven-plugin produces an executable fat JAR I can run with java -jar."

Flashcards

Q. What are Maven's GAV coordinates?
A. groupId (organisation, reverse domain), artifactId (project name), version (SNAPSHOT = in-development).

Q. List the Maven lifecycle phases.
A. validate, compile, test, package, verify, install, deploy. Running a later phase executes all the earlier ones.

Q. What does `mvn package` do vs `mvn install`?
A. package builds the artifact into target/. install does that AND installs it into the local ~/.m2 repository so other projects can depend on it.

Q. What is a Maven dependency scope?
A. compile (default, everywhere), provided (JDK/container provides it at runtime), runtime, test (test classpath only), system, import (BOM imports).

Q. What is a BOM / dependency management?
A. A 'bill of materials' POM that pins compatible versions of many libraries, so you declare dependencies without versions and never hit conflicts.

Q. What is a fat/uber JAR?
A. An executable JAR containing your classes plus all dependencies and an embedded server — built by spring-boot-maven-plugin, runnable with java -jar.

Q. Maven vs Gradle?
A. Maven uses declarative XML with a fixed lifecycle. Gradle uses a Groovy/Kotlin DSL, is generally faster with incremental builds and caching, and is the default for Android. Same core concepts.

Q. What does @RequiredArgsConstructor do and why is it popular in Spring?
A. Lombok generates a constructor for all final/@NonNull fields — which is exactly what constructor-based dependency injection needs, with zero boilerplate.

Quiz

Q1. Which Maven phase runs the tests and creates the JAR?

[ ] A. compile

[ ] B. test
[x] C. package
[ ] D. install

> **Why:** `package` runs the full lifecycle up to and including test, then builds the artifact in target/. `install` also copies it to your local repository.

Q2. Why use `spring-boot-starter-parent`?

[ ] A. It's required by Spring
[x] B. It provides dependency and plugin management with compatible versions, plus sensible defaults
[ ] C. It enables auto-configuration
[ ] D. It compiles Java faster

> **Why:** It's a parent POM that declares versions for the whole Boot ecosystem, so you omit <version> in your dependencies and avoid version conflicts.

Q3. Which scope should JUnit and Mockito use?

[ ] A. compile
[ ] B. provided
[x] C. test
[ ] D. runtime

> **Why:** Test-scoped dependencies are only on the test classpath and are never shipped in the final JAR.

Q4. What is a transitive dependency?

[x] A. A dependency of a dependency, pulled in automatically
[ ] B. A dependency in the test scope
[ ] C. An optional dependency
[ ] D. A circular dependency

> **Why:** Maven resolves the whole dependency tree automatically. Use `mvn dependency:tree` to see where something came from, or <exclusions> to drop an unwanted one.

7. Spring Security & JWT Edge 35 min

For a fresher they usually want the high-level flow, not deep configuration: the filter chain, authentication vs authorization, and how a JWT works.

Key points

- **Authentication** = who are you? **Authorization** = what may you do? (401 vs 403.)
- Spring Security is a chain of servlet **filters**; each request passes through them before reaching the controller.
- **JWT** (JSON Web Token) = header.payload.signature. It's **stateless** — the server stores nothing; the token carries the claims and is verified by signature.
- Never store secrets in a JWT — it's **Base64-encoded, not encrypted**. Anyone can decode the payload.
- **BCrypt** for password hashing — slow by design and salted. Never MD5/SHA or plain text, and never encrypt passwords (hash them).
- **CORS** (cross-origin) and **CSRF** (cross-site request forgery) are separate concerns: CSRF protection can be disabled for stateless JWT APIs, CORS must be configured explicitly.

Code

The JWT authentication flow (draw this in the interview)

```
1. POST /api/auth/login  { "username": "p", "password": "secret" }
       |
       v
2. Spring Security's filter chain intercepts it
       |
       v
3. AuthenticationManager -> DaoAuthenticationProvider -> UserDetailsService
  loads the user, BCrypt compares the hashed password
       |
       v
4. On success: generate a JWT signed with a secret (HS256)
  payload claims: subject (username), roles, issuedAt, expiration
       |
       v
5. Response: { "token": "eyJhbGciOi..." }
       |
       v
6. Client stores the token (memory / httpOnly cookie) and sends it on
  every request:  Authorization: Bearer eyJhbGciOi...
       |
       v
7. OncePerRequestFilter validates the signature + expiry on each request,
  loads the user, and populates the SecurityContext
       |
       v
8. @PreAuthorize("hasRole('ADMIN')") / the SecurityFilterChain's
  authorizeHttpRequests rules decide whether access is allowed
       |
       v
9. Controller runs (or 401/403 is returned)

A JWT is three Base64 parts joined by dots:  HEADER.PAYLOAD.SIGNATURE
 header   - { "alg": "HS256", "typ": "JWT" }
 payload  - { "sub": "parvez", "roles": ["ADMIN"], "exp": 1735689600 }
 signature- HMACSHA256(base64(header) + "." + base64(payload), secret)
```

A modern Spring Security 6 configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity          // enables @PreAuthorize
@RequiredArgsConstructor
public class SecurityConfig {

   private final JwtAuthFilter jwtAuthFilter;
   private final UserDetailsService userDetailsService;

   @Bean
   public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
       return http
           .csrf(csrf -> csrf.disable())                // stateless JWT API
           .cors(cors -> cors.configurationSource(corsConfigurationSource()))
           .authorizeHttpRequests(auth -> auth
               .requestMatchers("/api/auth/**", "/actuator/health").permitAll()
               .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
               .requestMatchers("/api/admin/**").hasRole("ADMIN")
               .anyRequest().authenticated()
           )
           .sessionManagement(s -> s
               .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
           .authenticationProvider(authenticationProvider())
           .addFilterBefore(jwtAuthFilter,
                            UsernamePasswordAuthenticationFilter.class)
           .build();
   }

   @Bean
   public PasswordEncoder passwordEncoder() {
       return new BCryptPasswordEncoder(12);       // cost factor 12
   }

   @Bean
   public AuthenticationProvider authenticationProvider() {
       DaoAuthenticationProvider p = new DaoAuthenticationProvider();
       p.setUserDetailsService(userDetailsService);
       p.setPasswordEncoder(passwordEncoder());
       return p;
   }
}

// Method-level security:
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public void delete(@PathVariable Long id) { }

@PreAuthorize("#userId == authentication.principal.id")   // own data only
@GetMapping("/{userId}/profile")
public Profile get(@PathVariable Long userId) { }
```

Password handling and the mistakes to avoid

```
@Service
class UserService {
   private final PasswordEncoder encoder;    // BCryptPasswordEncoder

   public void register(RegisterRequest req) {
       if (repo.existsByEmail(req.email()))
           throw new DuplicateEmailException();
       User u = new User();
       u.setEmail(req.email());
       u.setPassword(encoder.encode(req.password()));   // HASH, never encrypt
       u.setRole(Role.USER);
       repo.save(u);
   }
}

// BCrypt specifics:
//  - Automatically generates and stores a random salt with the hash
//  - Deliberately slow (a cost factor of 10-12) to defeat brute force
//  - encoder.matches(rawPassword, storedHash) is how you verify
//  - Each hash of the same password differs (different salt) - that's expected

// NEVER:
//   store plain text passwords
//   use MD5 / SHA-1 / SHA-256 alone (too fast -> GPU brute force)
//   implement your own crypto
//   log passwords or put them in a JWT
```

SAY IT LIKE THIS IN THE INTERVIEW

"Spring Security is a filter chain in front of my controllers. For a stateless API I use JWTs: the user logs in, I verify the password with BCrypt, issue a signed token carrying subject, roles and expiry, and a filter validates that token on each request and populates the SecurityContext. I disable CSRF for stateless APIs but configure CORS explicitly, and I use @PreAuthorize for method-level authorization. Passwords are BCrypt-hashed with a per-user salt — never encrypted or stored in plain text."

Flashcards

Q. Authentication vs authorization?
A. Authentication verifies identity (who are you? → 401 if it fails). Authorization verifies permissions (what can you do? → 403 if it fails).

Q. What is a JWT and what's in it?
A. A stateless, signed token: header (algorithm), payload (claims like subject, roles, expiry) and a signature. Base64-encoded — readable by anyone, so never put secrets in it.

Q. Why is JWT called stateless?
A. The server stores no session; all the information is in the token itself, verified cryptographically on each request. That's what makes it scale horizontally.

Q. How do you invalidate a JWT before it expires?
A. You can't easily — that's the main drawback. Use short expiry plus refresh tokens, or maintain a server-side denylist/token version in the DB.

Q. Why BCrypt instead of SHA-256 for passwords?
A. BCrypt is deliberately slow and includes a per-password salt, defeating rainbow tables and GPU brute force. General-purpose hashes are fast, which is exactly wrong for passwords.

Q. CSRF vs CORS?
A. CSRF tricks a browser into sending an authenticated request (mitigated by tokens/same-site cookies). CORS is a browser policy controlling which origins may read responses (configured on the server). They're unrelated problems.

Q. 401 vs 403?
A. 401 Unauthorized = not authenticated (no or bad credentials). 403 Forbidden = authenticated but lacking permission.

Q. What is the Spring Security filter chain?
A. A chain of servlet filters each request passes through before the DispatcherServlet — handling auth, JWT validation, exception translation and authorization.

Quiz

Q1. A user is logged in but tries to access an admin-only page. What status code?

[ ] A. 401 Unauthorized
[x] B. 403 Forbidden
[ ] C. 404 Not Found
[ ] D. 500

> **Why:** They're authenticated but not authorized for that resource → 403. 401 means we don't know who they are.

Q2. Is the JWT payload encrypted?

[ ] A. Yes, fully encrypted
[x] B. No — it's Base64-encoded and readable by anyone; only the signature is cryptographic
[ ] C. Only if you use HS256
[ ] D. It's hashed

> **Why:** JWTs are signed, not encrypted (unless you use JWE). Never put passwords or secrets in the payload.

Q3. How should passwords be stored?

[ ] A. Encrypted with AES so they can be recovered
[x] B. Hashed with BCrypt/Argon2 including a per-user salt
[ ] C. Hashed with MD5
[ ] D. In plain text, but the DB is secure

> **Why:** Passwords are hashed (one-way) with a slow, salted algorithm. Encryption is reversible, which you never want for passwords.

Q4. Which Spring Security setting makes the API stateless?

[ ] A. csrf().disable()
[x] B. SessionCreationPolicy.STATELESS
[ ] C. permitAll()
[ ] D. @PreAuthorize

> **Why:** STATELESS tells Spring Security not to create or use an HTTP session — appropriate when every request carries a JWT. CSRF disable is a related but separate choice.

8. Testing: JUnit 5 & Mockito Core 35 min

Often skipped by freshers, which is exactly why mentioning it stands out. Know the annotations, a basic Mockito test, and `@WebMvcTest` vs `@DataJpaTest` vs `@SpringBootTest`.

Key points

- **JUnit 5** (Jupiter) annotations: `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`, `@ParameterizedTest`, `@DisplayName`, `@Nested`, `@Disabled`.
- Assertions: `assertEquals`, `assertTrue`, `assertThrows`, `assertAll`, and AssertJ's fluent `assertThat(x).isEqualTo(y)`.
- **Mockito**: `@ExtendWith(MockitoExtension.class)`, `@Mock` for dependencies, `@InjectMocks` for the class under test, `when(...).thenReturn(...)`, `verify(...)`.
- **Test slices**: `@WebMvcTest` (controllers only, with `@MockBean` services), `@DataJpaTest` (repositories with an in-memory DB), `@SpringBootTest` (full context — slowest).
- **Test pyramid**: many unit tests, fewer integration tests, fewest end-to-end tests.
- Mock only what you own — don't mock the class under test, value objects, or (usually) the database in integration tests.

Code

A unit test with JUnit 5 + Mockito

```
@ExtendWith(MockitoExtension.class)      // enables @Mock/@InjectMocks
class OrderServiceTest {

   @Mock OrderRepository repo;            // the dependency is mocked
   @Mock PaymentClient payment;
   @InjectMocks OrderService service;     // real instance, mocks injected

   @Test
   @DisplayName("should place an order and charge the customer")
   void placeOrder_success() {
       // given (arrange)
       Order order = new Order(1L, BigDecimal.TEN);
       when(repo.findById(1L)).thenReturn(Optional.of(order));
       when(payment.charge(any())).thenReturn(true);

       // when (act)
       boolean result = service.placeOrder(1L);

       // then (assert)
       assertThat(result).isTrue();
       verify(payment, times(1)).charge(any());     // interaction check
       verify(repo).save(order);
       verifyNoMoreInteractions(payment);
   }

   @Test
   void placeOrder_missingOrder_throws() {
       when(repo.findById(99L)).thenReturn(Optional.empty());
       assertThrows(ResourceNotFoundException.class, () -> service.placeOrder(99L));
       verify(payment, never()).charge(any());
   }

   @ParameterizedTest
   @ValueSource(ints = {0, -1, Integer.MIN_VALUE})
   void rejectsInvalidQuantities(int qty) {
       assertThatThrownBy(() -> service.addItem(qty))
           .isInstanceOf(IllegalArgumentException.class);
   }
}
```

Testing the web layer and the persistence layer

```
// CONTROLLER slice: fast, no DB, no full context
@WebMvcTest(UserController.class)
class UserControllerTest {
   @Autowired MockMvc mvc;                 // drives HTTP requests in-process
   @MockBean UserService service;          // the service is mocked out

   @Test
   void getUser_returns200() throws Exception {
       when(service.findById(1L))
           .thenReturn(Optional.of(new UserResponse(1L, "Parvez", "p@x.com", 24)));

       mvc.perform(get("/api/users/1"))
          .andExpect(status().isOk())
          .andExpect(jsonPath("$.name").value("Parvez"))
          .andExpect(content().contentType(MediaType.APPLICATION_JSON));
   }

   @Test
   void getMissingUser_returns404() throws Exception {
       when(service.findById(2L)).thenReturn(Optional.empty());
       mvc.perform(get("/api/users/2")).andExpect(status().isNotFound());
   }
}

// REPOSITORY slice: runs against an embedded H2 with the real schema
@DataJpaTest
class UserRepositoryTest {
   @Autowired TestEntityManager em;
   @Autowired UserRepository repo;

   @Test
   void findByEmail_works() {
       em.persistAndFlush(new User("Parvez", "p@x.com"));
       assertThat(repo.findByEmail("p@x.com")).isPresent();
   }
}

// FULL integration test
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
class OrderApiIntegrationTest {
   @Autowired TestRestTemplate rest;
   @Test void fullFlow() { /* real HTTP against a running server */ }
}
```

AssertJ assertions and the naming convention

```
// Prefer AssertJ - fluent and far more readable than raw JUnit assertions
assertThat(name).isEqualTo("Parvez");
assertThat(list).hasSize(3).contains("a").doesNotContain("z");
assertThat(list).extracting(User::getName).containsExactly("a", "b", "c");
assertThat(map).containsKey("k").containsEntry("k", 1);
assertThatThrownBy(() -> service.run())
       .isInstanceOf(IllegalStateException.class)
       .hasMessageContaining("not initialised");
assertThat(optional).isPresent().get().extracting(User::getAge).isEqualTo(24);

// Naming: methodUnderTest_condition_expectedResult
//   placeOrder_validRequest_returnsCreated
//   findById_unknownId_throwsNotFound
//   withdraw_insufficientFunds_throwsException
// Or BDD style: givenX_whenY_thenZ

// Coverage target: high on business logic (services), don't chase 100% overall -
// testing getters/setters/framework code wastes effort.
```

SAY IT LIKE THIS IN THE INTERVIEW

"I write JUnit 5 tests with Mockito: @Mock the dependencies and @InjectMocks the class under test, then given/when/then with AssertJ's fluent assertions. For Spring layers I use test slices — @WebMvcTest with

MockMvc for controllers and @DataJpaTest for repositories — because they're far faster than loading the full context with @SpringBootTest. The pyramid I aim for is lots of unit tests, some integration tests, and a few end-to-end tests."

Flashcards

Q. @BeforeEach vs @BeforeAll?
A. @BeforeEach runs before every test method; @BeforeAll runs once before all tests in the class (and must be static unless the class is @TestInstance(PER_CLASS)).

Q. @Mock vs @InjectMocks?
A. @Mock creates a mock of a dependency. @InjectMocks creates a real instance of the class under test and injects the mocks into it.

Q. @WebMvcTest vs @DataJpaTest vs @SpringBootTest?
A. @WebMvcTest loads only the web layer (use @MockBean for services). @DataJpaTest loads JPA against an embedded DB. @SpringBootTest loads the whole context — slowest, used for integration tests.

Q. @Mock vs @MockBean?
A. @Mock is a plain Mockito mock (unit tests). @MockBean is Spring Boot's — it adds or replaces a bean in the Spring context (used in slice tests).

Q. What is MockMvc?
A. A Spring test utility that drives HTTP requests against your controllers without starting a real server, letting you assert status codes and JSON responses.

Q. What does verify() do in Mockito?
A. It asserts that an interaction happened — e.g. verify(mock, times(1)).method(args), verify(..., never()), verifyNoMoreInteractions().

Q. What is the test pyramid?
A. Many fast unit tests at the base, fewer integration tests in the middle, and few slow end-to-end/UI tests at the top.

Q. What should you NOT mock?
A. The class under test, simple value objects/DTOs, and (in integration tests) the database — mocking your own DB hides real query bugs.

Quiz

Q1. Which annotation injects a mock into the class you're testing?

[ ] A. @Mock
[x] B. @InjectMocks
[ ] C. @Autowired
[ ] D. @Spy

Why: @InjectMocks creates a real instance and injects the @Mock dependencies into it (via constructor, setter or field).

Q2. Which test slice is fastest for testing a controller in isolation?

[ ] A. @SpringBootTest
[x] B. @WebMvcTest
[ ] C. @DataJpaTest
[ ] D. @ContextConfiguration

Why: @WebMvcTest loads only the web layer and lets you @MockBean the service, so no DB or full context startup.

Q3. What does `verify(mock, times(1)).save(any())` assert?

[ ] A. The method returned once
[x] B. The mock's save method was called exactly once with any argument
[ ] C. The save took one second
[ ] D. The mock was created once

Why: verify checks interactions with the mock — useful when the method under test returns void and there's no return value to assert.

Q4. In JUnit 5, which annotation replaces JUnit 4's @Before?

[ ] A. @BeforeAll
[x] B. @BeforeEach
[ ] C. @Setup
[ ] D. @Initialize

Why: JUnit 5 (Jupiter) renamed it @BeforeEach; @BeforeAll replaces @BeforeClass and must be static by default.

9. Microservices & Architecture Overview Edge 30 min

Freshers aren't expected to run microservices, but they ARE expected to know the vocabulary and the trade-off. Being able to say "monolith first" correctly is a strong signal.

Key points

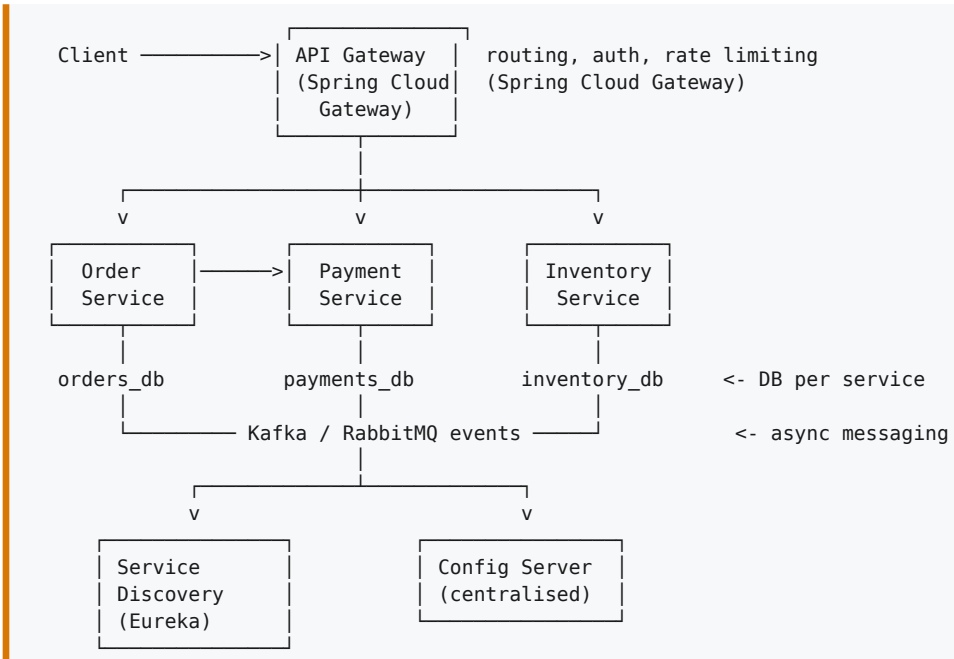
- **Monolith**: one deployable unit, shared DB. Simple to build, test and deploy; hard to scale teams and parts independently.
- **Microservices**: independently deployable services, each owning its data, communicating over HTTP (REST/gRPC) or messaging (Kafka/RabbitMQ).
- **Trade-offs**: microservices buy independent scaling, deployment and tech choice — at the cost of network failures, distributed transactions, debugging complexity and ops overhead. Start with a monolith unless you have a real reason.
- Key patterns: **API Gateway** (single entry point: routing, auth, rate limiting), **Service Discovery** (Eureka/Consul/K8s DNS), **Config Server** (centralised config), **Circuit Breaker** (Resilience4j — fail fast), **Distributed tracing** (Sleuth/Zipkin), **Centralised logging** (ELK).
- **CAP theorem**: in a network partition you must choose Consistency or Availability. (PACELC adds: else you trade latency vs consistency.)
- **12-factor app**: config in the environment, stateless processes, logs as event streams, etc.

Reference

Concern	Monolith	Microservices
Deployment	One unit, all or nothing	Independent per service
Scaling	Scale the whole app	Scale only the hot service
Data	One shared database	Database per service
Transactions	ACID, simple	Sagas / eventual consistency
Complexity	In the code	In the infrastructure
Best for	Small teams, early products	Large systems, many teams, uneven load

Code

What a typical Spring Cloud setup looks like




```
Resilience4j circuit breaker  -> fail fast when a downstream service is down
Spring Cloud Sleuth / Micrometer -> trace IDs across services
Actuator + Prometheus + Grafana  -> metrics and health
```

Service-to-service calls and resilience

```
// Declarative REST client (Spring Cloud OpenFeign)
@FeignClient(name = "payment-service", fallback = PaymentFallback.class)
public interface PaymentClient {
   @PostMapping("/api/payments")
   PaymentResponse charge(@RequestBody PaymentRequest req);
}

// Or the modern HTTP interface (Spring 6)
@HttpExchange("/api/payments")
public interface PaymentApi {
   @PostExchange PaymentResponse charge(@RequestBody PaymentRequest req);
}

// Circuit breaker: stop calling a failing service so it can recover
@CircuitBreaker(name = "paymentService", fallbackMethod = "chargeFallback")
@Retry(name = "paymentService")
@TimeLimiter(name = "paymentService")
public CompletableFuture<PaymentResponse> charge(PaymentRequest req) {
   return CompletableFuture.supplyAsync(() -> client.charge(req));
}
public CompletableFuture<PaymentResponse> chargeFallback(PaymentRequest req, Throwable t) {
   return CompletableFuture.completedFuture(PaymentResponse.pending());
}

// Saga pattern: instead of a distributed transaction, each service commits
// locally and publishes an event; compensating actions undo earlier steps
// if a later step fails (e.g. cancel order -> refund payment -> restock).
```

How to answer 'monolith or microservices?'

```
A strong answer:
"It depends on the problem and the team. I'd start with a well-structured
monolith — modular packages, clear boundaries, no shared mutable state —
because it's far cheaper to build, test, debug and deploy. I'd move to
microservices when there's a specific pressure: one part needs to scale
independently, teams are stepping on each other's deploys, or different
parts need different technology or availability guarantees.

If we did go distributed, each service would own its database, communicate
over REST for queries and events for state changes, sit behind an API
gateway, and use circuit breakers plus centralised logging and tracing —
because the hard part isn't splitting the code, it's handling partial
failure, eventual consistency and observability."

Red flags to avoid:
 "Microservices are always better"
 "Let's split it into 20 services on day one"
 Sharing one database across services (a distributed monolith)
```

SAY IT LIKE THIS IN THE INTERVIEW

"I'd start with a modular monolith and only split into microservices for a concrete reason — independent scaling, team autonomy or different availability needs. If we go distributed, each service owns its data, they communicate via REST or events, sit behind an API gateway, and need circuit breakers, centralised logging and distributed tracing, because partial failure and eventual consistency are the real costs."

Flashcards

Q. Monolith vs microservices — the core trade-off?
A. Monoliths are simpler to build, test, debug and deploy. Microservices give independent scaling, deployment and team autonomy, but add network failures, distributed data problems and operational complexity.

Q. What is an API Gateway?
A. A single entry point for clients that handles routing, authentication, rate limiting, CORS and request aggregation — so those concerns aren't duplicated in every service.

Q. What is a circuit breaker?
A. A resilience pattern (Resilience4j) that stops calling a failing downstream service and returns a fallback, giving it time to recover and preventing cascading failures.

Q. What is service discovery?
A. A registry (Eureka, Consul, Kubernetes DNS) where services register themselves and look each other up by name instead of hard-coded host:port.

Q. What is the Saga pattern?
A. A way to manage data consistency across services without distributed transactions: each service commits locally and emits an event, with compensating transactions to roll back if a later step fails.

Q. What is eventual consistency?
A. The system may be briefly inconsistent while updates propagate, but converges to a consistent state — the normal trade-off when each service owns its own database.

Q. State the CAP theorem.
A. During a network partition, a distributed system must choose between Consistency (all nodes see the same data) and Availability (every request gets a response) — it can't guarantee both.

Q. What is distributed tracing?
A. Propagating a trace ID across service calls so you can follow one request end to end (Sleuth/Micrometer + Zipkin or Jaeger).

Q. What's the database-per-service rule?
A. Each microservice owns its data and no other service reads it directly — sharing a database creates a distributed monolith and couples deployments.

Quiz

Q1. What is the biggest hidden cost of microservices?

[ ] A. They're harder to code

[x] B. Operational complexity: network failures, distributed data, debugging and observability

[ ] C. Java doesn't support them

[ ] D. They cost more in licences

> **Why:** The code is easy to split; the difficulty is partial failure, eventual consistency, tracing a request across services, and deploying/monitoring many moving parts.

Q2. What does a circuit breaker do when a downstream service is down?

[ ] A. Retries forever

[x] B. Opens and returns a fallback immediately, giving the service time to recover

[ ] C. Deletes the service

[ ] D. Switches to a backup database

> **Why:** After failures exceed a threshold the circuit opens; calls short-circuit to the fallback instead of piling on load, then it half-opens to test recovery.

Q3. Two services must both succeed or neither should. In microservices you'd typically use:

[ ] A. A distributed transaction (2PC)

[x] B. The Saga pattern with compensating actions

[ ] C. A shared database

[ ] D. synchronized blocks

> **Why:** 2PC doesn't scale and couples availability. Sagas commit locally per service and publish events, with compensating transactions to undo on failure.

Q4. According to CAP, during a network partition you must choose:

[x] A. Consistency or Availability

[ ] B. Speed or Security

[ ] C. SQL or NoSQL

[ ] D. REST or gRPC

> **Why:** Partition tolerance isn't optional in a distributed system, so the real choice is C or A during a partition.

10. Explaining Your Project (the round that decides it) Must-know 40 min

For a fresher, "tell me about your project" is the highest-leverage question in the whole interview. Most candidates ramble. You'll use a structure — and rehearse it out loud today, not tomorrow morning.

Key points

- Use **30-60 seconds** for the overview, then let questions drive the detail. Rambling for five minutes loses the room.
- Structure: **What** it does → **Who** uses it → **Tech stack** → **Your specific contribution** → **A challenge you solved** → **What you'd improve**.
- Name the *problem* before the features: "The manual process took 2 hours a day, so we automated it."
- Be ready for: "Why this technology?", "What was the hardest bug?", "How did you test it?", "What would you do differently?"
- Prepare numbers: users, records, response time, test coverage, % improvement. Specifics beat adjectives.
- **Never claim something you can't discuss for two minutes.** Every bullet on your resume is a question waiting to be asked.

Code

A 45-second project pitch you can adapt

```
"I built an Order Management System — a REST API that handles the full
order lifecycle: customers place orders, the service validates stock,
processes payment and sends a confirmation email.

The stack is Java 17 with Spring Boot 3, Spring Data JPA on PostgreSQL,
with Spring Security using JWT for authentication. Maven for the build,
JUnit and Mockito for tests, and it's Dockerised.

I owned the order and payment modules end to end. Specifically I designed
the schema, implemented the REST endpoints with DTO validation, and set up
the JPA entity relationships.

The hardest problem was performance: the order history endpoint took 3
seconds because of an N+1 query — one query for the orders and one for
each order's items. I found it by enabling Hibernate SQL logging, fixed it
with a JOIN FETCH and added pagination, which brought it down to under
200ms.

If I were rebuilding it, I'd add integration tests with Testcontainers and
move the email sending to an async queue so the API doesn't wait on SMTP."

// Why this works:
//   1. What it does (2 sentences, no fluff)
//   2. Concrete stack (shows breadth)
//   3. YOUR contribution (ownership, not "we did")
//   4. A real problem + how you diagnosed it + the measurable result
//   5. Self-awareness about improvements (maturity signal)
```

The five follow-up questions and how to handle each

```
Q: "Why did you choose Spring Boot?"
A: "It removes the boilerplate - auto-configuration gave me an embedded
  server, Jackson and a DataSource without manual config, and the starter
  POMs keep dependency versions compatible. It let me focus on features."

Q: "What was the hardest bug you faced?"
A: Use STAR: Situation, Task, Action, Result.
  "The order history page was slow. I enabled SQL logging and saw 200
  queries instead of 1 - a classic N+1 from lazy loading. I fixed it with
  JOIN FETCH plus pagination. Response time went from 3s to 180ms."

Q: "How did you handle security?"
A: "Spring Security with JWT. Passwords are BCrypt-hashed, stateless
  sessions, roles enforced with @PreAuthorize at the method level, and
  CORS configured explicitly. Validation on every request body."

Q: "How did you test it?"
A: "JUnit 5 and Mockito for unit tests on the service layer, @WebMvcTest
  with MockMvc for controllers, @DataJpaTest for repositories. Around
  70% coverage on the business logic."

Q: "What would you do differently?"
A: "Add integration tests with Testcontainers, extract the email sending
  into an async queue, and add Flyway for schema migrations instead of
  ddl-auto=update."
```

Questions YOU should ask them (prepare 3)

```
Good questions signal real interest:
 1. "What does the team's tech stack look like day to day, and which Java
     version are you on?"
 2. "How do you handle code reviews, testing and deployments here?"
 3. "What does success look like for someone in this role in the first
     six months?"
 4. "What's the biggest technical challenge the team is facing right now?"
 5. "Is there anything from my background you'd like me to clarify?"

Avoid in the first round:
 - Salary, leave policy, work-from-home (save these for HR)
 - Anything you could have found on their website

Closing the interview:
 "Thank you — I enjoyed discussing the role. I'm genuinely interested in
  the backend work you described, especially [specific thing]. What are
  the next steps?"
```

SAY IT LIKE THIS IN THE INTERVIEW

Structure beats talent here. Rehearse your 45-second pitch out loud today — problem, stack, your contribution, one hard problem with a measurable result, and one improvement. Then every bullet on your resume should survive two minutes of follow-up questions.

Flashcards

Q. Give the 6-part structure for explaining a project.
A. What it does → Who uses it → Tech stack → Your specific contribution → A challenge and how you solved it → What you'd improve.

Q. What is the STAR method?
A. Situation, Task, Action, Result — a structure for answering behavioural questions with a concrete story and a measurable outcome.

Q. How long should your initial project explanation be?
A. 30-60 seconds. Then let their questions drive the depth — rambling for five minutes loses the room.

Q. What's the strongest thing you can include in a project answer?
A. A specific problem you diagnosed, the technique you used, and a measurable result ("N+1 query, fixed with JOIN FETCH, 3s → 180ms").

Q. What should you never put on your resume?
A. Anything you can't discuss for two minutes. Every bullet is an invitation to ask about it.

Q. What questions should you ask the interviewer?
A. About the stack and Java version, engineering practices (reviews, testing, deploys), what success looks like in six months, and current technical challenges.

Q. How do you answer 'what's your weakness?'
A. Name a real, non-fatal weakness and show what you're doing about it — e.g. "I used to skip writing tests; now I write them alongside the service layer."

Q. What should you do if you don't know an answer?
A. Say so honestly, then reason aloud: "I haven't used that directly, but here's how I'd approach it..." Interviewers rate the reasoning, not the guess.

Quiz

Q1. What is the ideal length for your opening project explanation?

[x] A. 30-60 seconds
[ ] B. 5 minutes
[ ] C. 10 seconds
[ ] D. As long as it takes

Why: Short enough to hold attention, detailed enough to earn follow-ups. Then let their questions drive the depth.

Q2. Which element most strengthens a project answer?

[ ] A. Listing every technology you touched
[x] B. A specific problem you diagnosed, the fix, and a measurable result
[ ] C. Saying the team was great
[ ] D. Mentioning the project was complex

Why: Concrete, measurable problem-solving is what separates a real contributor from someone who was in the room.

Q3. You're asked something you genuinely don't know. Best response?

[ ] A. Guess confidently
[x] B. Say you don't know, then reason through how you'd approach it
[ ] C. Change the subject
[ ] D. Say 'we didn't cover that in training'

Why: Honesty plus visible reasoning scores far better than a bluff. Interviewers are testing how you think under uncertainty.

Q4. What does STAR stand for?

[ ] A. Stack, Tech, Architecture, Result
[x] B. Situation, Task, Action, Result
[ ] C. Start, Test, Analyse, Refactor
[ ] D. Scope, Time, Assignments, Review

Why: STAR is the standard structure for behavioural answers — set the scene, your task, what you did, and the outcome (with numbers if possible).

Rapid Fire: Rapid Fire: Traps, HR & Last-Minute Drill

Output prediction, behavioural questions, one-minute answers

5 topics · 31 flashcards · 20 quiz questions · ~2.1 hours

> **Goal.** The night before and the morning of: run the output traps until you get them all right, read your one-minute answers out loud, and walk in with your HR stories ready.

Schedule

- T-1 evening (1h): Output traps — aim for 90%+
- T-1 evening (45m): One-minute answers out loud, without notes
- T-1 evening (30m): Write your HR stories in STAR format
- Morning of (30m): Re-run the traps, skim the cheat sheet
- 10 min before: Breathe. You know more than you think.

1. Output Prediction Traps (Part 1: Strings & Numbers) Trap 25 min

These are the questions where the interviewer writes five lines on the board and asks "what does this print?" They're testing whether you actually understand the memory model, not whether you've memorised syntax. Run the quiz until you're perfect.

Key points

- String literal pooling: `==` compares references, `equals()` compares contents.
- Compile-time constant folding happens for literals, not for variables.
- Integer cache: -128 to 127 is cached; beyond that every boxing is a new object.
- Integer division truncates; floating-point division by zero gives Infinity, not an exception.
- `+` with a String anywhere converts the whole expression to concatenation, left to right.

Code

Six traps in one go — cover the output and check

```
// 1
String a = "hi", b = "hi";
System.out.println(a == b);                     // true  (pool)

// 2
String c = new String("hi");
System.out.println(a == c);                     // false (new object)

// 3
String d = "h" + "i";
System.out.println(a == d);                     // true  (constant folding)

// 4
String e = "h"; String f = e + "i";
System.out.println(a == f);                     // false (runtime concat)

// 5
Integer x = 100, y = 100;
Integer p = 200, q = 200;
System.out.println(x == y);                     // true  (cache)
System.out.println(p == q);                     // false (outside cache)

// 6
System.out.println(10 / 3);                     // 3
System.out.println(10 / 3.0);                   // 3.3333333333333335
System.out.println(1.0 / 0);                    // Infinity
System.out.println("5" + 2 + 3);                // "523"
System.out.println(5 + 2 + "3");                // "73"
```

```
Output:
true
false
true
false
true
false
3
3.3333333333333335
Infinity
523
73
```

Trap: null concatenation and StringBuilder equality

```
String s = null;
System.out.println(s + "!");          // "null!" - no NPE!
// s.concat("!");                     // NullPointerException
// s.length();                        // NullPointerException

StringBuilder sb1 = new StringBuilder("hi");
StringBuilder sb2 = new StringBuilder("hi");
System.out.println(sb1 == sb2);       // false
System.out.println(sb1.equals(sb2));  // false - NOT overridden!
System.out.println(sb1.toString().equals(sb2.toString()));  // true
```

Output:

```
null!
false
false
true
```

SAY IT LIKE THIS IN THE INTERVIEW

"String literals are interned in a pool and reused; `new String()` always creates a fresh object. Compile-time constants get folded, but concatenation involving a variable happens at runtime. Wrappers cache -128 to 127, so == works there and fails beyond it — which is why I always use equals()."

Flashcards

Q. `String a="hi"; String b=new String("hi"); a==b`?
A. false — new String() creates a separate heap object instead of reusing the pooled literal.

Q. `Integer a=127,b=127; a==b` and `Integer c=128,d=128; c==d`?
A. true then false — the Integer cache covers -128..127 only.

Q. `"5"+2+3` vs `5+2+"3"`?
A. "523" and "73" — evaluation is left to right and the first String encountered converts the rest.

Q. Does `null + "!"` throw NPE?
A. No — String concatenation converts null to the text "null". Calling a method on null throws NPE.

Q. Do StringBuilder and StringBuffer override equals()?
A. No — their equals() is reference equality. Convert to String first to compare contents.

Quiz

Q1. What does this print? `String s1="Java"; String s2="Ja".concat("va"); String s3=s1.intern(); System.out.println(s1==s2); System.out.println(s1==s3);`

[ ] A. true true
[x] B. false true
[ ] C. false false
[ ] D. true false

Why: concat() runs at runtime and returns a new String, so s1 != s2. intern() returns the pooled instance, so s1 == s3.

Q2. `System.out.println((byte) 130);`

[ ] A. 130
[x] B. -126
[ ] C. 127
[ ] D. Compile error

Why: byte is signed 8-bit (-128..127). 130 in binary is 10000010, which as a signed byte is -126. Overflow wraps around.

Q3. What is the output? `int i=0; System.out.println(i++ + i++ + i++);`

[x] A. 3
[ ] B. 6

[ ] C. 0
[ ] D. 1

> **Why:** 0 + 1 + 2 = 3. Each i++ yields the current value then increments: first is 0 (i→1), second is 1 (i→2), third is 2 (i→3).

Q4. `char c = 'A'; System.out.println(c + 1);`

[ ] A. B
[x] B. 66
[ ] C. A1
[ ] D. Compile error

> **Why:** char promotes to int in arithmetic: 'A' is 65, so 66. To get 'B' you'd cast: `(char)(c + 1)`.

Q5. Which of these throws NullPointerException?

[ ] A. String s = null; System.out.println(s + "");
[ ] B. String s = null; System.out.println(String.valueOf(s));
[x] C. String s = null; System.out.println(s.toString());
[ ] D. String s = null; System.out.println(s == null);

> **Why:** Calling any method on null throws NPE. Concatenation, String.valueOf() and == are all null-safe (String.valueOf returns "null").

Q6. `double d = 0.1 + 0.2; System.out.println(d == 0.3);

[ ] A. true
[x] B. false
[ ] C. Compile error
[ ] D. Throws ArithmeticException

> **Why:** 0.1 + 0.2 = 0.30000000000000004 in binary floating point. Never compare doubles with == — use a tolerance (epsilon) or BigDecimal for money.

2. Output Prediction Traps (Part 2: OOP & Exceptions) Trap 25 min

Part two covers method dispatch, initialisation order, static vs instance, and the finally block. These separate people who've written Java from people who understand it.

Key points

- Instance methods dispatch at runtime on the object; static methods and fields resolve at compile time on the reference type.
- Fields are never polymorphic — field access uses the declared type.
- Initialisation: static (once) → instance blocks → constructor, parent before child.
- `finally` always runs — and a `return` inside it overrides the try's return.
- The return value is captured before `finally` executes, so mutating the variable there changes nothing.

Code

Dispatch and hiding

```
class A {
   int v = 1;
   void show()     { System.out.println("A.show"); }
   static void st(){ System.out.println("A.static"); }
}
class B extends A {
   int v = 2;
   @Override void show() { System.out.println("B.show"); }
   static void st()       { System.out.println("B.static"); }
}

A ref = new B();
ref.show();            // B.show   <- runtime dispatch (overriding)
ref.st();              // A.static <- compile-time (static HIDING)
System.out.println(ref.v);   // 1   <- fields are NOT polymorphic

// Cast to call the parent's version:
((A) new B()).show();  // B.show - a cast doesn't change runtime dispatch!
```

```
Output:
B.show
A.static
1
B.show
```

Initialisation order, end to end

```
class Parent {
   static { System.out.println("P static"); }
   { System.out.println("P instance"); }
   Parent() { System.out.println("P ctor"); }
}
class Child extends Parent {
   static { System.out.println("C static"); }
   { System.out.println("C instance"); }
   Child() { System.out.println("C ctor"); }
}
public class Main {
   public static void main(String[] a) {
       System.out.println("--- first ---");
       new Child();
       System.out.println("--- second ---");
       new Child();
   }
}
// Static blocks run ONCE, parent before child, at first class load.
// Then per object: P instance -> P ctor -> C instance -> C ctor.
```

Output:
```
--- first ---
P static
C static
P instance
P ctor
C instance
C ctor
--- second ---
P instance
P ctor
C instance
C ctor
```

finally and return — the three cases

```
static int a() { try { return 1; } finally { return 2; } }
static int b() { int x = 1; try { return x; } finally { x = 2; } }
static int c() {
   try { return 1 / 0; }
   catch (Exception e) { return 3; }
   finally { System.out.print("finally "); }
}
System.out.println(a());   // 2  <- finally's return wins
System.out.println(b());   // 1  <- value snapshotted before finally
System.out.println(c());   // "finally 3" - finally runs before the return lands
```

Output:
```
2
1
finally 3
```

Trap: constructor calls an overridable method

```
class Base {
   Base() { print(); }               // calls an OVERRIDABLE method!
   void print() { System.out.println("Base"); }
}
class Derived extends Base {
   private String name = "derived";   // initialised AFTER super()
   @Override void print() { System.out.println("name = " + name); }
}
new Derived();   // "name = null" - the subclass field isn't set yet!

// LESSON: never call overridable methods from a constructor.
// The subclass's fields are still at their default values.
```

```
Output:
name = null
```

SAY IT LIKE THIS IN THE INTERVIEW

"Instance methods are dispatched at runtime by the object's type, but static methods are hidden and resolved at compile time by the reference type — and fields are never polymorphic at all. Initialisation runs static blocks once, parent before child, then instance blocks and constructors per object. And finally always runs; a return in finally overrides the try's return, though the returned value is snapshotted before finally executes."

Flashcards

Q. Are fields polymorphic in Java?
A. No. Only instance methods use dynamic dispatch. Field access is resolved by the declared (reference) type at compile time.

Q. What is method hiding vs overriding?
A. Static methods are hidden (resolved by reference type at compile time); instance methods are overridden (resolved by object type at runtime).

Q. Does a cast change which overridden method runs?
A. No — casting only changes the compile-time view. The JVM still dispatches to the actual object's method.

Q. Full initialisation order for `new Child()` where Child extends Parent?
A. Parent static → Child static → Parent instance block → Parent constructor → Child instance block → Child constructor. Statics run only once per class load.

Q. Why shouldn't a constructor call an overridable method?
A. The subclass's fields aren't initialised yet (super() runs first), so the overridden method sees default values — a classic source of NPEs.

Q. What does `try { return 1; } finally { return 2; }` return?
A. 2. A return in finally overrides the try's return and swallows any exception — which is why it's an anti-pattern.

Quiz

Q1. What does this print? `class A{ int x=1; void p(){System.out.print(x);} } class B extends A{ int x=2; } new B().p();`

[x] A. 1
[ ] B. 2
[ ] C. Compile error
[ ] D. 0

Why: Fields aren't polymorphic. A.p() prints A's x (1); B declares its own x which merely shadows it in B's scope.

Q2. `static int m(){ try{ throw new RuntimeException(); } finally{ return 5; } }`

[ ] A. Throws the exception
[x] B. Returns 5 — the exception is swallowed
[ ] C. Compile error
[ ] D. Returns 0

Why: A return in finally discards the in-flight exception entirely. This is exactly why returning from finally is discouraged.

Q3. What is the output order when creating a Child of a Parent for the second time?

[ ] A. All statics and instance blocks again
[x] B. Only instance blocks and constructors — statics ran once at first class load
[ ] C. Nothing
[ ] D. Only constructors

Why: Static initialisers run exactly once, when the class is first loaded. Instance initialisers and constructors run for every new object.

Q4. Why does `new Derived()` print "name = null" when the constructor calls an overridden print()?

[ ] A. The field is final
[x] B. super() runs before the subclass's field initialisers, so the field still holds its default value
[ ] C. print() is static
[ ] D. It's a compiler bug

Why: Order: allocate → zero fields → call super() (which invokes the overridden method) → then run the subclass's field initialisers and constructor body.

Q5. Which is true about calling an overridden method from a constructor?

[ ] A. It's recommended for initialisation
[x] B. It's a known anti-pattern — make such methods private, final or static
[ ] C. It's required by Java
[ ] D. It only fails with primitives

Why: Calling overridable methods from constructors leads to partially-constructed objects. Keep constructor-called methods private/final/static.

3. One-Minute Answers (Verbal Rapid Fire) Must-know 30 min

The rapid-fire round gives you about 60 seconds per question. These are the definitions you should be able to deliver without thinking — say them out loud until they're automatic.

Key points

- Answer in 2–3 sentences. Definition → key mechanism → why it matters.
- If you blank, say "I haven't used that directly" and reason from first principles — never bluff.
- Pauses are fine. Take two seconds, structure the answer, then speak.
- Volume and pace matter: slow down, and stop at the end of each answer instead of trailing off.

Code

20 answers to have on the tip of your tongue

```
1.  OOP?            Encapsulation, Inheritance, Polymorphism, Abstraction.
2.  Platform independent?  javac -> bytecode, the JVM runs it anywhere.
3.  JDK/JRE/JVM?    Dev kit / runtime / the engine that executes bytecode.
4.  == vs equals?   == compares references, equals compares contents.
5.  Why is String immutable?  Security, thread-safety, hash caching, pooling.
6.  final?          A constant variable, a non-overridable method, a non-extendable class.
7.  Overload vs override?  Compile-time vs runtime polymorphism.
8.  Abstract vs interface?  Shared state + IS-A vs a contract, multiple inheritance of type.
9.  Checked vs unchecked?  Compile-time enforced vs RuntimeExceptions from bugs.
10. Collection vs Collections?  The interface vs the utility class.
11. ArrayList vs LinkedList?  Array-backed O(1) access vs linked, fast end inserts.
12. HashMap internals?  Buckets by hash, chaining, treeify at 8, resize at 0.75.
13. Why override hashCode with equals? Equal objects must hash equally or hash collections
    break.
14. Fail-fast vs fail-safe?  Throw CME vs iterate a snapshot.
15. Generics?  Compile-time type safety implemented by type erasure.
16. PECS?  Producer extends, Consumer super.
17. Lambda?  An instance of a functional interface; captures must be effectively final.
18. Stream?  A lazy pipeline: source -> intermediate -> terminal; consumed once.
19. synchronized vs volatile?  Exclusion + visibility vs visibility only.
20. sleep vs wait?  Holds the lock vs releases it (and needs notify).
```

Full-stack rapid fire

```
21. IoC vs DI?    The principle vs the mechanism (constructor injection preferred).
22. Bean scope?   Singleton by default; also prototype, request, session.
23. Spring vs Boot?  Boot = auto-config + starters + embedded server + Actuator.
24. Auto-config?  Conditional beans (@ConditionalOnClass / @ConditionalOnMissingBean).
25. @Component/@Service/@Repository/@Controller?  Stereotypes per layer;
    @Repository adds exception translation.
26. DispatcherServlet?  Front controller that routes each request to a handler.
27. @PathVariable vs @RequestParam vs @RequestBody?  Path / query / JSON body.
28. Why DTOs?     Stable contract, no leaked fields, no JSON recursion.
29. 201 vs 204?   Created (POST) vs No Content (DELETE).
30. JPA vs Hibernate?  Spec vs implementation; Spring Data generates repositories.
31. N+1 problem?  1 query + N lazy loads; fix with JOIN FETCH / @EntityGraph.
32. Lazy vs Eager?  Load on access vs load immediately; @ManyToOne defaults EAGER.
33. @Transactional placement?  Service layer, readOnly for queries, proxy-based.
34. Dirty checking?  Hibernate auto-UPDATEs changed managed entities at flush.
35. LazyInitializationException?  Lazy access after the session closed.
36. Index trade-off?  Faster reads, slower writes, storage cost.
37. WHERE vs HAVING?  Filter rows before grouping vs groups after.
38. INNER vs LEFT JOIN?  Matching rows only vs all left rows with NULLs.
39. JWT flow?  Login -> signed token -> Bearer header -> filter validates -> SecurityContext.
40. BCrypt?  Salted, deliberately slow password hashing - never encrypt passwords.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Answer in three beats: what it is, how it works, why it matters. Two sentences, then stop and let them ask for more."

Flashcards

Q. Give a 2-sentence answer for 'what is polymorphism?'
A. Polymorphism lets the same call take many forms: overloading is resolved at compile time from argument types, overriding at runtime from the actual object type. It's what lets me write `List l = new ArrayList<>()` and swap the implementation without changing callers.

Q. Give a 2-sentence answer for 'what is dependency injection?'
A. DI means a class receives its dependencies from outside rather than creating them with `new`. I prefer constructor injection because it keeps fields final, makes dependencies explicit and enables unit testing with plain mocks.

Q. Give a 2-sentence answer for 'how does HashMap work?'
A. It's an array of buckets indexed by hash(key) & (n-1); on a collision entries are chained, and since Java 8 buckets over 8 entries are treeified to keep the worst case at O(log n). It doubles and rehashes when size exceeds 0.75 × capacity.

Q. Give a 2-sentence answer for 'what is the N+1 problem?'
A. One query fetches the parents and then one more fires per parent to lazily load each child collection. I fix it with JOIN FETCH or @EntityGraph so it becomes a single query.

Q. Give a 2-sentence answer for 'checked vs unchecked exceptions?'
A. Checked exceptions are verified at compile time — I must catch or declare them — while unchecked RuntimeExceptions usually indicate a bug I should fix. Error types like OutOfMemoryError should never be caught.

Q. How do you handle a question you don't know?
A. Say honestly that you haven't used it, then reason aloud from first principles about how you'd approach it. Interviewers score the reasoning far higher than a confident guess.

Q. Why is pausing before you answer a good thing?
A. A two-second pause signals thoughtfulness instead of panic, and it stops you from rambling into an answer you can't defend.

Quiz

Q1. What's the recommended structure for a rapid-fire answer?

[ ] A. Talk for as long as you can to show depth
[x] B. Definition → key mechanism → why it matters, in 2-3 sentences
[ ] C. Name as many related terms as possible
[ ] D. Ask the interviewer to clarify first

Why: Three beats, then stop. It shows structure and lets the interviewer pull more detail if they want it.

Q2. You freeze on a question. The best move is:

[ ] A. Guess loudly
[x] B. Pause, say you're not certain, and reason from first principles
[ ] C. Say 'next question'
[ ] D. Repeat the question back

Why: Visible reasoning is what's being assessed. A calm, structured attempt beats both silence and a bluff.

4. HR & Behavioural Questions Must-know 30 min

Technical rounds get you shortlisted; HR rounds get you the offer — or quietly cut you. The good news: there are only about ten questions, and you can prepare all of them tonight.

Key points

- Every behavioural answer should be a **story**: STAR (Situation, Task, Action, Result) with a measurable outcome.
- **Never badmouth** a previous employer, manager or college. Frame everything as a learning experience.
- For "why should we hire you?": match your skills to *their* requirements, don't recite your resume.
- For "where do you see yourself in 5 years?": show ambition that fits the role's growth path (senior dev / tech lead), not "your job".
- Have a genuine reason for wanting *this* company — their stack, their product, their scale.
- Prepare two questions to ask them (about the team's stack and engineering practices).

Code

The ten questions — with a model angle for each

```
1. "Tell me about yourself."
   Present -> Past -> Future. 90 seconds max.
   "I'm a Java full-stack developer. I trained at [institute], where I built
   an Order Management System with Spring Boot and JPA - I owned the order
   and payment modules. Before that I completed my degree in [field]. I'm
   looking for a role where I can grow on real backend systems, which is
   why this position interested me."

2. "Why should we hire you?"
   Match their needs: "You mentioned you need someone who can work across
   Spring Boot and SQL. I built [project] using exactly that, including
   fixing an N+1 query problem, so I can contribute from week one while
   picking up your domain quickly."

3. "What are your strengths?"
   Pick 2 that match the job, with evidence. "I'm persistent with debugging -
   I spent two days on a transaction rollback issue and ended up learning
   how Spring's proxy-based transactions actually work."

4. "What is your biggest weakness?"
   Real, non-fatal, with a fix in progress. "I used to under-invest in
   tests. Now I write JUnit tests alongside every service method, and my
   last project had around 70% coverage on business logic."

5. "Where do you see yourself in 5 years?"
   "Growing into a senior backend engineer - deeper on system design,
   performance and mentoring juniors. I want to be the person who owns a
   service end to end."

6. "Why do you want to work here?"
   Be specific: their product, scale, stack or engineering blog. Never
   "because it's a big company".

7. "Tell me about a conflict / difficult teammate."
   STAR. Depersonalise it, focus on communication and the outcome.

8. "Describe a time you failed."
   Real failure, ownership (no blaming), what you changed. Show the lesson.

9. "What are your salary expectations?" (usually a later round)
   Give a researched range and stay flexible: "Based on the market for
   Java developers with 0-2 years, I'm looking at around X to Y, but I'm
   open to discussing the full package."

10. "Do you have any questions for us?"
   ALWAYS have two. "What does the team's stack look like day to day?"
   "How do you handle code review and deployments?"
```

STAR template — fill in two stories tonight

```
STORY 1: A hard technical problem
 S - "Our order history endpoint took 3 seconds to load."
 T - "I needed to find and fix the bottleneck before the demo."
 A - "I enabled Hibernate SQL logging and saw 200 queries instead of one -
      a classic N+1 from lazy-loaded order items. I rewrote the query with
      JOIN FETCH and added pagination."
 R - "Response time dropped from 3 seconds to under 200ms, and I added
      SQL logging to our dev config so we'd catch it earlier next time."

STORY 2: A disagreement or failure
 S - "I disagreed with a teammate about returning entities directly from
      our controllers instead of DTOs."
 T - "We needed to decide before the API was published to the frontend
      team."
 A - "I built a quick prototype showing how bidirectional JPA relations
      caused infinite JSON recursion and leaked the password hash field."
 R - "We standardised on DTOs for every endpoint, and I wrote the mapping
      layer. The frontend contract stopped changing with our DB schema."

// Rules for these stories:
//   - Own your part ("I", not "we", when describing your action)
//   - Include a NUMBER in the result
//   - End with what you learned or changed going forward
//   - Never criticise a person - criticise the problem
```

Red flags to avoid

```
DON'T:
 x "We did everything as a team" for every answer (no ownership)
 x Badmouthing a past manager, company or college
 x "I don't have any weaknesses"
 x "I don't know, I haven't thought about it" with no follow-up
 x Lying about a technology on your resume - it WILL come up
 x Checking your phone, or slouching on camera
 x Answering salary first if asked early - deflect politely:
   "I'd like to learn more about the role first; I'm open to a
    discussion based on the full package."

DO:
 O Sit up, smile, make eye contact (or look at the camera)
 O Have your resume, a pen and a notepad ready
 O Say "that's a great question" sparingly - once is fine, five is filler
 O Mirror their language: if they say "backend", you say "backend"
 O Send a thank-you note within 24 hours: reference something specific
   you discussed
```

SAY IT LIKE THIS IN THE INTERVIEW

"Every behavioural answer is a short story with a measurable result, and I never criticise a past employer or manager. My weaknesses are real but paired with what I'm doing about them, and I always have two questions about the team and its practices."

Flashcards

Q. What is the 'Present → Past → Future' structure?
A. The shape for 'tell me about yourself': who you are now, what you've done that's relevant, and what you're looking for next. 90 seconds maximum.

Q. How do you answer 'what's your weakness?'
A. A real, non-fatal weakness plus concrete evidence you're fixing it. Avoid fake humility ('I work too hard') and anything central to the job.

Q. What is STAR?
A. Situation, Task, Action, Result — the structure for every behavioural answer. Own the action with 'I' and put a number in the result.

Q. How should you talk about a previous employer?
A. Neutrally or positively, always. Criticising a past manager or company is a major red flag regardless of whether you're right.

Q. Should you ask questions at the end?
A. Yes, always — two of them, about the team's stack, engineering practices or current challenges. Never about salary in a technical round.

Q. What's the biggest resume mistake?
A. Listing a technology you can't discuss for two minutes. Everything on the page is an invitation to ask about it.

Q. How do you handle a salary question you're not ready for?
A. Deflect politely: 'I'd like to understand the role better first, but I'm open based on the full package' — and research the market range beforehand so you're never guessing.

Quiz

Q1. What's the best structure for 'tell me about a time you faced a challenge'?

[ ] A. Describe the challenge in detail, then stop
[x] B. STAR: Situation, Task, Action, Result — with a number in the result
[ ] C. Explain what your teammate did
[ ] D. Focus on how stressful it was

Why: STAR keeps you concrete and short. Own the action with 'I' and quantify the outcome — that's what makes it credible.

Q2. How should you respond when asked about a previous employer you didn't like?

[ ] A. Explain everything that was wrong there
[x] B. Stay neutral and frame it as a learning experience
[ ] C. Say you can't discuss it
[ ] D. Blame your manager by name

Why: Complaining about past employers is one of the fastest ways to fail an HR round, however justified it feels.

Q3. How long should 'tell me about yourself' be?

[ ] A. 30 seconds
[x] B. About 90 seconds
[ ] C. 5 minutes
[ ] D. As long as they let you talk

Why: Present → Past → Future in ~90 seconds. Long enough to show relevance, short enough to keep their attention and earn follow-up questions.

Q4. Which is a GOOD question to ask the interviewer?

[ ] A. How much vacation do I get?
[x] B. What does the team's stack look like day to day, and how do you handle code reviews?
[ ] C. Did I get the job?
[ ] D. What does your company do?

Why: Ask about the work, the team and the practices. Never ask something you could have found on their website, and save compensation for HR.

5. Final 24 Hours: What To Do & Skip

Plan 15 min

You're four days in. Here's how to spend the last day so you walk in sharp instead of exhausted.

Key points

- **Do**: sleep 7+ hours (memory consolidation is real), re-run the quizzes you got wrong, say answers out loud, skim the cheat sheet, lay out your clothes and ID.
- **Do**: practise writing one full REST controller and one stream pipeline by hand, on paper.
- **Skip**: learning a brand-new framework today. Breadth crammed on day four will not stick, and it'll shake your confidence in what you do know.
- **Skip**: reading 50 new articles. Re-read your own notes instead — retrieval practice beats more input.
- **In the interview**: pause before answering, think out loud, admit gaps honestly, and ask for a hint if you're stuck on code.
- **If you blank on syntax**: say "I'd look up the exact method name, but the approach is..." — that's a perfectly good answer.

Code

The last-24-hours schedule

```
MORNING (3h)
 09:00  Re-run every quiz. Write down the ones you got wrong - TWICE.
 10:30  Flashcard sweep: Day 1 + Day 2 (fast, 30 min)
 11:00  Flashcard sweep: Day 3 + Day 4 (fast, 45 min)
 12:00  Lunch + a walk (yes, seriously)

AFTERNOON (3h)
 13:30  Write a full @RestController by hand on paper, from memory.
 14:00  Write 5 stream pipelines by hand (filter/map/sorted/collect,
        groupingBy, joining, flatMap, reduce).
 14:45  Write a HashMap from scratch? No. But EXPLAIN it out loud.
 15:30  Rehearse your project pitch out loud 3 times. Time it: 45 seconds.
 16:00  Write your two STAR stories.

EVENING (1.5h)
 18:00  Skim the cheat sheet once.
 19:00  Mock interview: have a friend ask you 10 random questions.
 20:00  Pack your bag, lay out clothes, confirm logistics.
 22:00  Sleep. Put the notes down - last-minute cramming hurts recall.

DAY OF
 Wake early, light exercise, breakfast.
 15 min: re-run the output traps only.
 Arrive 10 minutes early (or test your camera/mic the night before).
 First 2 minutes: breathe, smile, and remember they WANT you to be good -
 they wouldn't be interviewing you otherwise.
```

The cheat sheet — last page to read

```
STRINGS     literals pooled; new String() = new object; always .equals()
INTEGER     cache -128..127; never == on wrappers
EXCEPTIONS  checked = compile-time; finally overrides return
COLLECTIONS ArrayList default; HashSet O(1); TreeSet sorted; HashMap 0.75/8
GENERICS    erased at runtime; PECS (producer extends, consumer super)
STREAMS     source -> intermediate (lazy) -> terminal; consumed once
THREADS     start() not run(); sleep holds lock, wait releases it
VOLATILE    visibility yes, atomicity no
SPRING      IoC container; constructor injection; singleton beans
BOOT        auto-config backs off for your beans; starters; profiles
REST        DTOs, @Valid, @RestControllerAdvice, 201 for POST
JPA         Lazy @ManyToOne; JOIN FETCH for N+1; @Transactional on service
SQL         WHERE before GROUP BY, HAVING after; index trade-offs
SECURITY    BCrypt passwords; JWT = signed, readable, stateless
INTERVIEW   3-beat answers; STAR stories; 2 questions of your own

IF YOU BLANK: "I'd look up the exact API, but the approach is..."
That sentence has saved more interviews than any memorised fact.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Four days ago this all felt forgotten. You now have the language back, the framework story straight, and a project pitch that lands. Sleep tonight — recall beats cramming."

Flashcards

Q. What's the single best use of your last day?
A. Retrieval practice: re-run the quizzes and flashcards you got wrong, and say answers out loud. Reading new material makes you feel prepared without actually improving recall.

Q. What should you NOT do on the last day?
A. Start a brand-new topic or framework. It won't stick, and it undermines confidence in what you already know.

Q. What should you hand-write on paper?
A. One full REST controller, five stream pipelines, and your project pitch. Hand-writing exposes the gaps an IDE hides.

Q. What do you say when you blank on syntax?
A. "I'd look up the exact method name, but the approach is..." — then describe the approach. Interviewers care about the reasoning, not the spelling.

Q. Why is sleep actually part of the plan?
A. Memory consolidation happens during sleep. Cramming all night reliably produces worse recall than a rested brain.

Q. What's the fastest credibility-builder in a technical round?
A. Admitting a gap honestly and then reasoning toward an answer. It shows both self-awareness and how you think under pressure.

Quiz

Q1. It's the night before. What's the highest-value activity?

[ ] A. Read 20 new articles on microservices
[x] B. Re-run the quizzes and flashcards you got wrong, and rehearse out loud
[ ] C. Watch a 6-hour Spring tutorial
[ ] D. Re-read all four days from the top

Why: Retrieval practice (testing yourself) beats re-reading by a wide margin. Target your weak spots, then stop and sleep.

Q2. You're given a coding problem and freeze. Best move?

[ ] A. Stay silent and stare at the board
[x] B. Talk through your approach, then ask for a hint if needed
[ ] C. Start typing immediately
[ ] D. Say 'I can't do this one'

Why: Interviewers grade the process: clarify requirements, state a brute-force approach, then optimise. Thinking out loud turns a blank into a conversation.

Q3. You don't remember the exact method name for something. What do you do?

[ ] A. Guess and hope
[x] B. Say you'd look it up, then describe the approach clearly
[ ] C. Skip the question
[ ] D. Pretend it doesn't exist

Why: Nobody codes from memory in real life. Knowing the concept and the approach is what matters — the exact spelling is what the IDE is for.